

—— // ZTERMINAL // ——

zterminal

v0.29.9

THE COMPANION

the grid · the renderer · the extensions

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The Pitch	4
Chapter 2 — Lineage and Honesty	6
Chapter 3 — The Grid	8
Chapter 4 — The Renderer	11
Chapter 5 — Damage Tracking and Frame Scheduling	13
Chapter 6 — Escape Sequence Support	16
Chapter 7 — The Configuration Model	18
Chapter 8 — Window	21
Chapter 9 — Font	23
Chapter 10 — Grapheme Clusters, Shaping, and Ligatures	25
Chapter 11 — Colors	27
Chapter 12 — Cursor, Bell, Selection, Scrolling	30
Chapter 13 — Terminal and Mouse	32
Chapter 14 — Hints	33
Chapter 15 — Keyboard and Bindings	35
Chapter 16 — Debug	37
Chapter 17 — Vi Mode and Search	38
Chapter 18 — The Control Panel	40
Chapter 19 — The Settings Tab	43
Chapter 20 — Profiles	45
Chapter 21 — The Dashboard	46
Chapter 22 — Command and History Palettes	48
Chapter 23 — Native Splits and Layouts	50
Chapter 24 — Exposé, MRU, and the Quake Dropdown	52

Chapter 25 — Inline Graphics	53
Chapter 26 — Shell Integration	56
Chapter 27 — The tmux Client	58
Chapter 28 — Session Restore and Persistence	60
Chapter 29 — tmux Broadcast, Search, and Process Tree	63
Chapter 30 — Env Vars, Snippets, Triggers, Recent Dirs	64
Chapter 31 — IPC	66
Chapter 32 — The Inventions	68
Chapter 33 — The Process Model	70
Chapter 34 — Performance	72
Chapter 35 — Selection, Clipboard, and Copy	75
Chapter 36 — Security and Trust Boundaries	77
Chapter 37 — Terminfo and Capability Advertisement	79
Chapter 38 — Building and Installing	81
Chapter 39 — Where It Sits in the Stack	83
Appendix A — Command-Line Reference	85
Appendix B — Configuration Reference	86
Appendix C — Default Keybindings	89
Appendix D — Escape Sequence Support	91
Appendix E — The ~/.zterminal Directory	94
Appendix F — Glossary	95
Appendix G — Troubleshooting	97
Appendix H — Worked Configurations	99
Appendix I — The Escape Sequence Cookbook	102
Appendix J — Environment Variables	104

Before You Begin

This is the book about `zterminal` — the one you read in a chair, front to back, to understand not only *what* `zterminal` does but *why* it is shaped the way it is. It is not a man page. The man pages — `zterminal(1)`, `zterminal-msg(1)`, `zterminal(5)`, `zterminal-bindings(5)`, and `zterminal-escapes(7)` — enumerate every flag, every configuration field, every default binding, and every escape sequence the emulator answers, and you reach for them when you need one fact fast. This book is the other thing: the continuous account of the program, from the byte that arrives on the PTY to the pixel that lands on the screen, and of the layer of features built on top of that core that make `zterminal` a terminal *environment* rather than a terminal *window*. It is also exhaustive where the man pages are terse: every setting in `zterminal.toml` is walked here with its type, its values, its default, and its effect, and every feature in the extension layer gets a chapter or a section of its own.

`zterminal` is a fast, GPU-accelerated, cross-platform terminal emulator and part of the MenkeTechnologies stack. It runs on Linux, BSD, macOS, and Windows. Two sentences describe it accurately and together explain everything that follows. The first: its core is a faithful, high-performance terminal emulator in the Alacritty lineage — a `vte`-based xterm-compatible VT parser feeding an OpenGL ES 2.0 glyph-atlas renderer, with truecolor, scrollback, a built-in vi mode, search, and hints. The second: on top of that minimal, fast core MenkeTechnologies builds an integrated extension suite — an in-process control-panel GUI built entirely from the shared `zgui-core` component library, native i3-style tiling, inline GPU graphics in three protocols, OSC 133 shell integration, a direct tmux wire-protocol client that *live-edits* a running tmux server, profiles, snippets, output triggers, a telemetry dashboard, and more — none of which the core emulator it descends from has, and all of which ship inside the one binary.

This book is honest about that division, because the honesty is the point. The emulator core is not reinvented from a specification; it is built on the Alacritty terminal lineage, the same way the other books in this series credit their foundations — vimlrs ports Neovim’s eval engine, zshrs ports zsh’s C, `zterminal` builds on Alacritty’s VT and renderer. The repository’s own inventions file is explicit: “`zterminal` is an Alacritty derivative; base VT/grid/search/ hints are **not** claimed.” The configuration format, the escape-sequence support, the default keybindings, and the vi-and-search model are that inherited core, documented faithfully and completely. The original work — the work that makes `zterminal` a MenkeTechnologies product rather than a rebuild — is the extension layer, and the book devotes its second half to it, ending with a chapter that walks every documented invention with its source basis.

Everything here is grounded in the program and its documentation. Every configuration snippet is the real TOML the emulator parses; every escape code is one the parser actually handles, at the support level the escape-sequence table records; every keybinding is a default the binding table ships; and every claim about the control panel and the inventions is grounded in the running app or in a cited source file. `zterminal` is young and moves fast — its version is `v0.29.9` at the time of writing — and where a number or a behaviour is provisional this book marks it rather than rounding up. `zterminal` is the work of MenkeTechnologies, built on the foundation of the Alacritty authors and the VT and xterm tradition before them, and this book is grounded in that code.

Let’s start.

Chapter 1 – The Pitch

A terminal emulator is, on paper, a solved problem. The VT100 is older than most of the people writing software today; the escape sequences that move a cursor and set a colour were standardized while mainframes still filled rooms; and every operating system ships a terminal that works. The temptation, looking at that, is to conclude that a new terminal emulator can only ever be a marginal thing — a little faster, a little prettier, a slightly different set of defaults. For most new terminals that conclusion is correct, and the MenkeTechnologies project bar says so bluntly: a faster clone of an existing tool is not worth building. “World’s fastest at a duplicate” is dead on arrival.

zterminal is not pitched as a faster terminal. Its core *is* fast — that is inherited, and the inheritance is deliberate — but speed is the floor, not the pitch. The pitch is the integration. Consider what a working terminal user actually assembles around a bare emulator: a multiplexer for splits and sessions (tmux), a separate tool or protocol for inline images (kitty, or a sixel viewer), a shell-integration script for prompt marks (the iTerm2 or FinalTerm snippets), a session-restore plugin (tmux-resurrect), a dropdown-terminal daemon (a Quake-style hotkey window), an output-trigger tool, and a pile of configuration spread across all of them. Each of those is a separate program, with its own config language, its own failure modes, and its own seams where it does not quite talk to the others.

zterminal’s argument is that this assembly is itself the problem, and that the pieces belong inside the emulator, sharing one process, one config file, one event loop, and one GPU surface. It tiles natively, so splits are the emulator’s own panes rather than a multiplexer’s redraw of a single grid. It renders images itself, in all three common protocols, as GPU textures blitted over the cells. It reads the tmux server’s wire protocol directly — and goes further, live- editing that server’s options, paste buffers, and keybindings from its own GUI, which no other terminal does. It parses OSC 133 prompt marks out of the byte stream, so prompt-jumping and output-selection work without a plugin. It snapshots the whole terminal-plus-tmux state into switchable profiles. And it puts all of it behind a single in-process control panel and a searchable command palette, so none of it requires memorizing a keybinding.

That is the world’s-first-and-fastest test applied honestly. The *fastest* leg is the inherited core: a GPU renderer and a tight VT parser that already compete with the fastest emulators. The *first* leg is the combination and several of its pieces individually — a terminal that live-edits a running tmux server, that stores tmux state in profiles, that ships a real telemetry dashboard. The repository’s inventions file claims those three as outright firsts, and the broader stack’s inventions catalogue lists more than a dozen additional candidates. No single program in the terminal ecosystem occupies the same intersection. The rest of this book is the detailed account of both legs: the core it inherits and documents faithfully, and the layer it adds and owns.

The assembly tax, itemized

It is worth making the “assembly” argument concrete, because the cost it names is easy to wave at and easy to underestimate. A terminal power user’s real environment is not one program; it is a stack of them, each with its own configuration language, its own update cycle, and its own seams:

- a **multiplexer** (tmux) for splits, sessions, and persistence — its own `.tmux.conf`, its own key prefix, its own copy-mode;
- an **image protocol or viewer** (a sixel tool, `kitten icat`) for inline graphics — which the multiplexer then filters unless passthrough is configured;
- a **shell-integration script** (the iTerm2 or FinalTerm snippets) for prompt marks — sourced per shell, guarded per terminal;

- a **session-restore plugin** (tmux-resurrect, tmux-continuum) — its own save format, its own restore whitelist;
- a **dropdown-terminal daemon** for a Quake-style hotkey window;
- an **output-trigger tool** for reacting to matched text;
- and a **pile of glue** — aliases, keybindings, environment exports — holding the above together.

Each item is a separate failure mode and a separate place the pieces do not quite talk. The image tool's escapes get eaten by the multiplexer. The restore plugin does not know about the shell marks. The dropdown daemon does not share the multiplexer's sessions. zterminal's thesis is that this list is not a rich ecosystem to be celebrated but a *tax* to be collapsed: every item above has a chapter in this book's second half where it is instead a feature of the one process, sharing the one config file and the one event loop. The pitch is not that any single one of these is novel in isolation — several are not — but that assembling them out of separate programs is the problem, and integrating them is the product.

What zterminal is, precisely

zterminal is a Rust terminal emulator. It opens a window, allocates a pseudo-terminal, spawns your shell into it, and runs an event loop that does three things forever: read bytes from the PTY and feed them to a VT parser that mutates a grid of cells; read input events from the window and translate them to bytes written back to the PTY (or to actions); and, when the grid is dirty, upload the changed cells to the GPU and draw a frame. That loop is the whole of a terminal emulator, and zterminal's core runs it the way the Alacritty lineage does — minimally, with the parsing in `vte` and the drawing in OpenGL ES 2.0.

Onto that loop zterminal grafts the extension suite. The grafts are not afterthoughts bolted to the side; they sit in the same loop. The split tree owns multiple PTYs and scissors each pane into a sub-viewport of the same GL surface. A single scanner sits *ahead* of the VT parser, peeling graphics and OSC 133 sequences out of the same byte stream before the parser sees them. The tmux client opens a second socket to the tmux server and folds its events into the same loop. The control panel is an embedded WebView, served over a custom `zterminal://` protocol and built only from the shared `zgui-core` component library, that reads and writes the same live configuration the emulator hot-reloads. One process, one loop, one config.

Who it is for

zterminal is built for someone who lives in the terminal — specifically, for the MenkeTechnologies development workflow, where the shell is the IDE, the build system, the deploy tool, and the orchestrator, and where many synchronized shell instances run at once. Anything that slows the terminal slows that whole pipeline, which is why the core is the fast inherited one and why the extensions are integrated rather than shelled out to subprocesses. The terminal is the host for the rest of the stack — `zshrs` runs inside it, `zpw` paints inside it, the GUI applications embed the same shared terminal component — and zterminal is the window all of it lives in. Chapter 39 returns to that placement; the chapters between here and there are the terminal itself, in full.

Chapter 2 – Lineage and Honesty

Every faithful book in this series opens its second chapter by naming its foundation, because faithfulness to a foundation is only meaningful if you say what the foundation is. vimlrs names Neovim's `eval/*` tree and calls the C source its specification. zshrs names zsh. zterminal's foundation is the Alacritty terminal lineage, and this chapter says what that means, what is inherited from it, and what is not — because the rest of the book's originality claims only hold if the inheritance is stated plainly first.

The evidence of descent

You do not have to take the lineage on assertion; it is legible in the shipped artifacts. The configuration file is TOML with exactly the table structure — `[general]`, `[window]`, `[scrolling]`, `[font]`, `[colors]`, `[bell]`, `[selection]`, `[cursor]`, `[terminal]`, `[mouse]`, `[hints]`, `[keyboard]`, `[debug]` — that the Alacritty lineage uses, down to field names like `option_as_alt`, `draw_bold_text_with_bright_colors`, and `ipc_socket`. The escape-sequence support table in `zterminal-escapes(7)` is the Alacritty support matrix: the same `IMPLEMENTED` / `PARTIAL` / `REJECTED` vocabulary, the same charset caveat, the same set of private modes under `CSI ? h`. The man pages still credit the upstream maintainers in their `MAINTAINERS` sections. The renderer is OpenGL ES 2.0 with a glyph atlas; the parser is `vte`. The core crate is named `zterminal_core`. This is not a terminal written from the VT spec; it is a terminal carried forward from a specific, excellent open-source lineage, and the honest framing — the framing this series always uses — is that the core is *ported and inherited*, not invented.

That matters for the project's own rules. The MenkeTechnologies bar kills “X but faster” and “X but in Rust” outright; a faster Alacritty would be the awkrs trap, dead on the first test. zterminal survives that test only because of the second half of the book — the extension layer that does not exist upstream. So the lineage chapter is not a hedge; it is the setup for the originality claim. You cannot credibly say “this is the first terminal that live-edits a tmux server” while pretending the VT core underneath sprang from nothing. Name the foundation, then show what was built on it. The repository's own `docs/INVENTIONS.md` does exactly this, opening with the disclaimer that “base VT/grid/search/hints are **not** claimed.”

What faithful porting means here

“Inherited” is a precise word in this series, and it is worth saying what it does and does not mean for zterminal. It does not mean a fork left to drift. It means the emulator core — the VT parser's escape handling, the grid's semantics, the renderer's glyph-atlas architecture, the configuration schema, the vi-and-search model — is carried forward from the Alacritty lineage as the *specification* to be matched, the same way vimlrs treats Neovim's C as the spec and zshrs treats zsh's. The support table's exactness (Chapter 6) is the evidence: a terminal that half-implemented its inheritance would not be able to state, sequence by sequence and parameter by parameter, precisely what it handles and what it rejects. That precision is only possible because the behaviour is faithful to a known reference rather than approximated from a vague memory of “what xterm does.”

The discipline has a direct payoff for the reader of this book. Because the core is faithful, the first fifteen chapters can document it *as the reference documents it* — the same table names, the same field names, the same escape statuses — and a fact learned here transfers to the broader VT and xterm tradition rather than being a zterminal-only quirk. And because the inheritance is stated rather than hidden, the originality claims of the second half can be sharp: every invention names its basis (Chapter 32), and none of them has to pretend

the ground it stands on was also invented. Faithful porting is not a smaller ambition than rewriting; it is the thing that lets the ambition — the extension layer — rest on a floor that is known to be correct.

What is inherited

These are inherited from the Alacritty lineage and documented faithfully in Chapters 3–17. They are not zterminal's inventions, and the book does not claim them as such: the `vte`-based VT parser and xterm escape handling; the OpenGL ES 2.0 glyph-atlas renderer and built-in box drawing; the TOML configuration format and every table in it; the built-in vi mode and search; terminal hints; the keyboard/mouse binding model; and the `zt msg` IPC mechanism with its migrate subcommand.

What is original

These are the MenkeTechnologies extension layer, documented in Chapters 18–32 and catalogued with their source basis in Chapter 32. They are zterminal's own work, integrated into the one binary: the in-process `zgui-core` control panel and its managers; the live tmux-server editor (options, buffers, keybindings); profiles; the telemetry dashboard; the command, shell-history, and prompt palettes; native i3-style tiling with saveable layouts; Exposé and cross-pane search/process tree; inline GPU graphics in three protocols plus a config-set background image; OSC 133 shell integration with failed-command gutter marks; native tmux session save/restore and cross-session broadcast; a GUI env-var editor that hot-injects into running shells; per-pane output triggers; snippets; a recent-directories tracker; the bundled `zterminal-icat`; and the glassy overlay compositing.

Hold those two lists in mind. The chapters do not re-litigate the division each time, but it is always there: the first half documents a faithful, fast inherited core, exhaustively; the second half documents the original layer, exhaustively.

Chapter 3 – The Grid

Strip every extension away and a terminal emulator is a machine that maintains a two-dimensional grid of character cells and mutates it in response to a byte stream. Understanding that machine is the prerequisite for everything zterminal layers on top, because every extension is ultimately expressed as either a change to the grid or a thing drawn over it.

Cells, not characters

Each cell in the grid holds a character (a Unicode scalar, possibly a zero-width combiner attached to its neighbour), a foreground colour, a background colour, and a set of attributes — bold, italic, underline, strike, dim, inverse, hidden. The grid has a fixed number of columns and rows, derived from the window's pixel size divided by the cell size, which is itself derived from the font metrics (Chapter 4). When the window resizes, the column and row counts change and the grid reflows; `window.dimensions` can pin an initial column/row count, and `window.resize_increments` can constrain resizing to whole cells so the grid never has a partial row or column of pixels.

The parser

Bytes arriving from the PTY are not all characters to print; interleaved among them are control sequences — the escape codes that move the cursor, change colour, clear a region, scroll, switch to the alternate screen, and a hundred other things. zterminal's core uses the `vte` parser to tokenize this stream into printable characters and parsed control sequences, and a terminal state machine applies each one to the grid. The four families — ESC codes, CSI sequences, OSC sequences, and the narrow DCS support — are the subject of Chapter 6 and Appendix D; the point here is that the parser is faithful and that its faithfulness is what makes vim, tmux, htop, and arbitrary ncurses applications behave correctly.

The two screens and the scrollbar

A terminal has a primary screen and an alternate screen. The primary screen scrolls: lines that move off the top are pushed into a scrollbar buffer, sized by `scrolling.history` (default `10000`, capped at `100000`, `0` to disable). The alternate screen does not scroll — it is the fixed-size canvas full-screen applications paint on, and switching to it (private mode 1049) saves the primary screen and cursor, restored when the application exits. The scrollbar is what vi mode (Chapter 17) navigates, what the OSC 133 marks (Chapter 26) anchor to by absolute line number, and what cross-pane search (Chapter 29) greps.

Truecolor and palettes

Each cell's colour can come from three sources: the 16-colour ANSI palette (`colors.normal`, `colors.bright`), the 256-colour indexed palette (the first 16 being the ANSI colours, then a 6×6×6 cube and a greyscale ramp, with overrides via `colors.indexed_colors`), or a direct 24-bit RGB value from an SGR truecolor sequence. zterminal carries all three; truecolor is why a modern colour scheme renders exactly rather than quantized to 256 steps. The default palette is a base16 scheme — background `#181818`, foreground

`#d8d8d8` — and Chapter 11 covers replacing it field by field, while Chapter 19 covers replacing it through the control panel's theme picker.

Width, combiners, and the wide-cell problem

Not every character occupies one cell, and the grid has to know which do not. A CJK ideograph, a wide emoji, and a handful of symbols are *double-width*: they claim two horizontal cells, the second of which is a spacer that holds no glyph of its own. Zero-width combining marks are the opposite — an accent or a variation selector that attaches to the preceding cell's character rather than advancing the cursor. The grid resolves each incoming scalar to a width before placing it, using the East-Asian-width rules, so that a full-width character advances the cursor by two columns and a combiner advances it by none. Getting this wrong is not cosmetic: a single mis-measured wide glyph shifts every following cell on the line, and a TUI that assumed its own layout desynchronizes from the terminal's. The faithfulness of this width accounting is part of what the inherited core buys, and it is why an emoji-laden prompt or a Japanese `man` page lines up under `zterminal`.

Reflow

When the window widens or narrows, the column count changes, and the terminal must decide what happens to lines that were longer or shorter than the new width. The primary screen *reflows*: a logical line that had been wrapped across several rows is rejoined and re-wrapped to the new width, so widening the window un-breaks paragraphs rather than leaving ragged fragments, and narrowing re-breaks them cleanly. The alternate screen does not reflow — it is a fixed canvas the application owns, and on resize the application is simply told the new size and repaints. `window.resize_increments` (Chapter 8) constrains resizing to whole cells so a reflow never has to contend with a partial column of pixels. Reflow is also why the OSC 133 marks of Chapter 26 anchor to *absolute line numbers* rather than screen rows: a mark pinned to a row would drift the moment the buffer re-wrapped, while a mark pinned to a logical line survives the reflow intact.

Erasing, scrolling, and the scroll region

Between printing characters and moving the cursor, the third thing a byte stream does to the grid is clear and shift regions of it, and these operations are worth naming because they are what a full-screen application spends most of its sequences on. Erasing comes in three granularities: erase-in-line (`CSI K`, Chapter 6) clears from the cursor to the end of the line, the start, or the whole line; erase-in-display (`CSI J`) does the same for the screen and can additionally clear the scrollbar; and erase-characters (`CSI X`) blanks a count of cells in place without moving anything. Scrolling shifts whole rows: when the cursor is on the bottom line and a newline arrives, the top line moves into the scrollbar and every row shifts up one (Chapter 3's primary-screen behaviour). The *scroll region* (`CSI r`) is the refinement that makes fixed headers and footers possible: it sets a top and bottom margin, and thereafter scrolling happens only *within* those margins, leaving the rows outside them untouched. This is how a TUI keeps a status bar pinned to the bottom while the content above it scrolls — it sets a scroll region that excludes the last line, and the terminal honours it. Origin mode (above) then lets the application address rows relative to that region. These operations are the grid's verbs; printing is only the most visible of them.

The cursor and its saved state

The grid has a cursor — a row/column position where the next printed character lands — and the cursor is more than a coordinate. It carries the *current pen*: the foreground colour, background colour, and attribute set (Chapter 3's cell fields) that every printed character inherits until an SGR sequence changes them. It also carries two mode bits that change how printing behaves at the edges of the grid, discussed below. `ESC 7` (save cursor) and `ESC 8` (restore cursor) — implemented, Chapter 6 — push and pop the whole of this state:

not just where the cursor is, but the pen and the character set it was using. This is why a program can save the cursor, move away to draw something, and restore, and find its colours and position exactly as it left them. The cursor is the grid's write head and its style register at once.

Tab stops

The grid maintains a set of *tab stops* — columns the cursor jumps to on a horizontal tab (`\t`). By default a stop sits every eight columns, the ancient teletype convention, but the stops are individually settable: `ESC H` (or `CSI g` with a parameter, Chapter 6) sets a stop at the cursor's column, and `CSI g` clears one or all. Tab stops are grid state, not display state, which is why a program that lays out columns with tabs — the output of `ls -C`, a `column`-piped table — depends on the terminal and the program agreeing on where the stops are. `zterminal` carries the standard eight-column default and honours the set/clear sequences so that tabular output aligns.

Origin mode, autowrap, and the edges

Two mode bits decide what happens at the grid's boundaries, and both are private modes the application toggles (Chapter 6). **Autowrap** (private mode 7, on by default) decides what happens when a character is printed in the last column: with autowrap on, the cursor wraps to the start of the next line and the line is marked *wrapped* so reflow (below) can rejoin it; with autowrap off, further characters overprint the last column in place. **Origin mode** (private mode 6) decides whether cursor-addressing coordinates are measured from the top of the screen or from the top of the current scroll region (`CSI r`, Chapter 6) — so a program that has set a scroll region and turned on origin mode can address rows relative to that region without arithmetic. These two bits, small as they are, are why full-screen applications that set margins and print into them land their text where they intend; getting either wrong is a class of “the bottom line is in the wrong place” bug, and the inherited core carries both faithfully.

Chapter 4 – The Renderer

The grid is data; the renderer turns it into light. zterminal's core renderer is OpenGL ES 2.0, and its central data structure is a glyph atlas — the technique that lets a terminal redraw tens of thousands of cells per frame without the CPU ever touching a pixel.

The glyph atlas

Rasterizing a glyph — turning a font outline into a grid of coverage values — is expensive, and a terminal would be unusably slow if it rasterized every glyph every frame. The glyph atlas solves this by rasterizing each glyph *once*, the first time it is needed, and packing the resulting bitmap into a large GPU texture. After that, drawing a cell is just drawing a textured quad: the renderer emits two triangles at the cell's screen coordinates, textured from the glyph's rectangle in the atlas, tinted by the cell's foreground colour, over a quad of the cell's background colour. A frame is thousands of such quads in a few draw calls, and the GPU does the work the CPU used to. This architecture makes the core fast, and it is inherited from the Alacritty lineage.

Cell metrics and the grid's pixel size

The font's metrics define the cell. A monospace font has a fixed advance width and a line height; the cell width is the advance plus `font.offset.x` (letter spacing), the cell height is the line height plus `font.offset.y` (line spacing). The grid's column count is the window's content width divided by the cell width; the row count is the content height divided by the cell height. `font.glyph_offset` nudges where the glyph sits inside its cell. These numbers are why changing the font size reflows the whole grid: a larger cell means fewer columns and rows in the same window.

Built-in box drawing

Fonts disagree about box-drawing and block characters — the lines, corners, shades, and powerline separators in the Unicode ranges `U+2500–U+259F`, `U+1FB00–U+1FB3B`, and `U+E0B0–U+E0B3`. A font that draws them a pixel short leaves gaps in a TUI's borders. zterminal's core can draw these itself with a built-in geometric font, so lines meet exactly regardless of the text font; `font.builtin_box_drawing` controls it and defaults to `true`. This is what makes a `zpw` status bar or a `tmux` border render seamlessly.

Damage, frames, and synchronized output

A terminal does not redraw when nothing changed. The renderer tracks *damage* — the cells changed since the last frame — and `debug.highlight_damage` visualizes it. Synchronized output (private mode 2026) lets an application say “hold the frame until I am done,” so a complex repaint shows up as one atomic update rather than a tear. `debug.render_timer` overlays per-frame draw time, and `debug.renderer` forces a backend (`glsl3`, `gles2`, `gles2pure`) when the automatic choice misbehaves. These are troubleshooting tools, documented as unstable, but they are the windows into the frame loop.

Compositing: transparency, blur, background image, overlays

The renderer composites, and that composite path is the hook for several extensions. `window.opacity` makes the background translucent; `window.blur` asks the compositor (macOS and KDE Wayland) to blur what shows through; `colors.transparent_background_colors` extends opacity from the default background to every cell's background. The inline-graphics extension (Chapter 25) draws its images and its config-set background image as textured quads in this same pass, ordered by a z-index that can place an image behind the text. And the control-panel and palette WebViews (Chapter 18) composite *over* the live GL terminal as translucent glass, by driving the native window's alpha — the inherited renderer keeps drawing underneath. The background image being just another texture in the existing compositing pass is the clearest small example of the whole program's method: original features built on the inherited core's machinery.

The atlas is finite: packing and eviction

The glyph atlas is a fixed-size GPU texture, which raises a question the happy path hides: what happens when more distinct glyphs are seen than the atlas can hold? Each newly-rasterized glyph is packed into free space in the texture — the renderer keeps a simple allocator that finds a rectangle for the new bitmap and records where it went — and in steady state the atlas fills with the few hundred glyphs a session actually uses and then stops growing, because every subsequent character is already resident. The pathological case named in Chapter 34 is an application that cycles through an unbounded parade of *distinct* glyphs — an emoji rain, a font-showcase tool — faster than the atlas has room, at which point the renderer must evict resident glyphs to make space and re-rasterize them if they reappear. That is the one workload where the amortization of Chapter 4 breaks down, and it is vanishingly rare because real terminal text reuses a tiny alphabet relentlessly. The design bets on that reuse, and the bet almost always wins.

What one draw call actually contains

It helps to be concrete about the “few draw calls” claim. The renderer does not issue one draw per cell — that would be tens of thousands of GPU calls per frame, the exact overhead the architecture exists to avoid. Instead it builds, per frame, a single batch describing every damaged cell: for each, the screen rectangle, the atlas rectangle of its glyph, its foreground tint, and its background colour. That batch is uploaded once and drawn in a handful of GPU calls — the backgrounds as one set of quads, the glyphs as another textured from the atlas. The CPU's job per frame is therefore to *assemble a list*, not to paint pixels, and the GPU's job is to rasterize thousands of pre-described quads in parallel, which is precisely the work GPUs are built for. This is why the renderer scales to a large, dense grid without the frame time growing with the cell count the way a CPU blitter's would.

Chapter 5 – Damage Tracking and Frame Scheduling

Chapter 4 introduced the renderer and named damage in a paragraph. This chapter takes that paragraph apart, because the discipline of *not drawing* is where a terminal spends most of its cleverness, and because the same machinery — a set of dirty cells, a frame clock, and a small pile of timers — underlies the performance floor of Chapter 34 and several of the extensions above it. All of it is inherited from the Alacrity lineage, and none of it is claimed as zterminal's; it is described here because a reader who understands the frame loop understands why the terminal stays responsive under a feature suite that, done naïvely, would stall it.

What “damage” means, precisely

Damage is the set of cells that changed since the last frame the GPU presented. The word matters: the renderer does not ask “what does the grid look like now,” it asks “what is *different* from what is already on the screen,” and it draws only the difference. In the common case that difference is tiny. A shell sitting at a prompt with a blinking cursor damages exactly the cells the cursor occupies. A `tail -f` scrolling one line damages one row and the rows the scroll shifted. A full-screen editor repainting its status line damages that line and nothing else. The renderer tracks this as line-granular dirty ranges — which rows changed, and within a changed row which span of columns — so a redraw uploads only the altered cells to the glyph-atlas draw path of Chapter 4 rather than re-emitting the whole grid every frame.

Two events force *full* damage, where every cell is considered dirty: a resize, because the grid's dimensions changed and every screen position may now hold a different cell (Chapter 3's reflow), and a switch between the primary and alternate screens (private mode 1049), because the visible contents are wholesale replaced. A colour-scheme change through the control panel (Chapter 19) is a third — re-theming re-tints every cell, so the whole grid is redrawn once. These are rare and allowed to be expensive; the frequent events are cheap. That asymmetry — the frequent case cheap, the rare case dear — is the same principle Chapter 34 identifies in the read loop, and it is not a coincidence that the read half and the draw half of the loop are built on the same bet.

`debug.highlight_damage` (Chapter 16) tints the regions the renderer actually redraws, and running it while a TUI is live is the single most instructive thing in the book. Start `htop`, or scroll a `man` page, or let a prompt blink, and watch how little of the screen lights up: the terminal you thought was “redrawing” is touching a few hundred cells out of tens of thousands. The debug flag is unstable API (Chapter 16), but the behaviour it visualizes is the load-bearing one.

Coalescing: many writes, one frame

The PTY does not deliver its bytes on the display's schedule. A program can emit a thousand tiny writes — a progress bar redrawing its percentage, a log streaming lines — in the span of a single display refresh. If each write triggered a draw, the terminal would try to present a thousand frames the monitor cannot show, burn the GPU doing it, and fall behind. The frame scheduler refuses that. Bytes read from the PTY mutate the grid and accumulate damage, but the *draw* is deferred: the scheduler coalesces all the damage that arrived since the last presented frame and draws it once, at the display's refresh cadence. A thousand writes in one frame's worth of time cost one frame, not a thousand. The visible consequence is that a firehose of output

looks smooth rather than frantic, and the invisible consequence is that the CPU and GPU are doing a bounded amount of work per second no matter how noisy the program on the other end of the PTY is.

This is why the traditional torture test — `cat` an enormous file — is bounded by how fast the grid can be mutated and one frame drawn per refresh, not by how many times the screen “could” have changed. Most of those intermediate grid states are never drawn at all; they are overwritten by the next batch before a frame is due. The terminal is, in effect, sampling the grid at the refresh rate, and the sample is all the eye could have seen anyway.

Synchronized output: handing the batch decision to the application

Coalescing is the terminal’s own decision about when to draw. Synchronized output (private mode 2026, Chapters 6 and 35) hands a piece of that decision to the application. Between a `CSI ? 2026 h` (begin) and a `CSI ? 2026 l` (end) the terminal holds the frame: it keeps applying the byte stream to the grid but does not present a frame until the end mark arrives, so a complex multi-sequence repaint lands atomically instead of tearing through several partial frames. A status-bar tool that clears a region, moves the cursor a dozen times, and rewrites several coloured segments wants all of that to appear at once; without synchronization the user can catch a frame mid-repaint, with the bar half-cleared. The DCS form of the same idea is rejected in favour of the `CSI` toggle (Chapter 6, Appendix D) — one mechanism, cleanly, rather than two that half-overlap.

The relationship between coalescing and synchronization is worth stating exactly: coalescing is the terminal batching *because the display cannot show more*, while synchronization is the application batching *because a repaint is not yet coherent*. The first is always on and invisible; the second is opt-in and driven by the program. They compose — a synchronized repaint is still subject to the refresh-rate ceiling — and together they are why a well-behaved TUI never flickers under zterminal.

The timers that also wake the loop

Not every frame is provoked by PTY bytes. Several features run on their own clocks, and each is a timer that can mark damage and request a frame without any input arriving:

- **Cursor blink.** `cursor.blink_interval` (default 750 ms, Chapter 12) toggles the cursor’s visibility, damaging only the cursor cells each half-period. `cursor.blink_timeout` (default 5 s) stops the blink after a period of inactivity so an idle terminal is not waking the loop forever, and `Never/Always` versus `Off/On` decide whether an application may override the blink at all (Chapter 12).
- **The visual bell.** `bell.duration` and `bell.animation` (Chapter 12) run an easing curve over the configured flash colour; while the animation plays, each frame damages the flashed region, and when it completes the timer stops.
- **Inline-image animation.** A Kitty-protocol animated image (Chapter 25) plays its frames on a timer, damaging the cells the image covers as each frame advances — the one case where the graphics extension, not the text grid, drives the frame clock.

The important property is that all of these are *bounded* and *self-terminating*: the blink stops on timeout, the bell animation stops when the curve completes, an idle terminal with no timers pending draws nothing and consumes no frames. The event loop is not a busy render loop spinning at the refresh rate; it sleeps until either PTY bytes arrive, an input event occurs, or a timer comes due, and only then wakes, applies the change, marks damage, and — if damage exists and a frame is due — presents. An idle zterminal window is genuinely idle.

Why the extensions stay off this path

Chapter 34 makes the general argument that the extension layer is built to stay off the hot path; the frame loop is where that argument is concrete. The control panel is a `WebView`, and a `WebView` repainting on the render thread would be a disaster — so it does not (Chapter 18). It composites over the still-live GL terminal by driving the native window's alpha, which means the terminal's own frame loop keeps running underneath at its own cadence and never blocks on the panel's rendering. The pre-`vt` scanner (Chapters 25–26) adds one linear pass over the incoming bytes but marks damage exactly as the parser does, so an image or an OSC 133 mark is just more damage in the same coalesced frame. The `tmux` client's events (Chapter 27) fold into the same event loop as another wake source, not a separate render path. Every extension that touches the screen expresses itself as damage in the one frame loop, which is precisely why adding them did not cost the terminal its responsiveness — there is only ever one draw, and everything that wants to be seen has to earn its cells in it.

Chapter 6 – Escape Sequence Support

This chapter is the narrative companion to Appendix D and to `zterminal-escapes(7)`: what it means that zterminal supports a given escape sequence, why the support table is worth reading rather than assuming, and what the whole inherited support surface is. The support is inherited from the Alacritty lineage, and so is the discipline behind it.

The three statuses

Every sequence in the table carries one of three statuses. **IMPLEMENTED** means fully handled. **PARTIAL** means some parameters or modes are handled and the rest are not yet. **REJECTED** means deliberately not handled — and “rejected” is a feature, not a gap: a terminal that answers a sequence it does not really implement is worse than one that cleanly ignores it, because applications probe capabilities and adapt. The man page’s note that an **IMPLEMENTED** sequence which is “partially implemented” means “all other features were rejected” captures the philosophy: implement a thing fully or reject the remainder cleanly, never half-answer.

The four families

- **ESC codes** — `ESC 7/ESC 8` (save/restore cursor), `ESC D/ESC M` (index/reverse index), `ESC E` (next line), `ESC H` (tab set), `ESC =/ESC >` (keypad mode), `ESC # 8` (DEC alignment test), `ESC (/)*/+` (charset selection, charsets B and O only), and `ESC Z` (identify). All implemented.
- **CSI sequences** — the large `ESC [` family: cursor motion (`CSI A–CSI H`, `CSI a–CSI f`), erase (`CSI J`, `CSI K`, `CSI X`), insert/delete (`CSI L`, `CSI M`, `CSI P`, `CSI @`), scroll (`CSI S`, `CSI T`), the SGR colour- and- attribute setter (`CSI m`), device status (`CSI n`), tab control (`CSI g`), scroll regions (`CSI r`), cursor save/restore (`CSI s`), cursor style (`CSI SP q`), the keyboard-protocol family (`CSI u`, `CSI ? u`, `CSI = u`, `CSI < u`, `CSI > u`), and the private-mode toggles (`CSI ? h/CSI ? l`).
- **OSC sequences** — `ESC ]`: window title (`OSC 0/OSC 2`), palette set/query (`OSC 4`, `OSC 10/11/12`, and the reset forms `OSC 104/110/111/112`), hyperlinks (`OSC 8`), cursor shape (`OSC 50`, CursorShape only), and clipboard (`OSC 52`, clipboard and primary selection only, gated by `terminal.osc52`).
- **DCS** — narrow by design; the synchronized-output `DCS = s` is rejected in favour of `CSI ? 2026 h/l`.

The partials, precisely

The table is exact about its partials, and the precision is the inherited core’s most underrated quality. `CSI m` accepts SGR parameters `0–9`, `21–25`, `27–49`, `58`, `59`, `90–97`, `100–107`, and explicitly rejects `11–19` and `51–55`. `CSI h/CSI l` handle modes `4` and `20`. `CSI ? h/CSI ? l` handle the modes applications actually use: `1` (application cursor keys), `3` (132-column), `6` (origin), `7` (autowrap), `12` (cursor blink), `25` (cursor visibility), the mouse-reporting family `1000/1002/1005/1006/1007`, focus reporting `1004`, `1042`, the alternate-screen `1049`, bracketed paste `2004`, and synchronized output `2026`. `CSI t` answers window-manipulation parameters `22` and `23` and rejects the geometry-manipulation ones. When an application misbehaves under zterminal, this table is the first place to look, because it names exactly which sequence and parameter is handled and at what status; Appendix D reproduces the full matrix.

A worked example: reading an SGR sequence

The abstraction becomes concrete the moment you read a real sequence off the wire. Consider the bytes a shell prompt emits to draw bold cyan text on a dark background and then reset:

```
ESC [ 1 ; 38 ; 2 ; 117 ; 181 ; 170 m   the pen: bold, truecolor fg #75b5aa
ESC [ 48 ; 5 ; 236 m                   the pen: 256-palette bg, index 236
Hello                                   printed with the current pen
ESC [ 0 m                               reset the pen to default
```

Every one of those is a `CSI m` (SGR) sequence, the colour-and-attribute setter of this chapter, and reading it is a matter of splitting the numeric parameters on `;` and applying each. `1` sets the bold attribute. `38` introduces a foreground colour, and its sub-parameters say *how*: `38;2;r;g;b` is a 24-bit truecolor value (the `2`), here `#75b5aa`; `38;5;n` would be a 256-palette index (a `5`). `48` is the same for background — `48;5;236` selects palette index 236, a dark grey from the 6×6×6 cube (Chapter 3). `0` resets everything to the defaults of Chapter 11. The parameters zterminal accepts here are exactly the ones the partials table names — `0–9`, `21–25`, `27–49`, `58`, `59`, `90–97`, `100–107` — and a sequence carrying `11–19` or `51–55` is cleanly ignored rather than half-answered, per the REJECTED discipline above.

A worked example: a full-screen application's setup

The other place the table earns its keep is the handshake a TUI performs on startup and teardown, which is a small liturgy of the private modes:

```
ESC [ ? 1049 h   switch to the alternate screen (save the primary)
ESC [ ? 1 h      application cursor keys (arrows send SS3, not CSI)
ESC [ ? 25 l     hide the cursor while painting
ESC [ ? 1006 h   SGR mouse reporting (report clicks as CSI < ... M)
ESC [ ? 2004 h   bracketed paste (Chapter 35)
...the application runs...
ESC [ ? 2004 l   disable bracketed paste
ESC [ ? 1006 l   disable mouse reporting
ESC [ ? 25 h     show the cursor
ESC [ ? 1 l      normal cursor keys
ESC [ ? 1049 l   restore the primary screen
```

Every mode in that exchange is one the `CSI ? h/CSI ? l` partials row lists as handled (Appendix D), which is precisely why vim, htop, and any ncurses program enter cleanly and — the underrated half — *leave* cleanly, restoring your prompt and scrollbar exactly as they were. A terminal that mishandled `1049` would leave you staring at the application's last frame after it exited; one that ignored `25` would blink a cursor over the application's own. The support table is not bookkeeping; it is the contract that makes the alternate-screen round trip invisible.

Chapter 7 – The Configuration Model

zterminal runs with no configuration, but it is built to be configured, and the configuration story has one MenkeTechnologies twist on the inherited core: a canonical home under `~/.zterminal` and a GUI that edits it live. This chapter is the configuration model; Chapters 8–16 walk every table and every field, and Appendix B is the field index.

The file and its format

The configuration file is TOML — the format at toml.io — and its canonical location is `~/.zterminal/zterminal.toml`. On first run, when no config exists anywhere, zterminal seeds that file with sensible defaults, including an auto-detected installed Nerd Font as the font family so powerline and icon glyphs render immediately; if no Nerd Font is present it falls back to the platform mono font. Everything zterminal writes — config, logs, layouts, session snapshots, command history, recent directories, triggers, snippets, profiles — lives under `~/.zterminal` (Appendix E maps the whole directory).

Resolution order

The config is resolved from the first match in a fixed search order:

1. `--config-file <path>` on the command line.
2. `~/.zterminal/zterminal.toml` — the canonical file; legacy locations below are migrated here on first run.
3. `$XDG_CONFIG_HOME/MenkeTechnologies/zterminal.toml`
4. `$XDG_CONFIG_HOME/zterminal.toml`
5. `$HOME/.config/MenkeTechnologies/zterminal.toml`
6. `$HOME/.zterminal.toml`
7. `/etc/MenkeTechnologies/zterminal.toml`

On Windows the file is `%APPDATA%\zterminal\zterminal.toml`. The man page documents the inherited XDG-style search (`$XDG_CONFIG_HOME/zterminal/zterminal.toml` and friends); the `~/.zterminal` canonical home and the `MenkeTechnologies/` namespacing are the product's own convention layered on top.

Imports and environment

`general.import` is an array of additional config files, loaded in order with the importing file applied last, so a later import overrides an earlier one and the top file overrides them all. Paths are absolute (`/...`), home-relative (`~/...`), or relative to the importing file. The `[env]` table injects environment variables into every spawned process, including the shell — and the control panel's Env Vars manager (Chapter 30) both writes this table and hot-injects the export into every already-running shell.

```
[general]
import = [
  "~/.config/zterminal/base16-dark.toml",
  "~/.config/zterminal/keybindings.toml",
]

[env]
WINIT_X11_SCALE_FACTOR = "1.0"
```

Live reload, overrides, migration

`general.live_config_reload` defaults to `true`, and it is what makes the control panel feel like a settings app rather than a text editor: saved changes apply to running windows without a restart. A handful of fields are read once at window creation and are restart-only (`window.dimensions`, `window.startup_mode`, and a few others), marked in Appendix B. Two command-line mechanisms sit alongside the file: `-o`, `--option` overrides a single field for one launch (`zt -o 'cursor.style="Beam"'`), and is also how `zt msg config` pushes a live change (Chapter 31); and `zterminal migrate` upgrades an old config to the current schema, with `--dry-run`, `--skip-imports`, and `--skip-renames`. Migration is what lets the format evolve without breaking a file that worked yesterday — the durability discipline the whole stack holds to.

A complete starter config, annotated

The chapters that follow walk every table field by field; it helps to see them assembled first. The following is a realistic `~/.zterminal/zterminal.toml` that touches every major table without exotic values — the sort of file a user actually keeps — so that each later chapter's fields have a home to return to:

```

[general]
live_config_reload = true           # the control panel relies on this (Chapter 7)
import = ["~/.zterminal/theme.toml"]

[env]
TERM = "zterminal"                 # advertise the real capabilities (Chapter 37)

[window]
padding = { x = 6, y = 6 }
dynamic_padding = true
opacity = 0.92
decorations = "Full"
option_as_alt = "OnlyLeft"         # macOS: left Option is Alt (Chapter 8)

[font]
normal = { family = "JetBrainsMono Nerd Font", style = "Regular" }
size = 12.0
builtin_box_drawing = true          # crisp borders regardless of face (Chapter 4)

[cursor]
style = { shape = "Beam", blinking = "On" }
unfocused_hollow = true

[scrolling]
history = 50000                     # deep scrollbar; capped at 100000 (Chapter 12)
multiplier = 3

[selection]
save_to_clipboard = true            # every selection also hits the clipboard

[terminal]
osc52 = "OnlyCopy"                 # the security default (Chapters 13, 36)

[colors.primary]
foreground = "#d8d8d8"
background = "#181818"

[keyboard]
bindings = [
  { key = "F", mods = "Command", mode = "~Search", action = "SearchForward" },
]

```

Two things about that file are worth naming because they recur. First, nothing in it is required — delete the whole file and `zterminal` runs with the defaults of Chapters 8–16, seeding a fresh one with an auto-detected Nerd Font (Chapter 7). Second, every value in it is also reachable without editing the file at all: the Settings tab (Chapter 19) is a graphical front end to this same TOML, writing the same keys, and because `live_config_reload` is on, a change made either way lands in the running window immediately. The file and the GUI are two doors onto one configuration, and the config resolution order above decides which file both of them write.

Chapter 8 — Window

`[window]` governs the frame around the grid. It is the largest of the inherited tables, and `zterminal` adds three fields of its own to it. Each field, in full:

- `dimensions = { columns, lines }` — initial grid size in cells (not pixels). Both must be non-zero to take effect; columns must be at least 2. `{ 0, 0 }` falls back to the window manager's recommended size. Restart-only. Default `{ columns = 0, lines = 0 }`.
- `position = "None" | { x, y }` — startup window position in pixels, or `"None"` to let the window manager place it. No effect on Wayland. Default `"None"`.
- `padding = { x, y }` — blank space inside the window edges in pixels, scaled by DPI, added at both opposing sides. Default `{ 0, 0 }`.
- `dynamic_padding = true | false` — spread leftover padding evenly so the grid stays centered. Default `false`.
- `decorations = "Full" | "None" | "Transparent" | "Buttonless"` — `Full` is borders and title bar; `None` is neither; `Transparent` (macOS) is a title bar with transparent background and buttons; `Buttonless` (macOS) is the same with no buttons. Default `"Full"`.
- `opacity = <float>` — background opacity, `0.0` (transparent) to `1.0` (opaque). Default `1.0`.
- `blur = true | false` — ask the compositor to blur behind a transparent window (macOS, KDE Wayland). Default `false`.
- `startup_mode = "Windowed" | "Maximized" | "Fullscreen" | "SimpleFullscreen"` — `SimpleFullscreen` (macOS) fullscreens without taking a Space. Restart-only. Default `"Windowed"`.
- `title = "<string>"` — window title. Default `"Zterminal"`.
- `dynamic_title = true | false` — let applications change the title via OSC. Default `true`.
- `class = { instance, general }` — X11/Wayland window class; on Wayland `general` is the `app_id` and `instance` is ignored. Linux/BSD only. Default `{ "Zterminal", "Zterminal" }`.
- `decorations_theme_variant = "Dark" | "Light" | "None"` — override the system/GTK/CSD theme variant; `"None"` uses the system default. Default `"None"`.
- `resize_increments = true | false` — resize in whole-cell steps (macOS/X11/Wayland). Default `false`.
- `option_as_alt = "OnlyLeft" | "OnlyRight" | "Both" | "None"` — make the macOS Option key behave as Alt. macOS only. Default `"None"`.
- `level = "Normal" | "AlwaysOnTop"` — window z-order. Default `"Normal"`.

zterminal's window additions

Three fields here are `zterminal`'s own, not in the inherited man page:

- `background_image = "<path>"` — a GPU image drawn behind the cells (Chapter 25). Because it is config-set, not an escape sequence, it works everywhere including inside `tmux`. Reloads live when the path changes; PNG/JPEG/ GIF.
- `background_image_opacity = <float>` — `0.0–1.0`, how strongly the background image shows.
- `overlay_opacity = <float>` — `0.0–1.0`, the alpha of the glassy control-panel and palette overlays composited over the live terminal (Chapter 18); also driven by a live slider in the panel.

A compact example combining the inherited and the added:

Chapter 9 – Font

DPI, scaling, and multiple monitors

Before the fields, a word on the pixels those fields resolve to, because the same `font.size` produces a different cell size on different displays. A point is a physical unit; a pixel is not. The window manager reports a *scale factor* for the monitor the window is on — 2.0 on a typical macOS Retina panel, some fractional value on a scaled HiDPI Linux display — and zterminal multiplies the point-based font metrics by it to get pixel metrics, so 12pt text is the same physical size on a Retina laptop and a standard external monitor even though one has twice the pixels. Two consequences follow. First, dragging a window between monitors of different scale factors re-resolves the cell size and reflows the grid (Chapter 3), exactly as a font-size change would, because the pixels-per-cell changed. Second, the `WINIT_X11_SCALE_FACTOR` environment variable (settable in `[env]`, Chapter 7) overrides the reported factor on X11 when the automatic value is wrong — the one knob for the “everything is too big” or “too small” symptom on a misdetected display. `window.resize_increments` (Chapter 8) interacts with all of this by snapping window resizes to whole multiples of the current pixel cell size, so the grid never carries a partial row or column regardless of the scale factor in play.

`[font]` selects the faces and tunes their placement in the cell. Every field:

- `normal = { family, style }` — the base face. Defaults are platform- appropriate: `monospace/Regular` on Linux/BSD, `Consolas/Regular` on Windows, `Menlo/Regular` on macOS. First-run seeding prefers an installed Nerd Font so icon glyphs render without configuration.
- `bold = { family, style }` — the bold face; family falls back to `normal`'s if omitted. Default style `Bold`.
- `italic = { family, style }` — the italic face; family falls back to `normal`'s. Default style `Italic`.
- `bold_italic = { family, style }` — family falls back to `normal`'s. Default style `Bold Italic`.
- `size = <float>` — font size in points. Default `11.25`. Changed live by the font-size key bindings, which reflow the grid (Chapter 4).
- `offset = { x, y }` — extra space around each glyph: `y` is line spacing, `x` is letter spacing. These change the cell size. Default `{ 0, 0 }`.
- `glyph_offset = { x, y }` — position of the glyph within its cell; increasing `x` moves it right, increasing `y` moves it up. Default origin is the cell bottom.
- `builtin_box_drawing = true | false` — draw box-drawing, legacy- computing, and powerline glyphs with the built-in geometric font (Chapter 4). Default `true`.

Fallback and the Nerd Font default

No single font contains every glyph a terminal will be asked to draw, so the font stack is a *chain*, not a face. When the primary face lacks a glyph — a CJK ideograph, an emoji, a powerline separator, a Nerd Font icon — the rasterizer walks a fallback chain until it finds a face that has it, and caches the result in the atlas like any other glyph (Chapter 4). This is why a prompt studded with devicon glyphs renders even when your text font knows nothing of them: the system's icon and emoji fonts stand behind it. The one seam worth knowing is metric mismatch — a fallback glyph designed for a different cell size may sit high, low, or wide — which is exactly what `font.glyph_offset` and the built-in box-drawing font exist to paper over. zterminal leans into this by preferring an installed Nerd Font as the seeded default face on first run (Chapter 7), so powerline and icon glyphs are in the *primary* face rather than reached through fallback, sidestepping the metric problem for the glyphs most likely to hit it.

Styles are faces, not synthesis

The four `[font]` faces — normal, bold, italic, bold-italic — are real, separately loaded font files, not the bold and slant that a renderer can fake by smearing and shearing the normal face. Synthesized bold thickens strokes unevenly and synthesized italic shears the box-drawing glyphs into gaps; loading the designed face avoids both. When you omit a face's `family`, it falls back to `normal`'s family at the requested style, so a font that ships a genuine bold is used for bold automatically, and only a font that lacks one falls back to the normal weight rather than a synthesized approximation. The interaction with colour is worth recalling here: `draw_bold_text_with_bright_colors` (Chapter 11) couples the bold *weight* to the bright *palette*, a legacy convention that some colour schemes still expect, off by default so bold means heavier rather than brighter.

Chapter 10 — Grapheme Clusters, Shaping, and Ligatures

Chapter 3 established that the grid stores cells, not characters, and that width accounting decides how many cells an incoming scalar claims. Chapter 9 established that the font is a chain of faces, walked on fallback. This chapter sits between them and answers the question those two raise but do not settle: when a run of bytes decodes to something more complicated than one character per cell — a base letter with three combining marks, a family emoji built from five code points joined by zero-width joiners, a flag made of two regional-indicator symbols, a programming-font ligature like `!=` — what does a grid terminal actually do? The answer is inherited from the Alacritty lineage, and it is a set of deliberate choices, not gaps. A grid terminal is not a word processor, and understanding where it draws that line is the difference between a prompt that lines up and one that does not.

The grid is the contract

Everything in this chapter follows from one fact stated in Chapter 3: a terminal maintains a fixed grid, and every application that draws into it — vim, tmux, htop, your shell's line editor — computes its own layout by assuming a specific, stable mapping from characters to cells. When the shell's line editor puts the cursor after your twelfth typed character, it moves the cursor by the *cell width* it believes those twelve characters occupy. If the terminal and the application disagree about that width by even one cell, the cursor lands in the wrong place, redraws smear, and the display desynchronizes. So the terminal's first obligation, above prettiness, is to agree with every application about exactly how many cells each thing occupies. That contract is what constrains shaping, and it is why a terminal cannot freely reshape text the way a browser or a word processor can.

Grapheme clusters: many code points, one cell

A *grapheme cluster* is what a human calls “a character”: a base scalar plus any combining marks that attach to it. The grid resolves each cluster to a width and stores it against a single cell (a wide cluster against a cell plus its spacer, per Chapter 3). The cases the terminal must get right:

- **Combining marks.** `e` followed by U+0301 (combining acute) is one grapheme, `é`, one cell wide, advancing the cursor by one. The combiner is zero-width (Chapter 3); it is stored with the base and rendered composed. A naïve terminal that advanced the cursor on the combiner would push every following cell one column right.
- **Variation selectors.** U+FE0F (emoji presentation) and U+FE0E (text presentation) select whether the preceding scalar draws as a colourful emoji or a monochrome glyph, without advancing the cursor.  versus  is the same base scalar with and without the selector — the terminal must honour the selector's effect on rendering while treating it as zero-width for layout.
- **Zero-width joiners.** A ZWJ (U+200D) sequence fuses several emoji into one: the “family” and “profession” emoji are code-point sequences the Unicode standard says render as a single glyph. Whether the fallback emoji font actually *has* that composed glyph is a font question (Chapter 9); whether the terminal keeps the whole sequence anchored to one grid position, so the cursor advances once, is the terminal's question, and the grid keeps the cluster together.
- **Regional indicators.** A flag is two regional-indicator scalars; the pair is one grapheme, drawn as a flag glyph if the font has it and as two letter-boxes if it does not — but either way occupying the width the pair

resolves to, so layout is stable regardless of whether the flag renders.

The through-line is that grapheme clustering is *layout* work the grid must do, and it does it, because getting it wrong breaks the width contract. What the terminal does *not* promise is that every exotic cluster has a glyph — that is the font's job, and a missing composed emoji falls back to its component glyphs or a tofu box (Chapter 9) without the layout shifting.

Shaping, and why a grid terminal does little of it

Shaping is the process a full text engine performs to turn a run of characters into positioned glyphs: applying the font's substitution and positioning tables to form ligatures, position marks precisely, and adjust spacing (kerning) between letters. A browser shapes every run. A terminal in the Alacritty lineage deliberately does almost none of it, and the reason is the grid contract above. Kerning would pull letters closer than one cell, breaking the monospace assumption every TUI relies on. Contextual substitution that changed a glyph's advance would change how many cells it occupies, desynchronizing the application's cursor arithmetic. So the terminal renders essentially per-cell: each grapheme cluster is rasterized and placed in its cell (via the atlas of Chapter 4), marks are composed onto their base within that cell, and the pen does not carry kerning-adjusted positions across cell boundaries. This is not an oversight; it is the price of the guarantee that the terminal and the application always agree on the grid.

Ligatures: the honest position

Programming-font ligatures — `!=` drawn as a single “not-equal” glyph, `=>` as an arrow, `>=` and `<=` and `===` and `---` collapsed into single marks — are the most requested feature that the grid model resists, and the honest statement is that a per-cell grid terminal in this lineage does not perform them, by the same logic that rules out kerning. A ligature spans two or more cells and replaces their separate glyphs with one; but each of those cells still has to be individually addressable, selectable, and overwritable, because the application believes `!` and `=` are two distinct grid positions it can move the cursor between, erase one of, or colour differently. The moment the cursor lands between the `!` and the `=`, or the application repaints only the `=`, a fused ligature glyph has to come apart — and reconciling “draw as one” with “address as two” is exactly the complexity the Alacritty lineage chose not to carry, in exchange for a simpler, faster, always-consistent renderer. `zterminal` inherits that choice. The practical consequence is small and worth stating plainly so no reader expects otherwise: your `!=` renders as a clean `!` and `=` in adjacent cells, sharp and correctly spaced, rather than as a fused arrow glyph. Users who want ligatures choose a terminal that pays the cost of full shaping; users who want the grid's speed and exactness — the workflow of Chapter 39 — get the per-cell renderer, and this book documents what the program does rather than rounding up to what a different design would.

Where box drawing fits

There is one place a grid terminal *does* substitute glyphs wholesale, and it is instructive because it proves the rule. The built-in box-drawing font (Chapters 4 and 9, `font.builtin_box_drawing`) replaces the font's own box, block, and powerline glyphs with geometry the renderer draws itself. This is allowed — indeed preferred — precisely because those glyphs are *defined* to occupy exactly one cell and to meet their neighbours at the cell edge. Substituting them changes nothing about width or addressing; it only makes the lines meet. It is the one substitution that does not fight the grid contract, which is why it is the one the terminal makes eagerly while declining kerning and ligatures. The box-drawing case and the ligature case are the two poles of the same principle: substitute freely where the grid is preserved, never where it is broken.

Chapter 11 — Colors

`[colors]` is the largest configuration surface and the one the control panel's theme picker most visibly drives (Chapter 19). Colours are `#RRGGBB` hex strings, or — for the cell-relative fields — the special references `CellForeground` and `CellBackground`. The full structure, sub-table by sub-table:

`colors.primary`

- `foreground` — default text colour. Default `"#d8d8d8"`.
- `background` — default background. Default `"#181818"`.
- `dim_foreground` — dim text; auto-derived from `foreground` if unset. Default `"#828482"`.
- `bright_foreground` — used only when `draw_bold_text_with_bright_colors` is `true`; falls back to `foreground` if unset. Default `"None"`.

`colors.cursor` and `colors.vi_mode_cursor`

Each is `{ text, cursor }`, accepting hex or the cell references. The cursor colours apply to the terminal cursor; the vi-mode pair applies in vi mode, falling back to the normal cursor's active value. Both default to `{ text = "CellBackground", cursor = "CellForeground" }`.

`colors.search`

- `matches = { foreground, background }` — non-focused search matches. Default `{ "#181818", "#ac4242" }`.
- `focused_match = { foreground, background }` — the currently focused match. Default `{ "#181818", "#f4bf75" }`.

`colors.hints`

- `start = { foreground, background }` — the first character of a hint label. Default `{ "#181818", "#f4bf75" }`.
- `end = { foreground, background }` — the remaining label characters. Default `{ "#181818", "#ac4242" }`.

The bars and selection

- `line_indicator = { foreground, background }` — the history-position indicator during search and vi mode; `"None"` uses the opposing primary colour. Default `{ "None", "None" }`.
- `footer_bar = { foreground, background }` — the bottom bar used by the search regex input and hyperlink preview. Default `{ "#181818", "#d8d8d8" }`.
- `selection = { text, background }` — drawing colours for selections, hex or cell references. Default `{ "CellBackground", "CellForeground" }`.

The ANSI palettes: `normal`, `bright`, `dim`

Each is the sixteen-colour set keyed `black red green yellow blue magenta cyan white`. `normal` and `bright` ship a base16 palette (for example `normal.red` is `"#ac4242"`, `bright.red` is `"#c55555"`); `dim` is auto-calculated from `normal` if unset, with documented defaults (`dim.red` is `"#712b2b"`). Appendix B lists every default.

Indexed and the two booleans

- `indexed_colors = [{ index, color }]` — overrides for palette indices 16–255; unset entries are filled with sensible defaults. Default `[]`.
- `transparent_background_colors = true | false` — apply `window.opacity` to every cell background, not just the default one. Default `false`.
- `draw_bold_text_with_bright_colors = true | false` — draw bold text in the bright palette. Default `false`.

A complete theme, as one file

Because `[colors]` is large, a theme is usually kept in its own file and pulled in with `general.import` (Chapter 7), so a colour scheme is a single unit you can swap. A full example — the base16 default written out explicitly, the form any imported theme takes:

```
[colors.primary]
foreground = "#d8d8d8"
background = "#181818"

[colors.normal]
black = "#181818"
red = "#ac4242"
green = "#90a959"
yellow = "#f4bf75"
blue = "#6a9fb5"
magenta = "#aa759f"
cyan = "#75b5aa"
white = "#d8d8d8"

[colors.bright]
black = "#6b6b6b"
red = "#c55555"
green = "#aac474"
yellow = "#feca88"
blue = "#82b8c8"
magenta = "#c28cb8"
cyan = "#93d3c3"
white = "#f8f8f8"

[colors.selection]
text = "CellBackground"
background = "CellForeground"

[[colors.indexed_colors]]
index = 16
color = "#dc9656"
```

Three details in that file repay attention. The `dim` palette is absent, and that is deliberate: leave it out and

zterminal auto-derives it from `normal` (the documented `dim.red = "#712b2b"` and friends of Appendix B), so a theme need only supply the sixteen it cares about. The selection uses the `CellForeground` / `CellBackground` references rather than fixed hex, so a selection inverts whatever the cell already shows and stays legible over any content — the reason those special references exist. And the single `[[colors.indexed_colors]]` entry overrides exactly one slot of the 256-palette (index 16, the first colour past the ANSI sixteen) while every other indexed slot keeps its computed default; this is the array-of-tables form the Settings tab's indexed-colour editor (Chapter 19) appends to. Swap the imported file and the whole scheme changes live, because the import is re-read on reload like any other config.

Chapter 12 — Cursor, Bell, Selection, Scrolling

Four smaller tables finish the feedback surface. Each field in full.

[cursor]

- `style = { shape, blinking }` — `shape` is `Block`, `Underline`, or `Beam` (default `Block`); `blinking` is `Never`, `Off`, `On`, or `Always` (default `Off`). The distinction between `Off/On` and `Never/Always` is whether an application may override the blink: `Off/On` set a default an app can change, `Never/Always` force it.
- `vi_mode_style = { shape, blinking } | "None"` — the cursor in vi mode; `"None"` falls back to the normal cursor's active value. Default `"None"`.
- `blink_interval = <integer>` — blink period in milliseconds. Default `750`.
- `blink_timeout = <integer>` — seconds after which blinking stops; `0` never stops. Default `5`.
- `unfocused_hollow = true | false` — draw a hollow box when the window is unfocused. Default `true`.
- `thickness = <float>` — beam/underline thickness as a fraction of the cell width, `0.0–1.0`. Default `0.15`.

[bell]

- `animation` — the visual-bell easing curve: one of the `EaseOut*` family (`Ease`, `EaseOut`, `EaseOutSine`, `EaseOutQuad`, `EaseOutCubic`, `EaseOutQuart`, `EaseOutQuint`, `EaseOutExpo`, `EaseOutCirc`) or `Linear`. Default `"Linear"`.
- `duration = <integer>` — flash length in milliseconds; `0` disables the visual bell. Default `0`.
- `color = "<string>"` — flash colour. Default `"#ffffff"`.
- `command = "<string>" | { program, args }` — a program run on every bell; `"None"` runs nothing. Default `"None"`.

[selection]

- `semantic_escape_chars = "<string>"` — the characters that bound a “semantic word” (the unit double-click and the `Semantic*` vi motions select). Default is a set of punctuation and whitespace.
- `save_to_clipboard = true | false` — copy every selection to the clipboard automatically, not only to the primary selection. Default `false`.

[scrolling]

- `history = <integer>` — scrollbar line count; `0` disables scrolling, capped at `100000`. Default `10000`.
- `multiplier = <integer>` — lines moved per scroll-wheel notch. Default `3`.

Scrollback: what it costs and where the cap comes from

`scrolling.history` looks like a free dial, and it is not quite, which is why it has a cap. Every line of scrollback is retained cells — characters plus their colours and attributes (Chapter 3) — held in memory for as long as the line stays in the buffer, and a pane that scrolls a firehose keeps replacing the oldest lines with new ones once the buffer is full. The default of `10000` lines is a comfortable balance; raising it to the `100000` maximum keeps a very deep history at a correspondingly larger memory cost *per pane*, which matters precisely because each native pane and each tmux pane has its own scrollback (Chapters 23, 27) — deep history multiplied across many panes adds up. Setting `history = 0` disables scrolling entirely, which is the right choice for a pane that only ever runs a full-screen application on the alternate screen (Chapter 3), where scrollback is never consulted anyway. The cap is not arbitrary paternalism; it is the point past which a single pane's history would consume more memory than any real workflow benefits from, and it is why the field is bounded rather than unbounded.

The bell as a feedback channel

The bell is easy to dismiss and worth a second look, because `[bell]` turns a single event — the `BEL` byte, `0x07` — into a configurable feedback channel. An application rings the bell to say “something happened”: a completed compile, a finished download, a line matched in `less`, a message in an IRC client. By default zterminal does nothing visible with it (`bell.duration = 0` disables the visual flash), but the table lets you route the event three ways at once — a visual flash with a chosen colour and easing curve (`animation`, `duration`, `color`), and a program run on every ring (`command`). The `command` form is the powerful one: a bell can trigger a desktop notification, a sound, or any script, which turns “the build finished and rang the bell” into “the build finished and my notifier told me.” It overlaps deliberately with the output triggers of Chapter 30 — a trigger fires on matched *text*, a bell command fires on the `BEL` byte — and which you reach for depends on whether the program announces itself by ringing or by printing.

Chapter 13 – Terminal and Mouse

Two tables cover the shell and the pointer.

[terminal]

- `shell = "<string>" | { program, args }` — the shell to spawn; entries in `args` are passed to it. Default is `$SHELL` or the login shell on Linux/BSD/macOS, `powershell` on Windows. Example: `{ program = "/bin/zsh", args = ["-l"] }`.
- `osc52 = "Disabled" | "OnlyCopy" | "OnlyPaste" | "CopyPaste"` — controls clipboard access via the `OSC 52` escape. Reading the clipboard from a remote application is the abuse risk, so the default permits only writing. Default `"OnlyCopy"`.

[mouse]

- `hide_when_typing = true | false` — temporarily hide the pointer while typing. Default `false`.
- `bindings = [{ mouse, mods, mode, action | chars | command }]` — mouse bindings, sharing the `[keyboard]` model (Chapter 15). `mouse` is `Left`, `Right`, `Middle`, `Back`, `Forward`, `WheelUp`, `WheelDown`, or an integer button; the action set adds `ExpandSelection`. When an application captures the mouse, holding `Shift` suppresses reporting so the binding fires; if no Shift-binding exists, the un-Shifted action is used. Example: `{ mouse = "Right", mods = "Control", action = "Paste" }`.

Mouse reporting and the Shift override

The pointer has two lives in a terminal, and the boundary between them is the source of most mouse confusion. Ordinarily the mouse is the *terminal's* — a drag selects, a right-click expands the selection, a middle-click pastes the primary selection (Chapter 35). But a full-screen application can *capture* the mouse by enabling a mouse-reporting private mode (1000/1002/1006, Appendix D), after which clicks and drags are encoded as escape sequences and sent to the application instead: this is how `tmux` lets you click a pane, how `vim` positions the cursor on click, how `htop` sorts by clicking a column. When an application has captured the mouse, holding `Shift` suppresses the reporting for that action, so the terminal's own binding fires — a `Shift`-drag selects text even inside a mouse-grabbing TUI. The default bindings encode this: `Middle` (~Vi) pastes the primary selection, and the `Right` expand-selection bindings work whether or not an application is grabbing (Appendix C). The rule to remember is simple — the application owns the mouse when it asks to, and `Shift` takes it back for one action — and it is why “I can’t select text in this program” is answered by holding `Shift` while you drag.

Chapter 14 — Hints

Hints turn visible terminal text into actionable targets: find every URL, path, hash, or hyperlink on screen and open it, copy it, select it, or pipe it to a command — by keyboard or mods-click. They are configured in `[hints]`.

`hints.alphabet`

The keys used for hint labels. Default `"jfkdl;ahgurieowpq"` — a home-row-biased set so labels are fast to type. The label's first character is coloured by `colors.hints.start`, the rest by `colors.hints.end`.

`hints.enabled`

An array of hint definitions. Each must carry at least one *matcher* and at least one *effect*, plus optional refinements:

- `regex = "<string>"` — a pattern matched against each line.
- `hyperlinks = true | false` — match OSC 8 escape-sequence hyperlinks.
- `post_processing = true | false` — trim characters unlikely to belong to a match (a trailing `.` on a URL); `regex` matches only.
- `persist = true | false` — keep hints visible after one is chosen.
- `action = "Copy" | "Paste" | "Select" | "MoveViModeCursor"` — the built-in effect.
- `command = "<string>" | { program, args }` — a program run with the match appended as its last argument.
- `binding = { key, mods, mode }` — the key that starts keyboard hint selection.
- `mouse = { mods, enabled }` — `enabled` underlines the match on hover with `mods` held, making it clickable.

The default and a custom hint

The shipped default opens URIs and hyperlinks with the platform opener (`xdg-open`, `open`, or `cmd /c start`), bound to `Ctrl+Shift+O` and to mods-click, with `post_processing` on. Adding your own is a few lines — a hint that copies a git hash:

```
[[hints.enabled]]
regex    = "[0-9a-f]{7,40}"
action   = "Copy"
binding  = { key = "G", mods = "Control|Shift" }
```

You need not hand-edit the TOML: the control panel's **Hints** tab (Chapter 18) is a graphical editor for this same `[hints]` table — a form for the regex, the built-in action or piped command, the hyperlink / post-processing / persist flags, and the key and mouse triggers, plus the `hints.alphabet`. It rewrites the whole list on every change, and the running windows reload it at once.

How matching and refinement actually work

The two required ingredients — a matcher and an effect — and the optional refinements combine in an order worth understanding, because it explains the otherwise-mysterious `post_processing` flag. When you trigger hints, every enabled definition is matched against the visible text: a `regex` definition tests each line against its pattern, and a `hyperlinks = true` definition matches any `OSC 8` hyperlink on screen (Chapter 6). Each match becomes a labelled target, the label drawn from `hints.alphabet` and coloured by `colors.hints` (Chapter 11). You type the label; the effect — `action`, `command`, or the built-in open — runs on the matched text.

`post_processing` exists because regexes over-match at the edges. A URL regex will happily swallow the trailing `.` of a sentence or the `)` that closed a parenthetical, so the “copy this URL” you meant includes punctuation that is not part of the URL. With `post_processing = true` the matched span is trimmed of characters unlikely to belong to it before the effect runs (regex matches only — hyperlinks are exact by construction and need no trimming). `persist = true` keeps the hint overlay up after you pick one, for selecting several targets in a row rather than re-triggering each time. And a definition can carry *both* a keyboard `binding` and a `mouse` trigger, so the same target is reachable by typing a label or by mods-clicking an underlined match — the two access paths to one hint. The practical upshot is that a good hint is a matcher tight enough to catch the thing, `post_processing` on to clean the edges, and whichever trigger fits your hands.

Chapter 15 – Keyboard and Bindings

The binding model is the most important inherited surface to understand in detail, because every action zterminal can perform — inherited or original — is reachable as a binding, and the model is more expressive than “key maps to string.”

The shape of a binding

`keyboard.bindings` is an array of tables, each combining a trigger and an effect. The trigger is a `key`, optional `mods`, and optional `mode`; the effect is exactly one of `action`, `chars`, or `command`.

```
[keyboard]
bindings = [
  { key = "N", mods = "Control|Shift", action = "CreateNewWindow" },
  { key = "L", mods = "Control|Shift", chars = "l" },
]
```

- `key` is a named key ("A", "F1", "Enter", the "Numpad..." family), a literal character, or a decimal scancode. The named-key vocabulary is winit's.
- `mods` combine with |: `Command`, `Control`, `Option`, `Super`, `Shift`, `Alt`.
- `mode` requires (or, with a ~ prefix, forbids) a terminal mode: `AppCursor`, `AppKeypad`, `Search`, `Alt`, `Vi`. "`~Vi|Search`" means “not in vi mode, and in search mode.” This is how the same physical key does different things by context.
- `action` runs a built-in action; `chars` writes a literal string to the PTY; `command` forks a program.

Two escape hatches: `action = "ReceiveChar"` removes a default binding by letting the key fall through as input, and `action = "None"` inhibits the key entirely. Multiple bindings can fire on one press, in definition order.

The action catalogue, by group

Seeing the groups is more useful than memorizing the list (Appendix C is the full default table; `zterminal(5)` lists every action):

- **Always available** — `Paste`, `Copy`, `IncreaseFontSize/DecreaseFontSize/ResetFontSize`, the eight scroll actions (`ScrollPageUp/Down`, `ScrollHalfPageUp/Down`, `ScrollLineUp/Down`, `ScrollToTop/Bottom`), `ClearHistory`, `ClearLogNotice`, `Hide`, `Minimize`, `Quit`, `ToggleFullscreen`, `ToggleMaximized`, `ToggleViMode`, `ClearSelection`, `SearchForward/SearchBackward`, `SpawnNewInstance`, `CreateNewWindow`, `ReceiveChar`, `None`.
- **Vi mode** — the motions, selections, and inline/semantic searches of Chapter 17 (`Up/Down/Left/Right`, `First/Last/FirstOccupied`, `High/Middle/Low`, the `Semantic*` and `Word*` family, `Bracket`, `ParagraphUp/Down`, the `Toggle*Selection` family, `SearchNext/Previous/Start/End`, `Open`, `CenterAroundViCursor`, the `InlineSearch*` and `SemanticSearch*` families).
- **Search mode** — `SearchFocusNext/Previous`, `SearchConfirm`, `SearchCancel`, `SearchClear`, `SearchDeleteWord`, `SearchHistoryPrevious/Next`.
- **macOS exclusive** — `ToggleSimpleFullscreen`, `HideOtherApplications`, `CreateNewTab`, `SelectNextTab/PreviousTab`, `SelectTab1–9`, `SelectLastTab`.
- **Linux/BSD exclusive** — `CopySelection`, `PasteSelection`.

A deliberate non-collision

The MenkeTechnologies extension shortcuts (Chapters 18–30) are layered so that *no zterminal binding masks a shell or tmux key on any OS*. macOS uses `⌘`, which the shell never sees; Linux and Windows use `^↑` or route through the command palette so no bare key is consumed. This is a stated design rule, and it is why a `^`-heavy TUI or a tmux session runs under zterminal without the terminal stealing keys. The binding model is inherited; this discipline about how the extensions use it is the product's, and Chapter 32 notes the single-dispatch path (menu, key, and palette all routing the same action object) that enforces it.

Recipes: the model in practice

The three effect kinds — `action`, `chars`, `command` — and the `mode` guard combine into a small vocabulary that covers most real customization. A handful of worked bindings show the range:

```
[keyboard]
bindings = [
  # Write a literal string to the PTY only when NOT in vi or search mode, so the
  # same physical key still drives vi motions when vi mode is active (Chapter 17).
  # A TOML \u escape sends a control byte – here U+000C, form feed:
  { key = "K", mods = "Control", mode = "~Vi|~Search", chars = "\u000C" },

  # Fork a program with the current selection unavailable to it – a pure launcher:
  { key = "T", mods = "Command|Shift", command = { program = "tmux", args = ["new-window"]
    ~ } },

  # Remove a shipped default by letting the key fall through as ordinary input:
  { key = "N", mods = "Control|Shift", action = "ReceiveChar" },

  # Inhibit a key entirely so nothing – not even the character – reaches the PTY:
  { key = "Q", mods = "Control", action = "None" },

  # A binding that only exists inside search mode (Chapter 17):
  { key = "P", mods = "Control", mode = "Search", action = "SearchHistoryPrevious" },
]
```

The pattern worth internalizing is the `mode` guard. Because a binding can require or forbid a mode, the *same* key can mean one thing at the shell prompt, another in vi mode, and a third in search — which is exactly how the inherited core layers its own vi and search keymaps over the pass-through keys without collision, and how you extend that layering without stealing a key from an application. A binding with `mode = "~Vi|~Search"` is the idiom for “only when the terminal is behaving as a plain terminal,” and it is the safe default for any custom key that should not interfere with the modal features.

Reading the default table

Appendix C reproduces the shipped defaults, and two conventions there are easy to misread. The `~` prefix on a mode is a *negation* — `~Vi|~Search` means “not in vi mode and not in search mode,” so the binding is suppressed while either is active. And where two bindings share a key with different mode guards, both are in the table and the guard decides which fires: `Ctrl+L` maps to `ClearLogNotice` always and to the clear-screen character only under `~Vi|~Search`, so the key clears a transient log notice if one is showing and otherwise clears the screen. This is the binding model's expressiveness on display in the defaults themselves, and it is the reason the table has more rows than there are distinct keys.

Chapter 16 – Debug

[`debug`] exists to troubleshoot the emulator. Its fields can change or be removed without warning, so they are not stable API, but they are the windows into the core. Each field:

- `render_timer = true | false` — overlay the time to draw each frame. Default `false`.
- `persistent_logging = true | false` — keep the log file after quitting. Default `false`.
- `log_level = "Off" | "Error" | "Warn" | "Info" | "Debug" | "Trace"` — log verbosity. Default `"Warn"`. The `ZTERM_EXTRA_LOG_TARGETS` variable adds extra libraries (`ZTERM_EXTRA_LOG_TARGETS="winit;vte" zterminal -vvv`).
- `renderer = "glsl3" | "gles2" | "gles2pure" | "None"` — force a renderer backend; `"None"` picks the highest available. Default `"None"`.
- `print_events = true | false` — log all window events. Default `false`.
- `highlight_damage = true | false` — highlight the changed-cell damage regions. Default `false`.
- `prefer_egl = true | false` — use EGL where allowed (transparency may not work with EGL on Linux/BSD). Default `false`.

The command-line `-v/-q` flags and `--print-events` reach the same machinery; the control panel's Logs tab (Chapter 18) renders the log this produces.

Reading the log

The log at `~/zterminal/zterminal.log` is the terminal's own account of what it did, and `debug.log_level` decides how much of that account it keeps. The levels form the usual ladder — `Off`, `Error`, `Warn` (the default), `Info`, `Debug`, `Trace` — each including everything louder than it, so `Warn` records errors and warnings but not the routine `Info` chatter, while `Trace` records essentially everything the terminal considers worth a line. The command-line `-v` flags map onto this ladder (`-vvv` is the loudest), and `-q` moves the other way for a quieter run. Two refinements matter in practice. `debug.persistent_logging` decides whether the log survives quitting — off by default, so a clean exit does not leave a file behind, on when you are chasing a bug across restarts. And `ZTERM_EXTRA_LOG_TARGETS` widens the net to *other* libraries' logs — `winit` for window-system events, `vte` for the parser — which is how you distinguish “the terminal mishandled a sequence” from “the window system delivered a strange event” when a symptom could be either. The Logs tab (Chapter 18) renders this same file live, so the log is readable without leaving the terminal, and everything the panel shows is text that already went to disk.

Chapter 17 – Vi Mode and Search

zterminal's core includes a keyboard-driven mode for moving through scrollback, selecting text, and following links without a mouse, plus a search mode for finding text in the buffer. Both are inherited and modal in the vi sense.

Entering and leaving

`Ctrl+Shift+Space` toggles vi mode. A second cursor appears, decoupled from the terminal cursor, and the keyboard drives it. `i` or `Ctrl+C` leaves and returns to the bottom; `Escape` clears the active selection. The terminal keeps running underneath — vi mode is a reading overlay on a live buffer, not a frozen snapshot.

Motion

The motions are vi's, on a terminal grid: `h/j/k/l` and arrows move by cell; `0/$/^` go to first column / last column / first non-empty cell; `H/M/L` jump to top/middle/bottom of screen; `w/b/e` move by *semantic* word (bounded by `semantic_escape_chars`) and `W/B/E` by whitespace word; `{/}` move by paragraph; `%` jumps to the matching bracket; `g/G` go to top/ bottom of scrollback; `z` centers on the vi cursor. Scrolling: `Ctrl+b/Ctrl+f` page, `Ctrl+u/Ctrl+d` half-page, `Ctrl+y/Ctrl+e` line.

Selection and opening

`v` starts character selection, `V` line, `Ctrl+V` block, `Alt+V` semantic; pressing the same key toggles it off. `y` copies and clears. `Enter` runs `Open`, launching the URL under the vi cursor.

Inline, semantic, and buffer search

Inline search within the line: `f/F` to a character, `t/T` stopping short, `;/,` to repeat. Semantic search: `*/#` jump to the next/previous occurrence of the word under the cursor. Buffer search: `/` forward, `?` backward; matches highlight live using `colors.search`; `n/N` step through them; `Enter` confirms (or, outside vi mode, focuses the next match). In search mode, `Ctrl+U` clears, `Ctrl+W` deletes a word, `Ctrl+P/Ctrl+N` and the arrows walk the search history, `Escape/Ctrl+C` cancel. The footer bar shows the regex, coloured by `colors.footer_bar`.

The vi-mode key reference

Collected in one place, the vi-mode keymap — every key an action from the vi group of Chapter 15, so the prose above and the binding table agree:

Key	Effect
<code>h j k l</code> , arrows	move the vi cursor one cell

Key	Effect
<code>0 \$ ^</code>	first column / last column / first non-empty cell
<code>H M L</code>	top / middle / bottom of the visible screen
<code>w b e</code>	forward / back / end by <i>semantic</i> word
<code>W B E</code>	the same by <i>whitespace</i> word
<code>{ }</code>	previous / next paragraph
<code>%</code>	jump to the matching bracket
<code>g G</code>	top / bottom of scrollbar
<code>z</code>	center the view on the vi cursor
<code>Ctrl+b Ctrl+f</code>	page up / down
<code>Ctrl+u Ctrl+d</code>	half-page up / down
<code>Ctrl+y Ctrl+e</code>	line up / down
<code>v V Ctrl+V Alt+V</code>	start character / line / block / semantic selection
<code>y</code>	copy the selection and clear it
<code>Enter</code>	Open — launch the URL under the vi cursor
<code>f F t T</code>	inline search to / before a character, forward / back
<code>; ,</code>	repeat / reverse the last inline search
<code>* #</code>	next / previous occurrence of the word under the cursor
<code>/ ?</code>	start a buffer search forward / backward
<code>n N</code>	next / previous buffer-search match
<code>i or Ctrl+C</code>	leave vi mode, return to the bottom
<code>Escape</code>	clear the active selection

The table is the same action set the Keys tab (Chapter 18) exposes as rebindable rows, so nothing here is hard-wired: a user who prefers a different motion key rebinds it, and the change lands live like any other config. The two searches to keep distinct are *inline* search (`f/t`, within the current line, the classic vi character-find) and *buffer* search (`/`, across the whole scrollbar with live `colors.search` highlighting); they read similarly but operate at different scopes, and the semantic search (`*/#`) is a third, jumping by whole occurrences of the word the cursor sits on.

Chapter 18 – The Control Panel

Here the book crosses the line drawn in Chapter 2. Everything to this point is the inherited, faithfully-documented core. The control panel is the first piece of the original extension layer, and the most architecturally distinctive: zterminal's *entire* configuration and inspection surface is a single-binary, in-process WebView application built only from the shared `zgui-core` component library and served over a custom `zterminal://` protocol.

The architecture

The control panel is not a separate process, not a remote web page, and not a native dialog set. It is a WebView hosted inside zterminal's own process. The frontend assets — `settings/frontend` plus the `zgui-core/webui` component library — are embedded into the binary with `include_dir!` and served over a custom `zterminal://` scheme. A JavaScript bridge (`window.__ztermInvoke`) posts messages to the Rust side, which dispatches them. The same fuzzy matcher, `ZGui.fzf`, powers the command palette, the shell-history palette, and cross-pane search — one component, reused, not three reimplementations. The repository frames this as a first: a terminal whose whole control plane is a reusable design-system app rather than native dialogs or a TUI (Chapter 32).

The glassy overlay

The panel and palette WebViews composite as translucent glass over the still- visible GPU terminal beneath. On macOS this is done by driving the native window's `alphaValue` directly through AppKit (winit's own transparency crashes the view), and a live opacity slider — backed by the `window.overlay_opacity` config field — sets it. The terminal keeps rendering underneath; the control plane floats over it.

One action, three front doors

Every macOS menu item carries the same key-equivalent as its keybinding and dispatches the *identical* action object through the same `run_palette_action` path as the key press and the command palette. Menu, key, and palette are three views of one action set, with one source of truth. This is what keeps the non-collision rule of Chapter 15 honest and what makes the command palette (Chapter 22) a complete index of everything the terminal can do.

The tabs

The tab strip is far broader than a settings dialog. Observed in the running app: Settings, Dashboard, Commands, Sessions, Layouts, tmux, tmux config, tmux buffers, tmux keys, Profiles, Env Vars, Snippets, Triggers, Hints, Processes, Recent Dirs, Keys, and Logs (plus an About panel). Each is a manager for one facet of the environment, and each has its own chapter or section ahead: Settings (Chapter 19), Profiles (Chapter 20), Dashboard (Chapter 21), Commands and the palettes (Chapter 22), Sessions and Layouts (Chapters 23, 27), the four tmux tabs (Chapter 27), Env Vars / Snippets / Triggers / Processes / Recent Dirs (Chapter 30), Hints (the graphical editor for the `[hints]` table of Chapter 14), Keys (the rebinding table from Chapter 15), and Logs (the `debug.log_level` output from Chapter 16). The README's older

six-tab list (Settings, Dashboard, tmux, Keybindings, Logs, About) describes an earlier, smaller panel; the running app is the authority.

The bridge, concretely

The claim “an in-process WebView that reads and writes the live config” rests on a specific, small mechanism, and naming it removes the magic. The frontend — the `settings/frontend` assets plus the `zgui-core/webui` component library — is embedded into the binary at compile time with `include_dir!`, so there is no directory of web files to install and nothing to fetch over the network; the whole control plane ships inside the one executable. It is served to the WebView over a custom `zterminal://` scheme rather than `http://`, so it is not a server listening on a port that another process could reach. Communication runs one way through a JavaScript bridge — `window.__ztermInvoke` posts a message from the frontend to the Rust side — and the Rust side dispatches each message to the handler that reads or writes the relevant state: a Settings-tab change becomes a `toml_edit` write to `~/.zterminal/zterminal.toml` (Chapter 19), a tmux-tab action becomes a wire command to the server (Chapter 27), a palette action becomes a `run_palette_action` call (below). The pattern is the same in every tab — the frontend renders and posts, the Rust side owns the state and acts — which is why one component library can present config, tmux, telemetry, and process management without each being a bespoke integration.

One dispatch path, three front doors

The single most load-bearing architectural choice in the panel is that a menu item, a keybinding, and a palette entry are three *views* of one action object, not three code paths. Every macOS menu item carries the same key-equivalent as its binding and dispatches the identical action through the same `run_palette_action` path the key press and the palette use. This is what keeps the non-collision rule of Chapter 15 honest — there is one place an action is defined, so there is one place to reason about which key it consumes — and it is what makes the command palette (Chapter 22) a *complete* index of what the terminal can do, because an action that exists is, by construction, an action the palette can dispatch. Add a capability once and it appears in all three front doors; that is the dividend of routing them through one object.

The tabs, at a glance

Gathered as a table, the panel's tabs and the chapter that documents each — the map of the whole control plane:

Tab	What it manages	Chapter
Settings	the live <code>zterminal.toml</code> , plus panel theming	19
Dashboard	live telemetry (PTY, render, tmux, system)	21
Commands	the command palette's action index	22
Sessions	tmux session save / restore	27
Layouts	native split-layout save / restore	23
tmux	the live session → window → pane tree	27
tmux config	the server's options, edited live	27
tmux buffers	the server's paste buffers	27
tmux keys	the server's key tables, rebound live	27
Profiles	whole-environment snapshots	20
Env Vars	<code>[env]</code> plus hot-inject into live shells	30
Snippets	reusable command snippets	30
Triggers	per-pane output-reaction regexes	30
Hints	a graphical editor for <code>[hints]</code>	14

Tab	What it manages	Chapter
Processes	the per-pane CPU/MEM process tree	29
Recent Dirs	the harvested working-directory list	30
Keys	the rebindable keybinding table	15
Logs	the <code>debug.log_level</code> output, live	16
About	version and identity	18

The breadth of that list is the point of the chapter: this is not a settings dialog with a few panes but a full management surface for the terminal, its multiplexer, and its environment, every tab a manager over live state and every one built from the same `zgui-core` components.

Chapter 19 – The Settings Tab

The Settings tab is the control panel's front page and the friendly face of the TOML from Chapters 7–16. Its header reads `ZTERMINAL · SETTINGS · AUTO-SAVES`, and a status line at the bottom reads `Editing ~/.zterminal/zterminal.toml` — it edits the canonical config in place, and because the config hot-reloads, every change lands in the live terminal immediately. A search box (“Search settings...”) and a profile selector sit above the content; a Reload button is in the header. Observed in the running app, the tab opens with two panel-theming blocks — Appearance and Effects — and then a full, searchable editor for the config file itself, one labelled section per table of Chapters 7–16.

Appearance

A **Light Mode** toggle, and a grid of eight named colour-scheme presets that theme the panel itself:

- **Cyberpunk** — hot pink + cyan neon (the default).
- **Midnight** — deep blue + electric purple.
- **Matrix** — terminal green on black.
- **Ember** — warm amber + orange.
- **Arctic** — cool whites + icy blue.
- **Crimson** — rose-red + teal.
- **Toxic** — acid-lime + magenta.
- **Vapor** — vaporwave pastel pink + cyan.

Below the presets is a full **palette editor** — Accent, Cyan, Magenta, Green, Yellow, Orange, Red, the four background tiers (Primary, Secondary, Card, Hover), Text, Text-Dim, Text-Muted, Border, and Link — with a *Save custom as preset* button. `zterminal` also carries a global `hsv` (hue/saturation/value) adjustment in its UI config, so a whole scheme can be rotated rather than edited swatch by swatch.

Effects

Four toggles for the panel's own visual treatment, the cyberpunk house style rendered in the WebView: **CRT scanlines**, **Bezel vignette**, **Neon glow**, and **Animations**.

The configuration editor

Below the two theming blocks the tab becomes a graphical front end for nearly the whole of `zterminal.toml`, laid out as one labelled section per table and each field a typed control — a toggle, a number stepper, a picker, a hex swatch, or a command-line box — that writes its value straight back to the file. The sections, in order: **General** (the `general.live_config_reload`, `general.ipc_socket`, and `general.working_directory` fields); **Font**; **Cursor**; then the full **Colors** family — Primary, the Normal / Bright / Dim sixteen-colour ANSI sets, Selection, Search, Vi-mode cursor, Line indicator, Footer bar, Hint labels, and an **Indexed (256-palette)** list editor that overrides any single slot 16–255 via `[[colors.indexed_colors]]`; **Window** (including startup **position** and the X11/Wayland **app class**); **Scrolling**; **Bell** (with a *Command on bell* box that runs a program on every ring); **Selection**; **Mouse & Terminal** (including the **Shell** each new terminal launches); and **Debug**. Two field kinds are worth naming: the command-line boxes for `bell.command` and `terminal.shell` are split into a `{ program, args }` on save by their own backend rather than stored as

a bare scalar, and the indexed-colour editor owns its own save path, appending array-of-tables rows. The thing to take from the detail is the architecture, not the list: a GUI that is part of the terminal process, themed in the stack's cyberpunk style, editing the live config file, and covering essentially every table the man page documents.

Chapter 20 – Profiles

Profiles are the first of zterminal's three claimed in-repo firsts (Chapter 32). A profile is a named snapshot of the *entire* configuration that you switch between in one click — and it captures far more than the terminal config.

What a profile holds

- the full terminal config (`zterminal.toml`);
- the GUI look — colour scheme, light/dark, custom scheme, effects;
- and the **tmux server state** — the server's options, paste buffers, and key bindings.

Switching a profile restores the whole set, including reconfiguring the *live tmux server* over the wire protocol (Chapter 27). The repository's framing is direct: no emulator has profile-switchable tmux configuration. This is why profiles are listed as a first and not merely a convenience — a config-snapshot feature is ordinary, but one that snapshots and restores a running tmux server's state alongside the terminal's own is not.

Using them

The Settings header carries a **PROFILE** dropdown with **New profile** and **Delete** controls; the running app shows it reading **– NO PROFILES –** before any are created. Create a profile to capture the current whole-environment state; switch to it later to restore it. Because the restore reaches the live tmux server, switching a profile mid-session reconfigures tmux without a detach.

What switching restores, step by step

The claim that makes profiles a first is that a switch restores three normally- separate things at once, so it is worth walking what actually happens when you pick a profile. First, the terminal config is applied — the whole of `zterminal.toml` the profile captured — and because the config hot-reloads (Chapter 7), every open window re-themes and re-configures in place without a restart. Second, the GUI look is restored: the panel's colour scheme, light/dark, custom palette, and effects (Chapter 19), which are the WebView's own state rather than the terminal's. Third, and this is the novel part, the tmux server state is restored *over the wire* (Chapter 27): the profile carries the server's options, paste buffers, and keybindings, and the switch pushes them to the running server with the same `set-option`, `paste-buffer`, and `bind-key` commands the live tmux editors use. A config-snapshot feature that restored the first two would be ordinary; restoring the third — reconfiguring a running multiplexer to match a saved environment, without detaching the session — is what no other emulator does, and it is only possible because the terminal is a genuine tmux client rather than a program that shells out to `tmux`. The three restores land together, so switching a profile is switching your *whole* working environment, terminal and multiplexer alike, in one click.

Chapter 21 – The Dashboard

The Dashboard is the second of the three claimed firsts: a live, in-app telemetry surface built from a real component library. `^Z` opens it.

What it shows

The Dashboard polls and renders, using `zgui-core`'s gauges, sparklines, donuts, meters, tables, and stat strips:

- **PTY throughput** — bytes flowing through the pseudo-terminals.
- **Render timing** — the frame-draw time the renderer tracks (Chapter 4).
- **Scrollback** — buffer occupancy.
- **The full tmux summary** — clients, sessions, windows, panes, and the server identity, read over the wire protocol (Chapter 27).
- **System metrics** — CPU, memory, and the rest of a `sysinfo` snapshot.
- **Command history** — the commands recorded from OSC 133 marks (Chapter 26), each with its text and exit code.

The repository claims no terminal emulator ships a custom dashboard of this kind, and notes that it doubles as a showcase of the component library it is built from. Whether or not the “first” holds against every niche tool, the concrete fact is verifiable in the running app: a polling, component-built telemetry surface inside the terminal, fed by the same PTY, renderer, and tmux-client the rest of the program uses.

Where the numbers come from

The Dashboard is not a separate monitoring agent; it is a reader of counters the terminal already keeps, which is why it costs so little and why its numbers are the real ones. The PTY throughput is the byte volume flowing through the pseudo-terminals the process owns (Chapter 33) — the same reads the parser consumes, counted. The render timing is the per-frame draw time the renderer already tracks for `debug.render_timer` (Chapters 4, 16), surfaced without the debug flag. The scrollback occupancy is the buffer the grid already maintains (Chapter 12). The tmux summary — clients, sessions, windows, panes, server identity — is read over the one wire client the process holds (Chapter 27), not scraped from a subprocess. The system metrics come from a `sysinfo` snapshot, the same crate the process tree uses (Chapter 29). And the command history is the record built from the OSC 133 marks the scanner already recovers (Chapter 26). Every gauge is a view onto a counter the program keeps for its own reasons, which is the honest sense in which the Dashboard is a *showcase* of the component library rather than a new measurement system: the data was always there; `zgui-core`'s gauges, sparklines, and donuts just draw it.

The cost of looking

A telemetry surface that polled continuously would violate the performance discipline of Chapter 34, so it does not. The Dashboard samples on an interval while it is open and stops when it is closed — its cost is paid only when you are looking at it, and it is a periodic read of existing counters rather than a continuous instrument on the hot path. This is the same “cost proportional to use” choice the Processes tree, Recent Dirs, and the tmux

tab all make (Chapter 34): a manager that reads live state does so on a refresh cadence, so an idle-but-closed tab is free and an open one is cheap.

Chapter 22 – Command and History Palettes

The control panel is the broad surface; the palettes are the fast one. zterminal has three searchable overlays, and together they mean almost nothing in the terminal requires remembering a keybinding.

The command palette

`⌘K` opens it: a searchable list of every window, tab, terminal, panel, and pane action zterminal can perform, including every control-panel tab and a family of `tmux`: actions (new window, split horizontal/vertical, next/previous window or session, zoom/rotate/break pane, next layout, choose-tree switcher, new/kill session, detach). Because every action is *also* a rebindable shortcut (the Keys tab) and a menu item (Chapter 18), the palette, the keyboard, and the menu are three views of one action set. This is the mechanism that lets the extension layer add dozens of capabilities without adding dozens of mandatory keybindings.

The palette is also *user-programmable*. Above zterminal's built-in entries it merges a personal command set — `ZGui.userCommands`, the shared `zgui-core` feature the whole GUI suite carries — that you add from the app-shell settings. Each entry opens a URL, runs a snippet of JavaScript, applies a colour scheme, or dispatches an event, and giving it a keyword makes it argument-taking: typing `<keyword> <arg>` runs it with the argument substituted. Because the store is one JSON file in the shared config directory, a command you add here appears in every zgui app, and your own entries always rank above the terminal's.

The shell-history palette

`⌘R` opens a fuzzy search over your shell history that *inserts* the chosen command at the prompt rather than running it, so you review it before pressing Enter. Its distinctive engineering — a claimed first (Chapter 32) — is that it searches the *actual* history file of the focused shell, discovered by reading that child process's live `HISTFILE` environment variable, and parses zsh-extended (`: <ts>:<dur>;cmd`), bash, and fish formats, rather than assuming a default path. It uses the shared `ZGui.fzf` matcher.

Prompt navigation, MRU, and Quake

With OSC 133 sourced (Chapter 26), `⌘↑/⌘↓` jump to the previous/next prompt and `⌘⇧0` selects the output of the most recently finished command — navigation by the *semantic* structure of the session, not by text search. `⌘E` is the MRU quick-switch to the most-recently-used other tab. And `⌘`` summons the Quake dropdown: a global-hotkey terminal that slides down from the top of the screen, available from any application. Every one of these is in the command palette too.

One fuzzy matcher, reused everywhere

The three palettes feel consistent because they are, literally, the same matcher. `ZGui.fzf` — the fuzzy matcher from the shared `zgui-core` library (Chapter 18) — ranks the command palette's actions, the history palette's shell commands, and the cross-pane search's scrollbar (Chapter 29), and it is one component

reused rather than three search boxes that happen to look alike. That reuse is why typing behaves identically in all of them: the same subsequence matching (the query characters must appear in order but need not be contiguous, so `nw` finds “New Window”), the same scoring that rewards matches at word boundaries and against shorter candidates, and the same highlight of the matched characters in the results. When the book says the palettes mean “almost nothing in the terminal requires remembering a keybinding,” this matcher is why: you do not recall the exact name, you type a few characters of what you want and the ranking surfaces it. Building that matcher once and wiring three surfaces through it is the same single-source discipline the action dispatch of Chapter 18 follows — find the first implementation, reuse it, do not reimplement.

The palette as an open surface

The command palette is not a closed list of built-ins. Above `zterminal`'s own entries it merges `ZGui.userCommands`, the shared personal command set the whole GUI suite carries, so an entry you add — open a URL, run a snippet of JavaScript, apply a colour scheme, dispatch an event — appears here and in every other `zgui` app, ranked above the terminal's entries. Giving an entry a keyword makes it argument-taking: typing `<keyword> <arg>` runs it with the argument substituted, which turns the palette from a menu into a tiny command line. Because the store is one JSON file in the shared config directory, the palette is a place you *extend* rather than only a place you *browse*, and the extension travels across the suite with you.

Chapter 23 – Native Splits and Layouts

A multiplexer like `tmux` draws multiple panes by repainting regions of a single grid. `zterminal` does it the other way: it tiles *natively*, with its own i3-style binary split tree, one independent shell and PTY per pane, each pane GL-scissored into its own sub-viewport. There is no multiplexer in the loop and no single grid being subdivided — each pane is a real terminal.

The split tree

Any pane splits vertically (`⌘D`) or horizontally (`⌘⇧D`), and splits nest arbitrarily, so the window is a binary tree whose leaves are panes and whose internal nodes are splits. Each leaf owns its own shell, PTY, and scrollbar, and renders in its own scissored viewport so the GPU draws them independently in the one frame. Because the panes are the emulator's own, inline images (Chapter 25) and OSC 133 marks (Chapter 26) work in a native pane exactly as in a full window, with no passthrough dance and no multiplexer redraw latency.

Moving around the tree

The navigation set is i3's: `⌘]/⌘[` focus next/previous; `⌘←↑↓→` focus the pane in a spatial direction; `⇧⌘←↑↓→` move (swap) the focused pane; `⌘⇧←↑↓→` resize along the split; `⌘⇧↔` zooms a pane to fill the window and toggles back; `⌘⇧W` closes the focused pane. Every one is in the command palette.

Layouts

`⌘⇧L` saves and restores a window's split tree. Saving captures the split geometry plus each pane's working directory and foreground command to `~/.zterminal/layouts/<name>.json`. Restoring rebuilds the tree, respawns each pane in its saved directory, and *replants* the saved command at the prompt — without running it; you review and press Enter. Live process state is not restored, because no tool can restore it; panes return in the right geometry and directory as fresh shells. This is the native counterpart to the `tmux` session save/restore in Chapter 27, and the two share the same honest limitation.

The scissor: why native panes are cheap

The mechanism that makes native panes different from a multiplexer's is worth seeing directly. A multiplexer like `tmux` owns *one* grid and paints several logical panes into regions of it, which means every pane's output flows through the multiplexer's own parser and redraw before it reaches the terminal — a layer of latency and a place where escape sequences (images, OSC 133) get filtered. `zterminal` instead gives each leaf pane its own real terminal — its own PTY, its own grid, its own scrollbar — and draws them all into the one GL surface by *scissoring*: before drawing a pane, the renderer restricts drawing to that pane's sub-rectangle of the window, draws that pane's damaged cells there, then moves to the next. The GPU clips to the scissor rectangle, so a pane physically cannot draw outside its bounds, and the frame loop of Chapter 5 coalesces all the panes' damage into the one frame. The consequences are the ones Chapter 23 opened with: because

each pane is a genuine terminal, an inline image (Chapter 25) or an OSC 133 mark (Chapter 26) works in a pane exactly as in a full window — there is no multiplexer between the application and the emulator to strip it — and because they share one frame and one process, splitting costs a scissor rectangle and a PTY, not a subprocess and a redraw pipeline.

The tree, precisely

The split tree is a binary tree, and every operation is a tree edit. A split replaces a leaf with an internal node whose two children are the old leaf and a new pane; closing a pane removes a leaf and collapses its parent, promoting the sibling into the parent's place so no empty space is left. Focus is a pointer to the current leaf, and the spatial navigation (`\C + arrow`) walks the tree geometry to find the nearest leaf in a direction, while the ordinal navigation (`C]/C[`) walks the leaves in order. A resize adjusts the split ratio stored at an internal node, which is the single number that says how the node's area divides between its children — which is exactly what a saved layout serializes (Chapter 23's `layouts/<name>.json`) so a restore can rebuild the same proportions. Seeing the tree as the data structure makes the whole keybinding set legible: every split key adds a node, every close key removes one, every focus key moves the pointer, and every resize key edits a ratio.

Chapter 24 — Exposé, MRU, and the Quake Dropdown

Three window-level features sit between the splits and the tmux client, because they span both native panes and tmux panes.

Exposé

`^⌘E` opens Exposé: a tile grid of every zterminal window *and* every tmux pane at once; click a tile to focus it. This is the unifying view that treats the native split tree and the reparented tmux server as one spatial surface. The repository lists the blend — native i3 splits plus tmux, tiled together in one Exposé, with inline-image protocols decoded even through tmux passthrough — as a combination candidate (Chapter 32).

MRU quick-switch

`⌘E` jumps to the most-recently-used other tab — the terminal equivalent of an editor’s “switch to last buffer,” so two tabs you are alternating between are one keystroke apart.

The Quake dropdown

`^⌘`` is a global hotkey: a dropdown terminal that slides down from the top of the screen over whatever application is focused, and slides back. It is the always-one-keystroke-away terminal, independent of which window has focus.

Chapter 25 — Inline Graphics

zterminal renders images inline in the grid, on the GPU, in all three common terminal image protocols, plus a config-set background image. This is original extension work built on the inherited renderer's compositing path (Chapter 4), and it is gated by an architectural decision worth stating first: a single scanner runs *ahead* of the VT parser.

The scanner ahead of the parser

The `vte` parser zterminal inherits discards the escape sequences these protocols use — it is a faithful VT parser, and these are non-VT extensions. So zterminal places one scanner in front of the parser that peels image sequences out of the PTY byte stream before the parser sees them: Kitty's APC sequences, Sixel's DCS strings, and iTerm2's OSC 1337. The *same* scanner also extracts the OSC 133 marks of Chapter 26 — one pre-`vte` stage recovers four protocols `vte` would drop. Sequences *wrapped by tmux* (`ESC Ptmux;...`, with `set -g allow-passthrough on`) are unwrapped and decoded, so images survive a multiplexer. The repository lists this unified pre-`vte` recovery — incl. OSC 133 and the `tmux-unwrap` — as a candidate (Chapter 32).

The three protocols

- **Kitty graphics protocol** — the most complete: transmit (`a=t`), transmit-and-display (`a=T`), put-by-id (`a=p`), support query (`a=q`, so capable tools auto-detect zterminal), delete (`a=d`); RGB/RGBA/PNG (`f=24/32/100`); direct and file (`t=f`) transmission; zlib compression (`o=z`); chunked payloads; stacking order (`z=`); Unicode placeholders (`U=1`) for grid-anchored placement; and animation (`a=f` frames, `a=a` play/stop/loop) played on a timer. Placeholder placement works through `tmux`.
- **Sixel** — `ESC P ... q ... ST`, with RGB and HLS colour registers, run-length encoding, and raster attributes.
- **iTerm2 inline images** — `OSC 1337;File=...` (the `imgcat` protocol), in any format the decoder handles (PNG/JPEG/GIF).

In all three, images are uploaded as GPU textures and blitted over the cells they occupy. They scroll with the buffer, clip at pane edges, draw behind (`z < 0`) or over the text, and are freed when they scroll out of the scrollbar.

The bundled icat

zterminal bundles a zero-dependency display tool, `extra/zterminal-icat`, which emits the Kitty protocol from any image using only `base64` (and `sips` on macOS for non-PNG input). Uniquely for an `icat`-style tool, it enables `tmux allow-passthrough` itself and places the image via Kitty *Unicode placeholders*, so the picture survives `tmux` redraws where a plain display escape would be erased on the next repaint (Chapter 32 lists this). Symlink it onto `PATH` and it stays current with the repo:

```
ln -sf "$PWD/extra/zterminal-icat" /usr/local/bin/zterminal-icat
zterminal-icat picture.png
```

Any other inline-image tool works too — `kitten icat`, `timg`, `chafa`, `img2sixel`, `imgcat`.

The background image

Distinct from inline images, zterminal draws an image *behind* the cells, set in config (`window.background_image`, `window.background_image_opacity`) rather than by an escape sequence — so it works everywhere, including inside tmux:

```
[window]
opacity = 0.85
background_image = "~/Pictures/wall.png"
background_image_opacity = 1.0
```

The image is uploaded once as a GPU texture and drawn each frame (cover-fitted) behind any cell whose background is the default colour; lowering `opacity` reveals more of it. It reloads live when the path changes. The config-survives- tmux angle and the negative-z behind-text behaviour are the parts the repository calls distinctive (Chapter 32).

How a Kitty transmission is read

The scanner-ahead-of-the-parser design is easiest to believe once you see the bytes. The Kitty graphics protocol wraps its control data in an APC envelope — `ESC _ G ... ESC \` — carrying a set of `key=value` pairs and, after a `;`, a base64 payload. A minimal “transmit RGBA and display now” looks like:

```
ESC _ G a=T,f=32,s=64,v=64,t=d ; <base64 RGBA bytes> ESC \
```

Read left to right: `a=T` is the action, transmit-and-display; `f=32` says the pixels are 32-bit RGBA (`f=24` is RGB, `f=100` is PNG); `s` and `v` give the image width and height in pixels; `t=d` says the payload is *direct* (inline base64) rather than `t=f` (a file path). The `vte` parser zterminal inherits would discard this whole envelope as an unknown APC string — which is exactly why the pre-`vte` scanner (above) exists to peel it out first, decode the payload into pixels, upload them as a GPU texture, and blit the result over the cells the cursor occupies. A large image is split across several chunks with `m=1` on all but the last (`m=0`), so the transmission survives the terminal's own read batching; the scanner reassembles the chunks before decoding. A capable tool sends `a=q` first — the support query — and only commits to the Kitty path if zterminal answers, which is the runtime capability detection of Chapter 37 applied to graphics.

Placement, ordering, and the lifecycle

Once decoded, an image is not a character; it is a textured quad with a position, a size, a z-order, and a lifetime, all resolved against the grid. Placement anchors the image to the cell the cursor was on when it arrived, or — with Kitty Unicode placeholders (`U=1`) — to explicit placeholder cells the application prints, which is the mode that survives a tmux redraw because the *placeholders* are ordinary text tmux will faithfully repaint (Chapter 25's `icat` uses exactly this). The z-order (`z=`) slots the image into the same compositing pass as the text (Chapter 4): a negative `z` draws it behind the cells, a positive `z` over them. The lifetime is tied to the scrollbar — the image scrolls with the buffer it was placed in, clips at pane edges so it never bleeds into a neighbour (Chapter 23), and its texture is freed when the anchoring lines scroll out of the scrollbar and can no longer be reached. This is why inline images do not accumulate as a memory leak over a long session: they are owned by the grid region they belong to, and they die with it.

The other two protocols, briefly

Kitty's protocol is the most complete, but the same scanner-decode-blit path serves the other two, and their envelopes are worth recognizing. Sixel wraps its data in a DCS string — `ESC P ... q ... ST` — where the payload is a run-length-encoded raster built from colour registers, six pixels tall per band:

```
ESC P q "1;1;100;100 #0;2;0;0;0 #1;2;100;100;100 ... ST
```

The **q** introduces the sixel data, the **#n;...** sequences define colour registers in RGB or HLS, and the raster attributes give the image geometry. iTerm2's inline-image protocol is simpler still — an OSC 1337 carrying a base64 file, the **imgcat** form:

```
ESC ] 1337 ; File=inline=1;size=<bytes>;width=auto;height=auto : <base64> ST
```

Both decode to the same thing a Kitty transmission does — pixels uploaded as a GPU texture and blitted over the cells (Chapter 4's compositing pass) — and both are peeled out by the same pre-**vte** scanner (above), which is the whole point of that one interceptor: three image protocols and OSC 133 recovered by a single stage rather than four bolt-ons. Which protocol a tool uses is the tool's choice; the terminal decodes all three, and the support query (**a=q**, Chapter 37) lets a capable tool prefer Kitty when it detects it.

Chapter 26 — Shell Integration

OSC 133 is a small escape-sequence convention with a large payoff: if the shell emits a mark at each prompt, before each command, and around each block of command output, the terminal learns the *semantic* structure of the session — where every prompt, typed command, and output block begins and ends. `zterminal` reads OSC 133, and the features that fall out of it are this chapter.

Why a scanner, again

The inherited `vte` parser discards OSC 133, just as it discards the image protocols. So the same pre-`vte` scanner from Chapter 25 peels the OSC 133 marks out of the PTY stream and anchors them to *absolute scrollbar line numbers*. Anchoring to absolute lines rather than screen positions is what lets prompt navigation survive scrolling and reflow.

What the marks enable

- **Prompt jumping** — `⌘↑/⌘↓` (`^↑/^↓` on Linux/Windows) jump to the previous/next prompt: navigation by command, not by line.
- **Output selection** — `⌘⇧0` selects the entire output of the most recently finished command, ready to copy or pipe.
- **Failed-command gutter marks** — a thin bar in the left margin flags every prompt line and turns red when that command exited non-zero, driven by the recovered OSC 133 `D;exit` mark. A claimed addition (Chapter 32); the exit status rides in the mark.
- **Command history** — each command, with its text and exit code, is recorded and shown in the Dashboard (Chapter 21); it is also the data the `⌘R` history palette reads.

Turning it on

Source the snippet for your shell from `extra/shell-integration/` (bash, zsh, fish):

```
source /path/to/zterminal/extra/shell-integration/zterminal-shell-integration.zsh
```

The marks are zero-width and other terminals ignore them, so it is safe to source everywhere — your `.zshrc` needs no per-terminal guard. Inside `tmux`, enable `set -g allow-passthrough on` so the marks survive the multiplexer, the same passthrough the image protocols use.

The four marks, precisely

OSC 133 is four marks, and knowing which is which makes the features above legible. The shell integration emits them at the moments their letters name:

- **OSC 133 ; A** — *prompt start*. Emitted just before the shell prints its prompt. This is the anchor prompt-jumping lands on (`⌘↑/⌘↓`), and the gutter mark's position.
- **OSC 133 ; B** — *prompt end / command start*. Emitted after the prompt, before the command you type. The span from `B` to the next mark is your typed command line.

- `05C 133 ; C` — *command output start*. Emitted after you press Enter, before the command's output. This begins the output block.
- `05C 133 ; D ; <exit>` — *command end*. Emitted when the command finishes, carrying its exit code. The span from `C` to `D` is the command's output — which is exactly what `⌘⇧0` selects (Chapter 35) — and the `<exit>` is what turns the gutter mark red on failure.

The elegance is that four zero-width marks give the terminal the whole semantic structure of the session: where every prompt, command, and output block starts and ends, and whether each command succeeded. Anchoring them to absolute scrollback line numbers (rather than screen rows) is what lets that structure survive scrolling and reflow (Chapter 3), so a `D` mark's exit status stays attached to the right prompt no matter how the buffer re-wraps. Everything the chapter lists — prompt jumping, output selection, gutter marks, command history — is a query over these four marks, which is why sourcing the one snippet lights all of them up at once.

Chapter 27 – The tmux Client

The most ambitious extension is the tmux client. zterminal does not run a `tmux` subprocess and scrape its output; it speaks tmux's client/server *wire protocol* directly to the server's Unix socket — `imsg` framing, protocol version 8 — resolving the target from `$TMUX` or the default socket, through the `ztmux-core` crate. It is a native tmux client living inside the terminal, and it is the basis for the repository's strongest claimed first: live-editing a running tmux server.

The wire, concretely

It is worth being precise about what “speaks the wire protocol directly” means, because the phrase does a lot of work. tmux is a client/server program: the long-lived `tmux` server owns the sessions and panes, and each `tmux attach` is a thin client that connects to the server's Unix-domain socket and exchanges length-prefixed messages framed with OpenBSD's `imsg` library — a compact binary envelope of a type tag, a peer id, and a payload, the same framing tmux uses between its own processes. zterminal opens that socket itself and becomes such a client: it resolves the target socket from `$TMUX` (whose value carries the path, the server pid, and the session id) or the default location, performs the version- 8 handshake, and thereafter sends commands and reads the server's replies and notifications as first-class messages. There is no `tmux` binary in the loop, no pseudo-terminal scraped for text, no fragile screen-parsing of `list-panes` output — the structured data the server sends to its own clients is the data zterminal reads. That is the difference between *integrating with* tmux and *shelling out to* it, and it is what makes the live editors of this chapter trustworthy rather than a race against a subprocess's stdout.

Reparenting and the one-surface goal

Because zterminal is a genuine client, the panes the server owns can be surfaced alongside the native split panes (Chapter 23) as one spatial thing: Exposé (Chapter 24) tiles both, cross-pane search (Chapter 29) greps both, and the process tree (Chapter 29) descends into the server's reparented children as if they were the terminal's own. The ownership genuinely differs — a native pane's PTY belongs to zterminal, a tmux pane's belongs to the server — but the wire client is the seam that lets the two read as a single surface to the extensions built above them. This is the combination the inventory files as a candidate rather than an outright first (Chapter 32), and the honesty of that label is the same as everywhere: the pieces exist elsewhere in parts, the unification is the claim.

The live pane manager

The control panel's `tmux` tab is a live view of the full session → window → pane tree, each pane carrying a `capture-pane` text preview. Click a pane to switch the attached client to it; toolbar and per-session buttons create, split, zoom, rotate, and kill windows, panes, and sessions. It refreshes while open.

Why the structured data beats screen-scraping

It is worth dwelling on what the wire client buys, because the alternative is what almost every “tmux integration” in other tools actually is. Screen-scraping tmux means running `tmux list-panes -F '...'` as a subprocess,

reading its stdout, and parsing the text — which works until a pane title contains the delimiter you split on, or the format string drifts between tmux versions, or two `list-panes` calls race a pane being created between them and you parse a tree that no longer exists. The wire client sidesteps all of it by reading the same structured messages the server sends its own attached clients: a pane is a record with fields, not a line of text to be re-parsed, and a change to the tree arrives as a *notification* the server pushes rather than a poll the client has to time correctly. `set-option`, `bind-key`, `capture-pane`, `paste-buffer` and the rest are issued as command messages and their results read as reply messages, framed with `imsg` (the version-8 handshake described earlier in this chapter). The live editors of this chapter are trustworthy for exactly this reason: editing a running server's options over the wire is a request/reply against the authoritative state, not a hopeful `sed` of some text representation of it.

Live editing the server: options, buffers, keys

This is the claimed first (Chapter 32). Three separate control-panel tabs edit a *running* tmux server over the wire:

- **tmux config** — reads every `show-options` scope (server, session, window) and edits any option in place with `set-option`, applied instantly.
- **tmux buffers** — lists, views, edits, creates, pastes, and deletes the server's paste buffers.
- **tmux keys** — lists every binding across the key tables and rebinds or unbinds them live with `bind-key/unbind-key`, the command re-tokenized for the wire.

The repository states the claim plainly: no other terminal emulator edits a live tmux server's options, buffers, and keybindings from its own UI. This is also what makes profiles (Chapter 20) able to snapshot and restore tmux server state.

Sessions: native save and restore

`^⌘5` saves and restores the full tmux layout — a native replacement for `tmux-resurrect` and `tmux-continuum`, done over the wire rather than by a plugin (Chapter 32). Saving snapshots every session → window → pane, with each pane's exact `window_layout`, working directory, and *full captured command line*, to `~/.zterminal/snapshots/`. The command line is read natively from the pane process (`libproc KERN_PROCARG52` on macOS, `/proc` on Linux — no `pgrep`). Restoring rebuilds the tree over the wire, including exact split geometry and the active window/pane, starting a server if none is running. It defaults to a plain shell per pane; opt-in *relaunch* re-runs each pane's full command with arguments; *save pane contents* captures and replays each pane's scrollback. A `tmux_restore_processes` preference — edited right in the Sessions panel — gives the resurrect-style whitelist (`name` or `name->replacement`, e.g. `nvim->nvim -S Session.vim`); empty restarts everything except shells and `sudo`. Marking a snapshot *Set startup* auto- restores it at launch. As always, live process state is not restored.

Chapter 28 — Session Restore and Persistence

Persistence is scattered across several earlier chapters — the native layouts of Chapter 23, the tmux session snapshots of Chapter 27, the profiles of Chapter 20, and the state directory of Appendix E — because each grew out of the feature it serves. This chapter gathers them, because they share one architecture, one honest limitation, and one design decision that recurs in every restore path zterminal ships. Reading them together is the fastest way to understand what “restore” can and cannot mean for a terminal, and why zterminal draws the line exactly where it does.

Four things that persist

zterminal writes four distinct kinds of durable state under `~/.zterminal` (Appendix E), and they are worth separating because they capture different scopes and restore through different mechanisms:

- **Native split layouts** (`layouts/<name>.json`, Chapter 23) — the geometry of one window’s split tree, plus each native pane’s working directory and foreground command.
- **tmux session snapshots** (`snapshots/<name>.json` and the sibling `<name>.contents/`, Chapter 27) — the full session → window → pane tree of a *tmux server*, captured and restored over the wire protocol.
- **Profiles** (Chapter 20) — a whole-environment snapshot: the terminal config, the GUI look, and the tmux server’s options, buffers, and keybindings.
- **Harvested lists** — recent directories, command history, triggers, snippets (Chapter 30), each a JSON file the control panel manages.

The first two restore a *shape*: they rebuild panes in the right places running the right shells in the right directories. The third restores a *configuration*. The fourth restores *data*. The interesting engineering is in the first two, because rebuilding a live arrangement of processes is where every terminal and multiplexer has to decide what is honestly recoverable.

The one limitation, stated once for all of them

No restore path in zterminal claims to bring back a running program’s live state, and the book has said so at each site (Chapters 23 and 27); here is the reason, once, because it is the same reason everywhere. A pane’s live state is the memory of a running process — the unsaved buffer in your editor, the position in your pager, the half-typed pipeline, the open connection. That state lives in the process’s address space and the kernel’s file tables, and nothing outside the process can serialize it. tmux-resurrect cannot; a checkpoint/restore-in-userspace tool can freeze a process tree but not a network connection or a pty peer; and zterminal does not pretend to. What a terminal *can* honestly restore is the arrangement — where the panes were, how big, in what directory, running what command *line* — and then respawn fresh processes into that arrangement. That is the promise zterminal makes and keeps, and refusing to over-promise is the same honesty the rest of the book applies to its invention claims (Chapter 32).

Capturing the command line, natively

Restoring the *arrangement* means one hard sub-problem: discovering what command was running in each pane, so it can be replanted or relaunched. The naïve approach shells out to `ps` or `pgrep` and parses text, which is slow, racy, and fragile. `zterminal` reads the command line natively from the pane's process instead — on macOS via `libproc` with `KERN_PROCARG52`, on Linux via `/proc/<pid>/cmdline` (Chapter 27) — recovering the full argument vector, not a truncated process name. The distinction matters for a faithful restore: a pane running `nvim -S Session.vim +Gstatus` should come back as that whole line, arguments intact, not as a bare `nvim`. Reading the kernel's own record of the process arguments is how the snapshot gets the real command instead of a lossy approximation, and it is the same native-over-scraping discipline the `tmux` client applies to the wire protocol in Chapter 27.

Replant versus relaunch

Given a captured command line, a restore has two defensible things to do with it, and `zterminal` offers both because the right choice depends on the command:

- **Replant** (the default for native layouts, Chapter 23) — respawn a plain shell in the saved directory and *type the saved command at the prompt without running it*. You see exactly what was there and press Enter yourself. This is the safe default: it never re-executes something with side effects — a `git push`, a `rm`, a deploy — behind your back, and it lets you review a command that may no longer be appropriate in the restored context.
- **Relaunch** (opt-in for `tmux` snapshots, Chapter 27) — actually re-run each pane's full command with its arguments. This is what you want for the editors, pagers, and long-running monitors that are the *point* of a saved session, and it is gated behind an explicit toggle plus the `tmux_restore_processes` whitelist so it re-runs what you named and leaves the rest as shells.

The whitelist is the resurrect model made precise. Each entry is a process name, optionally with a replacement (`name->replacement`), so `nvim` can be restored as `nvim -S Session.vim` — reopening the editor *and* the saved editor session in one step. An empty whitelist restarts everything except plain shells and `sudo` invocations, the two things you almost never want blindly re-run. The replacement arrow is what turns a dumb “re-run the command” into a smart “re-run the command the way it should come back,” and it is edited right in the Sessions panel rather than a config file.

Pane contents: replaying scrollback

An arrangement restore brings back fresh processes with empty scrollback, which loses the *output* you may have wanted to keep reading. The opt-in *save pane contents* mode (Chapter 27) closes that gap by capturing each pane's scrollback at save time into the `<name>.contents/` directory beside the snapshot, and replaying it into the restored pane so the history is visible above the fresh prompt. This is honestly a *replay of text*, not a restoration of process state — the replayed lines are inert scrollback, not a live program's output — and the book is careful with the distinction, but for the common case of “I want to see what that build printed yesterday,” inert replayed text is exactly what is wanted.

Startup auto-restore

A snapshot marked **Set startup** (Chapter 27) is restored automatically when `zterminal` launches, which turns the save/restore feature into a persistent working environment: quit at the end of the day, relaunch in the morning, and the session's shape — panes, directories, and (with relaunch) the editors and monitors — comes back. Combined with the `--daemon` model of Chapter 33, where one resident process spawns windows on demand, the startup snapshot is how a login session brings up a fully-formed multi-pane environment without a shell script orchestrating `tmux new-session` calls. The arrangement is data in `~/.zterminal/snapshots/`,

versioned as JSON, restored over the wire — not a pile of imperative setup commands that drift out of sync with what you actually use.

Why this belongs to the single process

The persistence layer, like nearly everything in the extension suite, leans on the single-process model of Chapter 33. A layout save enumerates the split tree of a window the process owns; a tmux snapshot reads the server the process's one wire client is attached to; a profile switch reconfigures that same live server and re-themes every window in the process at once. Restore is cheap for the same reason the dashboard and the broadcast overlay are cheap: the state being captured and rebuilt is all reachable from one address space, so a snapshot is a walk over in-process structures and a few wire commands, not a distributed query across a fleet of per-window processes. Persistence is another dividend of keeping the panes together.

Chapter 29 – tmux Broadcast, Search, and Process Tree

Three further tmux capabilities go beyond what native tmux offers, treating native and tmux panes as one surface.

Cross-session broadcast

`^⌘B` opens a broadcast overlay that sends keystrokes — or a saved snippet (Chapter 30) — to an arbitrary *checked set* of tmux panes spanning any windows and sessions at once. Native tmux cannot do this: `synchronize-panes` is scoped to a single window. Check the panes, type the command, Send; the per-window `synchronize-panes` toggle is available from the same overlay. A claimed addition (Chapter 32).

Cross-pane search

`^⌘P` fuzzy-matches (the shared `ZGui.fzf`) the recent scrollbar of *every* tmux pane at once: type a query, see panes ranked by score with the matched characters highlighted, and press Enter or click to jump to that pane. Combined with Exposé (Chapter 24), the native and tmux panes are one searchable, tileable surface.

The per-pane process tree

The **Processes** tab renders a live CPU/MEM process tree rooted at every pane's shell — including descending into the *tmux server's* reparented children — built from a `sysinfo` snapshot and a ppid → children map, with roots seeded from the native panes' shells and the tmux panes' pids (labelled `tmux <s>:<w>.<p>`). It can signal or kill, but only pids that are descendants of its own panes — the kill path is gated to the seen descendant set. The repository frames it as an inspector spanning native panes and the reparented tmux server, Unix-only (Chapter 32).

The kill gate, precisely

A process tree that can signal is a process tree that can do damage, so the kill path is deliberately narrow, and the narrowness is the point. The tree is built from a `sysinfo` snapshot and a parent-to-children map, with its roots *seeded* from a known set: the pids of the native panes' shells and the pids of the tmux panes (labelled `tmux <s>:<w>.<p>`). From those roots it descends, so the tree it shows is exactly the set of processes reachable as descendants of panes the process owns or is attached to — your shells and what they launched, not the whole system's process table. The kill and signal actions are then gated to that same descendant set: a pid the tree did not reach from a seeded root cannot be signalled through it. This means the inspector can end a runaway process *in one of your panes* but cannot be turned into a general `kill` for arbitrary system pids, which is the correct boundary for a tool embedded in a terminal — powerful over your own sessions, inert everywhere else. It is Unix-only because the ppid-to-children mapping and the native pid seeding depend on Unix process semantics (Chapter 32).

Chapter 30 – Env Vars, Snippets, Triggers, Recent Dirs

Four more control-panel managers round out the environment, each backed by a JSON file under `~/.zterminal` and each, in its own way, an original addition.

Env Vars – hot-injected into running shells

Editing an env var in the control panel does two things at once: it persists the variable to `[env]` in `zterminal.toml` (via `toml_edit`), and it *live-broadcasts* the change — `export NAME=value` or `unset NAME` — into the PTY of every already-running shell across all panes, so it takes effect without relaunching. The export is POSIX-single-quoted and prefixed with a leading space to dodge `HISTCONTROL=ignorespace`. The caveat is honest: it injects a shell command, so it affects shells sitting at a prompt, not arbitrary child programs. A claimed addition (Chapter 32).

Snippets

Saved command snippets, reusable from the command palette and — notably — sendable through the cross-session broadcast overlay (Chapter 29), so a snippet can be typed into a chosen set of panes at once.

Triggers – per-pane output reactions

User-defined regexes match each pane's freshly *rendered* output as it streams, firing a desktop notification, a bell, or a shell command (with `$ZT_TRIGGER_TEXT` and `$ZT_TRIGGER_NAME` in its environment). Only lines completed since the last wakeup are scanned, with a cooldown, backed by `~/.zterminal/triggers.json`. The repository is candid that iTerm2's "Triggers" is direct prior art — this is *not* claimed as a first; the only distinguishing angle is the per-pane, zterminal-pane-model framing (Chapter 32).

Recent Dirs

Because OSC 133 and OSC 7 do not carry the working directory, zterminal harvests each open pane's live foreground-process cwd — from its own panes *and* every tmux pane (via `#{pane_current_path}`) — into a pinnable, most-recent-first Recent Dirs list at `~/.zterminal/recent_dirs.json`, sampled on tab refresh. A claimed addition (Chapter 32), Unix-only.

The four managers share a shape

Read together, the four managers of this chapter are variations on one pattern, and seeing the pattern explains why they were cheap to build and why each is backed by its own small JSON file under `~/.zterminal` (Appendix E). Each is a control-panel tab that reads some live state, presents it in `zgui-core` components, writes an edit back, and persists the result — the same frontend-posts, Rust-owns-state shape the whole

panel follows (Chapter 18). What differs is *what* they read and write. Env Vars reads and writes `[env]` in the config and additionally broadcasts an `export` into every live shell (the hot-inject). Snippets reads and writes a saved command list and additionally feeds the broadcast overlay (Chapter 29). Triggers reads and writes a regex-and-action list that a per-pane scanner then consults on each wakeup. Recent Dirs is read-mostly — it *harvests* each pane's live cwd on refresh rather than taking an edit — and writes a pinnable list. The through-line is that none of them is a new subsystem; each is a thin manager over a JSON file and a hook into machinery the process already has (the config, the broadcast path, the scanner, the pane cwd), which is exactly why the extension layer could add four distinct capabilities without four distinct architectures.

Chapter 31 – IPC

zterminal exposes an inherited IPC mechanism — `zt msg` — that controls a running instance from the command line over a Unix socket, without touching the window. `general.ipc_socket` (default `true`, Unix only) enables it.

The messages

`zterminal-msg(1)` documents three:

- `create-window` — open a new window in the *same* zterminal process, optionally with `--hold`, `--working-directory`, `--title`, `--class`, `-o/--option` overrides, and a trailing `-e/--command`.
- `config` — push a live configuration change. `zt msg config 'cursor.style="Beam"'` applies the override; `-w/--window-id` targets a window (default `$ZTERM_WINDOW_ID`, `-1` for all); `-r/--reset` clears all runtime overrides. This is the command-line door to the same hot-reload the control panel drives through the GUI.
- `get-config` — read the runtime config back, per-window or global (`-w -1`).

The socket path defaults to the per-instance socket but can be set with `--socket` on both ends, which is how you address a specific instance among several. `zt msg` is the seam between zterminal and the rest of the shell-driven workflow: a script, a keybinding in another tool, or a `zshrs` function can open windows, retheme a running terminal, or query its config, all without a GUI.

Worked `zt msg` recipes

The three messages compose into small, useful commands. A few worked examples show the range, all against a running instance over its socket:

```
# Open a window in the running process, in a directory, running a command:
zt msg create-window --working-directory ~/src/zshrs --title build -e cargo build

# Switch every window to a beam cursor for one session (no file edit):
zt msg config -w -1 'cursor.style="Beam"'

# Retheme the running terminal by pushing a background colour to one window:
zt msg config -w "$ZTERM_WINDOW_ID" 'colors.primary.background="#0d1117"'

# Read back what the running instance currently believes, globally:
zt msg get-config -w -1

# Clear every runtime override, returning to the on-disk config:
zt msg config -r
```

Two things make these more than a curiosity. First, `config` pushes the *same* override machinery that the control panel and the `-o` command-line flag use (Chapter 7), so a `zt msg config` change is a live reconfiguration indistinguishable from one made in the GUI — it hot-loads into the running window. Second, the `-w` targeting (`-1` for all windows, `$ZTERM_WINDOW_ID` for the current one) means a script can retheme the specific window it is running in, which is how a `zshrs` function might, say, tint the terminal red while a production deploy is in progress and restore it after. The socket is per-instance by default and selectable with `--socket` (Chapter 31,

Appendix A), which is how a script addresses one instance among several. `zt msg` is what turns “configure the terminal” from a GUI action into a scriptable one.

Chapter 32 – The Inventions

This chapter gathers what `zterminal` claims as new, with the honesty the rest of the book has kept: the base VT/grid/search/hints are *not* claimed — they are the Alacritty inheritance of Chapter 2. Everything below is the extension layer's own, and each entry names its source basis so a reader can check it. Three are the repository's own documented firsts; the rest are candidates the broader stack inventory records, with their confidence and caveats.

The three in-repo firsts

These are stated in `zterminal/docs/INVENTIONS.md` as firsts “to our knowledge — no shipping terminal emulator did it before”:

1. **Live-editing a running tmux server.** Reading and editing the server's options, paste buffers, and keybindings from the terminal's own UI, over the native wire protocol with no `tmux` subprocess (Chapter 27). Basis: `crates/ztmux-core` plus the `tmux config/buffers/keys` tabs.
2. **tmux state in switchable profiles.** Profiles that snapshot the whole config — terminal, GUI look, *and* tmux server state — and restore it, including reconfiguring the live server, in one click (Chapter 20). Basis: the profile manager.
3. **A custom live telemetry dashboard.** A polling telemetry surface built from the `zgui-core` component library — PTY throughput, render timing, scrollback, the tmux summary, system metrics, command history — that no emulator ships (Chapter 21).

The catalogued candidates

The stack's inventory (`INVENTIONS.md`) records these additional entries, each with a confidence label and a caveat. They are reproduced here as the inventory frames them — candidates, not settled firsts, and several explicitly *not* firsts:

- **First-class tmux client over the native wire protocol (high)** — the umbrella for the three above: a terminal that is a first-class tmux client and the first to live-edit options/buffers/keys, capture tmux state into profiles, and ship the dashboard. Basis: `ztmux-core`, dashboard on `zgui-core`.
- **Unified Exposé + scrollback search across native and tmux panes (low)** — the blend of i3 native splits with tmux so Exposé (`⌘E`) tiles both and `⌘P` greps every tmux pane, with image protocols decoded through tmux passthrough. Caveat: the pieces exist individually; the combination is the candidate.
- **Per-pane process/activity monitor (med)** — the Processes tree spanning native panes and the repaired tmux server, with kill gated to descendants (Chapter 29). Basis: `src/event.rs pane_process_tree()`. Caveat: Unix-only, an inspector not a `top`.
- **GUI env-var editor that hot-injects into running shells (med)** — Chapter 30. Basis: `src/settings.rs save_env_var()` then `EventType::BroadcastInput`. Caveat: affects shells at a prompt.
- **Native tmux session save/restore over the wire (med)** — Chapter 27. Basis: `crates/ztmux-core/src/snapshot.rs, proc.rs` native command-line capture. Caveat: resurrect-as-concept is prior art; doing it natively over the wire from an emulator is the novelty.
- **Cross-session/cross-window tmux broadcast (med)** — beyond `synchronize-panes` (Chapter 29). Basis: `ztmux-core/src/ops.rs broadcast_list()`.
- **Whole control plane as an in-process zgui-core webview app (med)** — Chapter 18, served over `zterminal://`. Basis: `src/settings.rs include_dir!`

- the `IPC_BRIDGE`.
- **Shell-history palette resolving the focused shell's real HISTFILE (low)** — Chapter 22. Basis: `src/daemon.rs shell_histfile(pid)`. Caveat: history pickers exist; the per-shell HISTFILE-from-process-env resolution is the distinctive bit.
- **Recent-directories tracker from live pane cwd (low)** — Chapter 30. Basis: `src/recent_dirs.rs`. Caveat: sampled on refresh, Unix-only.
- **Single pre-vte interceptor recovering four protocols (low)** — Chapters 25–26, including OSC 133 and tmux-unwrap. Basis: `zterminal_core/src/graphics/scanner.rs`. Caveat: WezTerm supports the image protocols; the unified recovery incl. OSC 133 is the combination.
- **Failed-command gutter marks from OSC 133 exit codes (low)** — Chapter 26. Basis: `zterminal_core/src/shell.rs`. Caveat: exit-status decorations exist (fish/iTerm2).
- **Config-driven background image surviving tmux (low)** — Chapter 25. Basis: `src/config/window.rs`, `src/renderer/graphics.rs`. Caveat: bg images exist; the config-survives-tmux and behind-text angles are distinctive.
- **Zero-dependency icat self-enabling tmux passthrough (low)** — Chapter 25. Basis: `extra/zterm-icat`, `placeholder.rs`. Caveat: a narrow helper.
- **Native menu, keybindings, and palette sharing one dispatch path (low)** — Chapter 18. Basis: `src/macos/menu.rs`. Caveat: an architectural single-source-of-truth, not a user-visible first.
- **Glassy translucent webview overlays over the live GL terminal (low)** — Chapter 18, with a live opacity slider. Basis: `src/settings.rs apply_overlay_alpha()` via `NSWindow.alphaValue`. Caveat: macOS-only, cosmetic.
- **Per-pane output triggers (low)** — Chapter 30. Caveat: iTerm2 “Triggers” is direct prior art — explicitly **not a first**; included for completeness.

The honest reading of this catalogue is the same as the book's thesis. A handful of the entries are genuine firsts the repository documents and stands behind (the tmux live-editing, the profiles, the dashboard); many are low-confidence combinations or are openly marked as prior art; and the whole list sits on top of an inherited core that is not claimed at all. That is exactly the shape a faithful product should present — a clear foundation, a clearly-bounded set of real additions, and no rounding up.

Reading the confidence labels

The catalogue's `high` / `med` / `low` labels are not decoration; they encode how much of a claim survives scrutiny, and reading them correctly is the difference between a marketing list and an honest inventory. A `high`-confidence entry is one the repository stands behind as a genuine first — the first-class tmux client and the three firsts it umbrellas (live editing, profiles, dashboard) — where the authors could not find prior art and the capability is concrete. A `med` entry is a real, distinctive capability with a plausible-but-unverified originality claim: the env-var hot-inject, the native session save/restore, the cross-session broadcast, the whole-control-plane-as-webview — each does something the authors have not seen elsewhere, but the label admits they have not exhaustively proven no one else does it. A `low` entry is either a combination whose pieces exist separately (the unified Exposé, the pre-vte interceptor) or a narrow helper (the self-enabling icat, the glassy overlays), included for completeness rather than asserted as a landmark. And at the bottom, per-pane output triggers are labelled explicitly *not a first* — iTerm2's Triggers is named as direct prior art — which is the strongest signal that the catalogue is honest, because a list willing to print “this one is not ours” next to a feature it ships is not a list rounding up. The labels are a calibration, not a sales pitch: they tell you exactly how hard to squint at each claim, and the book reproduces them rather than laundering the `lows` into `highs`.

Chapter 33 – The Process Model

The book has now documented both halves — the inherited core in Chapters 3–17, the original extension layer in Chapters 18–32. The remaining chapters step back from individual features to the behaviour of the whole running program: how one process holds many windows, how it spends its microseconds, how it copies text, what it trusts, and how it announces its capabilities. This chapter is the first of those, and it is deliberately first, because the process model is the quiet reason so many of the preceding extensions can exist at all. A feature that must reach *every* pane — the dashboard, the command palette, the broadcast overlay, Exposé — is trivial when every pane lives in one address space and nearly impossible when it does not.

One process, many windows

zterminal is single-process, multi-window. `CreateNewWindow` (`⌘N`) opens another window *inside the same running process*; `SpawnNewInstance` forks a brand-new `zterminal` process that shares nothing. The distinction is not cosmetic. When a window is a peer inside the one process, everything the process holds is shared across all of them: the registry of open PTYs, the tmux client and its single socket to the server, the loaded configuration, the profile set, the command history harvested from OSC 133, the recent-directories list, and the control panel's state. That shared world is what makes the cross-cutting extensions cheap. The Dashboard (Chapter 21) polls one PTY registry and sees every pane in every window; the command palette (Chapter 22) can index actions that target any pane; the broadcast overlay (Chapter 29) can enumerate the panes to send to; Exposé (Chapter 24) can tile every window and every tmux pane at once. A per-window-process terminal would have to build an IPC layer to do any of this, and most do not bother, which is why most terminals do not have it.

Windows, tabs, and panes – three nestings

Three distinct levels of containment are easy to conflate, and the keybindings act on different ones, so it is worth naming them precisely:

- **Windows** are operating-system windows, each with its own title, decorations, and placement. `⌘N` makes another; `⌘W`/`⌘Q` close and quit.
- **Tabs** (macOS) are multiple grids stacked in one window behind a native tab bar. `⌘T` opens one, `⌘1–9` select by index, `⌘⇧ ]/[` and `⌘Tab` cycle, and `⌘E` jumps to the most-recently-used other tab (Chapter 24). Tabs are the OS window server's feature, not a multiplexer's.
- **Panes** are the leaves of the native split tree *within* a single tab or window (Chapter 23), each its own shell and PTY, GL-scissored into a sub-viewport. `⌘D`/`⌘⇧D` split; the i3 navigation set moves among them.

A tmux pane is a fourth thing again — a pane owned by the reparented tmux server, surfaced through the wire client (Chapter 27) — and part of zterminal's ambition is to let the native and tmux panes read as one surface in Exposé and cross-pane search, even though their ownership differs.

The daemon

`--daemon` starts zterminal with no initial window: the process comes up, opens its IPC socket, and waits. `zt msg create-window` (Chapter 31) then materializes windows in that already-running process on demand — the fast path, since the process, its fonts, its config, and its tmux connection are already warm. This is the

model a login session or a launcher uses to keep one terminal process resident and spawn windows into it instantly, and it is only coherent *because* the process is the shared host described above.

Why single-process is load-bearing

It is tempting to read the single-process design as a mere implementation convenience. It is closer to a precondition. Nearly every claimed addition in Chapter 32 assumes a shared address space: the live tmux client is one socket the whole process talks over; profiles snapshot a state that is process-global; the env-var hot-inject broadcasts to every PTY the process owns; the process tree seeds its roots from every pane the process knows about. Move to one-process- per-window and each of these becomes a distributed-systems problem with a socket protocol, a leader election, and a consistency story. zterminal declines that complexity by keeping the panes together, and the extension layer is the dividend.

The PTY, concretely

Under every pane is a pseudo-terminal, and its behaviour explains several things users notice. A PTY is a kernel object with two ends: the *primary* end, which zterminal holds and reads and writes, and the *secondary* end, which the shell has open as its stdin, stdout, and stderr. When the shell prints, it writes the secondary end and the bytes arrive at the primary end for zterminal to parse; when you type, zterminal writes the primary end and the bytes arrive at the shell's stdin. The kernel's line discipline sits between them, which is why `Ctrl+C` at a naive prompt sends `SIGINT` to the foreground process group — the discipline, not zterminal, translates the byte into a signal. A window resize propagates through this object too: when the grid's dimensions change (Chapter 3), zterminal sets the PTY's window size, the kernel delivers `SIGWINCH` to the foreground process, and a full-screen application repaints to the new size. This is the plumbing beneath the whole book — the grid on one side, the shell on the other, the PTY the pipe between — and it is per-pane, which is what “each pane is a real terminal” (Chapter 23) means at the kernel level.

Lifecycle: what closes when

The containment levels of this chapter have a matching teardown, and knowing it avoids surprises. Closing a pane (`⌘ W`) closes that pane's PTY, which sends the shell an end-of-file on its input and a hang-up (`SIGHUP`) — the shell exits, its children get the hang-up too, and the split tree collapses the leaf (Chapter 23). Closing the last pane in a tab or window closes the window. `⌘ Q` quits the whole process, closing every window and PTY it owns and dropping the tmux client's socket — the tmux server keeps running, because it is a separate process the client was merely attached to (Chapter 27), which is exactly why a tmux session survives quitting zterminal and a native pane does not. That asymmetry — native panes die with the process, tmux panes outlive it — is the single most important thing to internalize about the two kinds of pane, and it falls directly out of who owns the PTY: zterminal for a native pane, the tmux server for a tmux pane.

Chapter 34 – Performance

The pitch in Chapter 1 said speed is the floor, not the ceiling — the inherited core is fast and the value is the integration on top. But a floor still has to hold, and a terminal that stutters under load makes the whole shell-centric workflow of Chapter 39 stutter with it. This chapter examines the floor: where the microseconds go between a byte arriving on the PTY and a pixel landing on the screen, and the specific techniques that keep that path short.

The read loop

Bytes do not arrive one at a time to be handled one at a time; that would spend more time in syscalls than in work. `zterminal` reads from the PTY in large batches and feeds each batch to the `vte` parser as a run. The parser is a table-driven state machine — the classic VT500-series design — so the per-byte cost of the common case, a printable character, is a state lookup and a write into the current grid cell, with no allocation and no branching pyramid. The expensive events are the rare ones: a resize that reflows the grid, an alternate-screen switch that saves and restores, a scroll that shifts the scrollbar. The architecture is built so the frequent thing is cheap and the infrequent thing is allowed to be dear. This is why `cat`-ing a large file, the traditional terminal torture test, is bounded by how fast the grid can be mutated and the screen redrawn, not by parser overhead.

Damage and the frame cadence

The second half of the loop is drawing, and the governing principle is that a terminal does not redraw when nothing changed and does not redraw *more* than the display can show. The renderer tracks damage — the set of cells altered since the last frame (Chapter 4) — and coalesces however many PTY updates arrived since the last frame into a single redraw at the display's refresh cadence. A program that emits a thousand tiny writes in one frame's worth of time costs one frame, not a thousand. Synchronized output (private mode 2026) hands that batching decision to the application: between the begin and end marks the terminal holds the frame, so a complex full-screen repaint lands atomically instead of tearing through several frames. `debug.render_timer` and `debug.highlight_damage` (Chapter 16) are the two windows into this machinery — the first prints the per-frame draw time, the second tints the regions actually being redrawn, and watching the second while a TUI runs is the fastest way to see how little of the screen most updates touch.

The glyph atlas as amortization

The renderer's central performance idea is amortization: rasterizing a glyph is expensive, so it happens once, the first time a glyph is seen, and the bitmap is cached in the GPU atlas forever after (Chapter 4). Every subsequent appearance of that glyph is a textured quad — no CPU rasterization at all. The steady state of a running terminal, where the same few hundred glyphs recur endlessly, therefore does almost no rasterization; the cost is paid at warm-up and then not again. The only pathological case is an application that cycles through an unbounded set of distinct glyphs faster than the atlas can hold them, forcing eviction and re-rasterization, and that case is vanishingly rare in real terminal use.

The extension layer's performance discipline

The original layer is built to stay off the hot path, and each of its major pieces reflects a deliberate choice to do so. The pre-`vte` scanner (Chapters 25–26) adds one linear pass over the byte stream, but it is a cheap scan for a handful of sequence prefixes, not a second parser. The control panel is a `WebView`, which could have been a stall, but it composites over the still-live GL terminal by driving the window's alpha (Chapter 18) — the read-and-draw loop underneath never waits on it. The `tmux` client speaks over its own socket and folds the server's events into the same event loop rather than blocking on synchronous polls (Chapter 27). Output triggers scan only the lines completed since the last wakeup, under a cooldown, rather than re-scanning the whole buffer (Chapter 30). None of these is an accident; each is the extension layer paying rent to the principle that the terminal must stay fast under the features.

Measuring it

The instrumentation is built in and consistent with the rest of the program. `debug.render_timer` overlays frame time; the Dashboard (Chapter 21) renders live PTY-throughput and render-timing gauges from the same counters, so the numbers are visible without a debug flag; and `--ref-test` records a reference session for regression testing the renderer. The honest posture, as everywhere in this book, is that these are the real measurements the program exposes, not a benchmark table manufactured for a marketing page.

The end-to-end path, named

Pulling the pieces together, the life of a byte from PTY to pixel is a short list of cheap steps, and naming them in order is the clearest summary of where the microseconds go:

1. **Read.** A batch of bytes is read from the PTY in one syscall, not one byte at a time (the read loop above), amortizing syscall overhead across the whole batch.
2. **Scan.** The pre-`vte` scanner (Chapters 25–26) makes one linear pass to peel out image and OSC 133 sequences — a prefix check, not a second parser.
3. **Parse.** The remaining bytes feed the `vte` state machine, whose common case — a printable character — is a table lookup and a cell write, no allocation.
4. **Mutate.** Each parsed token applies to the grid, marking damage as it goes (Chapter 5).
5. **Coalesce.** All the damage since the last frame accumulates; the draw is deferred to the display's cadence (Chapter 5).
6. **Draw.** When a frame is due, the damaged cells become textured quads from the glyph atlas in a few draw calls, and the frame is presented.

Nothing in that path allocates or rasterizes in the common case: the atlas already holds the glyphs (step 6 is a lookup), the parser already knows the transition (step 3 is a lookup), and the read already batched the syscall (step 1). The expensive things — rasterizing a newly-seen glyph, reflowing on resize, switching screens — are pushed to the edges of the path where they happen rarely. This is the whole performance story in one list, and it is why the terminal's speed is a property of its architecture rather than a tuning pass laid over a slow core.

The cost of the features, honestly

The chapter would be dishonest if it claimed the extension layer is free. It is not free; it is *bounded*, and the bound is what matters. The scanner adds a pass whose cost is linear in the byte volume and tiny per byte. The control panel, when open, is a `WebView` doing its own rendering — but on its own, off the terminal's frame loop (Chapter 5), so it costs the terminal nothing while the panel is idle and never blocks a terminal frame even while it is busy. The `tmux` client holds a socket and processes notifications, which is real work, but event-driven work that sleeps when the server is quiet. The Processes tree, the Dashboard, and the Recent Dirs list sample on a refresh interval rather than continuously, so their cost is paid only while their tab is open and looking.

Each of these is a deliberate choice to make a feature's cost proportional to its use rather than a constant tax on the hot path, and the sum of those choices is a terminal that carries an unusually large feature set without the feature set showing up as latency at the prompt.

Latency, not just throughput

Throughput — how fast the terminal can consume a firehose — is the metric torture tests measure, but latency is the one a user feels. Latency here is the time from a keystroke to the character appearing, and it is a different path from the read loop: an input event arrives, translates to bytes written to the PTY, the shell echoes them back, and the echo travels the read-scan-parse-draw path to the screen. The governing constraint is the same frame cadence of Chapter 5 — the echoed character appears at the next frame — so keystroke latency is bounded below by the display's refresh interval and the input-to-frame alignment, not by any per-character work the terminal does, because that work is trivial. What the architecture avoids is *adding* latency: the extension layer's off-the-hot-path discipline means a keystroke is not waiting behind a WebView repaint, a tmux poll, or a trigger scan, because none of those sits between the key and the frame. The honest statement is that a terminal cannot beat the display's refresh for echo latency, and zterminal's goal is to not lose to it — to keep the input path as short as the frame loop allows, which is what the single event loop and the deferred-draw scheduler together buy.

Chapter 35 – Selection, Clipboard, and Copy

Copying text out of a terminal looks trivial and is not. A selection has a shape, a semantic unit, and a destination, and there are two clipboards on the free- desktop platforms with different rules. This chapter is the model, because it is the substrate several extensions build on and because the defaults reward understanding.

The selection model

A selection is a span of cells, and it has one of four shapes. **Character** selection (`v` in vi mode, a plain drag) runs cell to cell, wrapping at line ends. **Line** selection (`V`) takes whole rows. **Block** selection (`Ctrl+V`, or a drag with the block modifier) is rectangular — a column range across a row range, which is how you lift one column out of tabular output. **Semantic** selection (`Alt+V`, double-click) takes a “word” as bounded by `selection.semantic_escape_chars` (Chapter 12): the same field that defines what `w/b/e` traverse in vi mode, so the mouse and the keyboard agree on where a word ends. That agreement is the point of routing both through one configurable set of boundary characters rather than hard-coding “whitespace.”

The two clipboards

On X11 and Wayland there are two independent clipboards, and conflating them is the source of most “paste did the wrong thing” confusion. The **clipboard** proper is the explicit copy-and-paste buffer, written by `Copy` and read by `Paste`. The **primary selection** is the implicit one: any selection populates it, and a middle-click pastes it, with no explicit copy step — the traditional X model. `zterminal` keeps them distinct: `Copy/Paste` use the clipboard, `CopySelection/PasteSelection` use the primary selection (both Linux/BSD-only actions, Appendix C), and `selection.save_to_clipboard` (Chapter 12), when set, mirrors every selection into the clipboard as well so the two behave as one for users who want that. macOS has only the single system pasteboard, so the distinction collapses there and only the clipboard actions apply.

OSC 52 – the programmatic clipboard

An application can read or write the clipboard without a mouse via the `OSC 52` escape (Chapters 6, 13), which is what lets a remote editor over SSH copy to your local clipboard. That power is also the risk: a program that can *read* your clipboard can exfiltrate whatever you last copied, including a password. So `terminal.osc52` defaults to `OnlyCopy` — programs may write the clipboard but not read it — and the four settings (`Disabled`, `OnlyCopy`, `OnlyPaste`, `CopyPaste`) let you widen or close that gate deliberately. The default is the security-conscious choice, and Chapter 36 returns to why.

Bracketed paste

When you paste multiple lines into a naive shell, each newline runs as a command the instant it arrives — a classic way to execute something you meant only to review. Bracketed paste (private mode 2004) fixes this: a supporting application asks the terminal to wrap pasted text in `ESC [ 200 ~ ... ESC [ 201 ~`, so the

program can tell pasted bytes from typed bytes and decline to execute them immediately. `zterminal` implements the mode (Appendix D); modern shells and editors enable it, which is why a multi-line paste into `zsh` or `vim` lands as inert text rather than a runaway.

Selection meets the extensions

Two extensions produce selections without a mouse, and they reuse this same machinery. OSC 133 output selection (`⌘⇧O`, Chapter 26) builds a selection from the semantic prompt marks — the span from one command's output-start to its output-end — and hands it to the ordinary copy path, so a keystroke yields exactly the last command's output on the clipboard. Hints with a `Copy` or `Select` action (Chapter 14) build a selection from a regex match. In both, the *origin* of the selection is novel but the *destination* — the clipboard model of this chapter — is the shared, inherited one, which is the whole program's method in miniature.

Block selection and tabular data

The rectangular block selection deserves a paragraph of its own, because it solves a problem the other shapes cannot. Character selection wraps line to line; line selection takes whole rows; but neither can lift a *column* out of tabular output — the third field of every row of `ps`, the size column of `ls -l`, one column of a `|`-delimited table. Block selection (`Ctrl+V` in vi mode, or a drag with the block modifier) takes a rectangle: a column range across a row range, ignoring everything outside those columns. The result is that you can select the PID column of a process listing without dragging through the command names beside it, or pull one field from a wide log without the surrounding noise. This is not a cosmetic variant; it is the only selection shape that treats the grid as a grid rather than as a stream of wrapped lines, and it pairs naturally with the semantic word boundaries of the other shapes — where semantic selection respects *what a word is*, block selection respects *where a column is*. Both are ways of saying “select the meaningful unit, not the raw span,” which is the theme the whole chapter returns to.

Chapter 36 — Security and Trust Boundaries

A terminal emulator sits on a trust boundary that is easy to forget: it renders a byte stream produced by arbitrary programs, some of them remote and some of them hostile, and at the same time it holds the keyboard and the clipboard. Every escape sequence it answers is a small capability granted to whatever is on the other end of the PTY. This chapter names zterminal's trust boundaries and the defaults that keep them defensible. None of it is exotic; it is the standard terminal threat model, stated plainly.

The bytes are not trusted

The stream arriving from the PTY can come from a remote host over SSH, a file you `cat` by mistake, or a program that has been compromised, and none of those is entitled to control your machine merely by printing. This is part of why the REJECTED discipline of Chapter 6 is a *security* posture as much as a correctness one: a terminal that answers every sequence it can parse offers a wider attack surface than one that cleanly ignores what it does not need. The classic hazards are well known — a title-set-then-report sequence that echoes attacker-chosen text back onto your command line, a clipboard read that exfiltrates a secret, a cursor-position query used to smuggle a response — and zterminal's answer is the inherited one: implement what applications legitimately need, reject the rest, and never report back data an attacker could have planted.

Clipboard: the OnlyCopy default

The single most important security default is `terminal.osc52 = "OnlyCopy"` (Chapters 13, 35). Writing the clipboard is a convenience; *reading* it remotely is an exfiltration primitive, so the default withholds the read. A user who trusts their remote sessions can open it to `CopyPaste`, but the shipped posture assumes the bytes are hostile until told otherwise, which is the correct default for a tool that spends its life connected to other people's machines.

Passthrough is opt-in, on purpose

The inline-image protocols and OSC 133 marks survive tmux only when the user sets `allow-passthrough on` (Chapters 25–26). It is off by default in tmux for a reason: passthrough lets a program *inside* a multiplexer pane speak escape sequences directly to the outer terminal, bypassing tmux's own sanitization. That is exactly what zterminal needs to decode an image through tmux, and it is exactly the capability a careful operator might not want to hand every program in every pane. This book documents how to enable it and does not enable it for you; the choice is the user's, and the default-off is the safe one.

The IPC socket

`general.ipc_socket` (default on, Unix-only) opens a Unix-domain socket over which `zt msg` can create windows and, more consequentially, push configuration overrides and read the running config back (Chapter 31). Anyone who can write that socket can retheme your terminal, change its cursor, or spawn a window in

your session. The boundary here is filesystem permissions: the socket lives under your user-owned state directory, so the protection is the same as any per-user socket — adequate on a single-user machine, and a thing to be aware of on a shared one. If the capability is unwanted, the field turns it off entirely.

Injection and command execution are user-authored

Several extensions run programs or write to shells, and it is worth being exact about who authorized each. The env-var hot-inject (Chapter 30) writes `export NAME=value` into the PTY of shells sitting at a prompt — it is, honestly, a keystroke injection, which is why the book states plainly that it affects shells at a prompt and not arbitrary child programs, and why the export is single-quoted and space-prefixed to behave. `bell.command`, a hint's `command`, and a trigger's shell command all fork a program — but every one of those is a line *you* put in your own config or trigger list, run with your own privileges, not a path a remote program can reach on its own. The distinction the chapter wants to leave you with is that zterminal's code-execution surfaces are user-configured, not remotely reachable, and its remotely-reachable surfaces (the escape parser, the clipboard) are gated by the conservative defaults above.

Hyperlinks open with the platform opener

An OSC 8 hyperlink (Chapter 6) and a hint's URL (Chapter 14) are opened with the system opener — `xdg-open`, `open`, `cmd /c start`. That is the same trust decision a browser makes when you click a link: the target is attacker-influenced text, so the hint post-processing that trims stray trailing characters is a small correctness-and-safety aid, and the ordinary caution about where a link goes applies here as it does anywhere.

The boundaries, as a checklist

The chapter's argument reduces to a short checklist a careful operator can keep in mind, each item tied to a default this book documents:

- **The escape parser** answers only what applications legitimately need and cleanly rejects the rest (Chapters 5, 6). The REJECTED status is a smaller attack surface, not a missing feature.
- **The clipboard** withholds *reading* by default (`terminal.osc52 = "OnlyCopy"`, Chapters 13, 35): a remote program may set your clipboard but not read it, so a copied password cannot be exfiltrated by a sequence.
- **tmux passthrough** is off until you enable it (Chapters 25–26): images and OSC 133 marks work through tmux only when you accept that a program in a pane can speak escape sequences past tmux's sanitization.
- **The IPC socket** is a per-user socket under your state directory (Chapter 31): its protection is filesystem permissions, adequate on a single-user machine, and the field turns it off entirely if unwanted.
- **Code-execution surfaces** — `bell.command`, a hint's `command`, a trigger's command, the env-var inject — are all *user-authored*, run with your privileges, not reachable by the remote byte stream.

The dividing line the whole chapter draws is between what the *bytes* can do and what *you* can do. The bytes get the conservative defaults and the rejecting parser; you get the full set of code-execution features, because you are trusted and the byte stream is not. Every default in the list is chosen on that principle, and none of them is a prompt or a nag (the stack forbids those) — they are quiet, correct defaults that a knowing user can widen deliberately.

Chapter 37 – Terminfo and Capability Advertisement

The final system chapter is about how `zterminal` *tells the world what it can do*, which is a surprisingly load-bearing question: an application decides which sequences to send based on what it believes the terminal supports, and if that belief is wrong the symptoms are the familiar ones — missing colours, broken arrow keys, garbled box drawing. There are two mechanisms, a static database and a set of runtime queries, and `zterminal` participates in both.

TERM and terminfo

The static mechanism is `terminfo`: a compiled capability database keyed by the `TERM` environment variable, which every curses application consults to learn how to move the cursor, set a colour, or clear the screen for *this* terminal. `zterminal` ships its own entry, `extra/zterminal.info`, and installing it lets `TERM=zterminal` advertise the emulator's real capabilities. When the entry is not installed — on a fresh machine, or the far side of an SSH hop — `zterminal` falls back to identifying as `xterm-256color`, a nearly-universal entry that is correct about the common capabilities and silent about `zterminal`'s extras. Installing the entry (Chapter 38) is never required to run the binary, but it closes the gap between what the terminal can do and what applications believe it can.

The runtime alternative: queries

The static database is a belief; the runtime queries are a measurement. Rather than trust `TERM`, a modern application can ask the terminal what it supports and adapt to the answer. `zterminal` answers the standard probes — Device Attributes and the Device Status Report family (`CSI n`, Chapter 6), the identify sequence (`ESC Z`) — and, importantly, the Kitty graphics support query (`a=q`, Chapter 25), which is how an image-capable tool auto-detects that it can send Kitty graphics to `zterminal` without being told. Query-based capability detection is strictly more robust than `TERM`, because it survives the SSH hop that leaves the remote `terminfo` stale, and `zterminal` supports it precisely so tools that prefer to measure rather than assume get a truthful answer.

Keyboard protocol advertisement

Keys are their own advertisement problem. The legacy encoding cannot distinguish, say, `Ctrl+I` from `Tab`, and the modern fix is the Kitty keyboard protocol — the `CSI u` family (Chapters 6, 15) — which disambiguates keys and reports modifiers unambiguously. An application enables it by pushing a flag with `CSI > u` and querying the active mode with `CSI ? u`; `zterminal` implements the family, so an editor that wants unambiguous keys can negotiate them at runtime and pop the mode when it exits. This is capability advertisement for input rather than output, and it follows the same query-and-adapt shape as the graphics probe.

Over SSH, concretely

The practical failure mode is the SSH hop: you connect to a host that has never heard of `TERM=zterminal`, ncurses cannot find the entry, and applications either error out or fall back badly. There are two honest fixes. Copy the entry to the remote once — `infocmp zterminal | ssh host tic -x -` compiles it into the remote user's terminfo — and thereafter `TERM=zterminal` works there too. Or, for a host you do not control and will not revisit, set `TERM=xterm-256color` for that session and accept the common-denominator capability set. This book prefers to state the seam rather than pretend it is not there: a terminal that advertises a name the world does not know must either teach the world the name or fall back to one it does, and `zterminal` is built to do both.

What the entry advertises

It helps to know what a terminfo entry actually *is*, because it demystifies both the fallback and the SSH fix. The entry is a set of named capabilities — booleans (does it do colour?), numbers (how many colours?), and strings (what bytes move the cursor to row R, column C?) — that curses reads once at startup and uses to drive the terminal without knowing which terminal it is. The `xterm-256color` entry `zterminal` falls back to is correct about the capabilities almost every terminal shares — 256 colours, the standard cursor motions, the alternate screen — which is why the fallback works for the overwhelming majority of applications; what it omits is `zterminal`'s extras, the capabilities beyond the common set. Installing the `zterminal` entry replaces the common-denominator description with the accurate one. The reason this matters over SSH specifically is that the entry lives on the machine running the application, not the machine drawing the screen: a remote host has its own terminfo database, and if `zterminal` is not in it, `TERM=zterminal` names an entry the remote curses cannot find — hence the `infocmp zterminal | ssh host tic -x -` fix, which copies the description across so the name resolves on the far side too. The static database and the runtime queries are two answers to one question — “what can this terminal do?” — and an application that asks the second (Device Attributes, the graphics and keyboard queries above) is robust against the first being stale, which is the deeper reason `zterminal` answers the queries at all.

Chapter 38 – Building and Installing

zterminal builds from source with `cargo`. The build is vendored and self-contained — no external runtime dependencies — but the platform graphics and font libraries are needed at compile time, because the renderer is real OpenGL and the font stack is real system fontconfig.

Cargo and from source

```
cargo install zterminal          # binary only

git clone https://github.com/MenkeTechnologies/zterminal.git
cd zterminal
cargo build                     # debug binary at target/debug/zterminal
```

Pin the stable toolchain with `rustup override set stable`. The minimum build dependencies vary by platform; on Debian/Ubuntu:

```
apt install cmake g++ pkg-config libfontconfig1-dev libxcb-xf86dev libxkbcommon-dev
python3
```

`INSTALL.md` carries the per-distribution lists for Arch, Fedora, RHEL, openSUSE, Void, the BSDs, NixOS, Gentoo, Alpine, Windows, and more.

The macOS installer

`make pkg` builds a single `.pkg` that drops `Zterminal.app` into `/Applications` and the `zterminal-icat` helper into `/usr/local/bin`:

```
make pkg                        # -> target/release/osx/Zterminal-<version>.pkg
```

It builds a release binary, assembles and ad-hoc-signs the app bundle (stamping the version from `zterminal/Cargo.toml`), and packages both targets. The package is unsigned, so the first launch needs a right-click → Open. `make app` and `make dmg` build just the bundle or a disk image; an already-built binary can be packaged with `ZTERMINAL_BINARY=target/debug/zterminal extra/osx/make-pkg.sh`.

After the build

Install the terminfo entry from `extra/zterminal.info` so `TERM=zterminal` advertises the real capabilities (otherwise zterminal falls back to `xterm-256color`); install the man pages from `extra/man/`; install the shell completions from `completions/`. None is required to run the binary, but each closes a seam between zterminal and the surrounding system.

Vendored, self-contained, and durable

The build is vendored on purpose, and the purpose is durability. “No external runtime dependencies” means the shipped binary carries what it needs rather than reaching out to fetch it at run time, so there is no version of a

helper program to drift, no download to fail on a fresh machine, no curl-pipe-bash step to rot. The compile-time dependencies that remain are the *real* platform libraries the renderer and font stack genuinely use — OpenGL for drawing, fontconfig for face resolution on Linux/BSD — which is why the Debian dependency line names `libfontconfig1-dev` and the XCB and xkbcommon development headers: these are not conveniences the build could vendor away, they are the system's graphics and font services the terminal must bind to. The discipline is the stack's general one — a `cargo build` on a fresh machine should work years from now with whatever toolchain exists then — and it shows up here as a build that pins the stable toolchain, depends only on foundational system libraries, and ships everything else inside the one binary.

What “installing” actually places

It is worth separating the binary from its satellites, because only the first is required. The binary is the terminal; `cargo install` or `make pkg` places it (the macOS package additionally drops `zterminal-icat` into `/usr/local/bin`, Chapter 25). Everything else — the terminfo entry (Chapter 37), the man pages, the shell completions, the shell-integration snippets (Chapter 26) — is a *satellite* that closes a seam with the surrounding system but is not needed to run. This separation is why the book can say each of them is optional without hedging: a fresh binary runs and falls back gracefully (identifying as `xterm-256color`, sourcing no integration), and each satellite you install upgrades one interaction — capability advertisement, documentation, completion, prompt marks — from “works by fallback” to “works fully.” Install the binary to have a terminal; install the satellites to have it fully integrated.

Chapter 39 – Where It Sits in the Stack

zterminal is not a standalone product in the MenkeTechnologies world; it is the window the rest of the stack lives in, and the final chapter places it there.

The host

The stated development workflow is shell-centric to an unusual degree: the shell is the IDE, the build system, the deploy tool, and the orchestrator, and many synchronized shell instances run at once. In that world the terminal is the substrate every other tool sits on, and anything that slows it slows the whole pipeline. That is why the core is the fast inherited one and the extensions are integrated rather than shelled out: a multiplexer subprocess, an image-viewer subprocess, and a session-manager plugin are three more things between a keystroke and a result, and zterminal's thesis is to collapse them into the terminal itself.

The neighbours

zterminal hosts the stack's own tools. `zshrs` — the Rust `zsh` — runs inside it. `zpw` paints its prompts, status bars, and TUIs inside it, and the built-in box drawing (Chapter 4) and truecolor (Chapter 3) are what make a `zpw` status bar render seamlessly. The control panel is built from `zgui-core`, the same shared component library the stack's GUI applications use, so the design system is one across the suite. The screenshot that grounds Chapter 19 shows exactly this neighbourhood: a `zpw`-themed session running behind the zterminal control panel, the two halves of the stack in one window.

The honest summary

zterminal earns its place in the MenkeTechnologies catalogue not by being a faster terminal — that would be the `awkrs` trap, dead on the project's own rules — but by adding, on top of a fast inherited GPU core, a set of genuine firsts (live `tmux` editing, `tmux`-aware profiles, the telemetry dashboard) and a broad, clearly-bounded layer of integration the bare emulator lacks. The first half of this book documented that core faithfully and completely, every setting walked. The second half documented the original layer, every feature and every claimed invention, with its basis. The product is the sum: a fast, inherited, faithfully-documented emulator core, wrapped in an original, integrated extension suite, hosting the stack it was built for. That is what zterminal is, and that is where it sits.

The one binary, as an argument

If there is a single sentence that captures the design, it is that all of this is one binary and one process. The core emulator, the split tree, the `tmux` wire client, the inline-graphics decoders, the OSC 133 scanner, the control panel, the palettes, the dashboard, the profiles, the persistence layer — none of them is a subprocess, a plugin, a daemon, or a remote page. They share one address space, one event loop, one config file, and one GPU surface (Chapters 1, 33). The book has argued throughout that this is not merely tidy but *enabling*: the cross-cutting features — a dashboard that sees every pane, a broadcast that reaches every `tmux` pane, an Exposé that tiles native and `tmux` panes together, a profile that restores terminal and multiplexer at once — are cheap because the state they touch is all reachable from the one process, and would each become a

distributed-systems problem in a per-window-process design. The integration is the pitch (Chapter 1) and the single process is how the integration is affordable. A reader who takes one structural idea from this book should take that one: the features are not bolted to the side of a terminal, they are what becomes possible when the terminal, the multiplexer, the image viewer, the session manager, and the control plane stop being separate programs and become one.

Appendix A — Command-Line Reference

The flags, options, and subcommands of `zterminal(1)`. The binary is `zterminal` (commonly aliased `zt`).

Flags

`-h, --help` · `--hold` (stay open after the child exits) · `--daemon` (no initial window) · `--print-events` · `-q` (quieter, min `-qq`) · `-v` (louder, max `-vvv`) · `--ref-test` · `-V, --version`.

Options

`--class <general>` or `<general>,<instance>` (window class; default `Zterminal,Zterminal`) · `-e, --command <cmd>...` (run a command; must be last) · `--config-file <path>` · `--embed <parent>` (X11 window ID) · `-o, --option <opt>...` (override config, e.g. `-o 'cursor.style="Beam"'`) · `--socket <path>` · `-T, --title <title>` (default `Zterminal`) · `--working-directory <dir>`.

Subcommands

`msg` — send IPC messages (`create-window`, `config`, `get-config`; Chapter 31). `migrate` — upgrade a config file: `-c/--config-file`, `-d/--dry-run`, `-i/--skip-imports`, `--skip-renames`, `-s/--silent`.

Appendix B – Configuration Reference

A field index of `zterminal.toml`, by table. Full prose is in Chapters 7–16 and `zterminal(5)`; the `zterminal`-specific additions are marked **(zt)**.

[general]

`import` (paths) · `working_directory` (string | "None") · `live_config_reload` (bool, default `true`) · `ipc_socket` (bool, default `true`, Unix).

[env]

Arbitrary `KEY` = "value" pairs, injected into every spawned process.

[window]

`dimensions` {columns, lines} (restart) · `position` · `padding` {x, y} · `dynamic_padding` · `decorations` (Full/None/Transparent/Buttonless) · `opacity` · `blur` · `startup_mode` (Windowed/Maximized/Fullscreen/ SimpleFullscreen, restart) · `title` · `dynamic_title` · `class` {instance, general} · `decorations_theme_variant` · `resize_increments` · `option_as_alt` · `level` · **(zt)** `background_image` · **(zt)** `background_image_opacity` · **(zt)** `overlay_opacity`.

[scrolling]

`history` (default 10000, max 100000) · `multiplier` (default 3).

[font]

`normal/bold/italic/bold_italic` {family, style} · `size` (default 11.25) · `offset` {x, y} · `glyph_offset` {x, y} · `builtin_box_drawing` (default `true`).

[colors]

`primary` {foreground #d8d8d8, background #181818, dim_foreground #828482, bright_foreground None} · `cursor/vi_mode_cursor` {text, cursor} (default {CellBackground, CellForeground}) · `search` {matches #181818/#ac4242, focused_match #181818/#f4bf75} · `hints` {start #181818/#f4bf75, end #181818/ #ac4242} · `line_indicator` {None, None} · `footer_bar` {#181818, #d8d8d8} · `selection` {CellBackground, CellForeground} · `normal/bright/dim` (16 ANSI each) · `indexed_colors` [{index, color}] · `transparent_background_colors` · `draw_bold_text_with_bright_colors`.

Default **normal**: black #181818 red #ac4242 green #90a959 yellow #f4bf75 blue #6a9fb5 magenta #aa759f cyan #75b5aa white #d8d8d8. Default **bright**: black #6b6b6b red #c55555 green #aac474 yellow #feca88 blue #82b8c8 magenta #c28cb8 cyan #93d3c3 white #f8f8f8. Default **dim**: black #0f0f0f red #712b2b green #5f6f3a yellow #a17e4d blue #456877 magenta #704d68 cyan #4d7770 white #8e8e8e.

[bell]

animation (ten curves, default **Linear**) · **duration** (ms, default **0**) · **color** (default #ffffff) · **command**.

[selection]

semantic_escape_chars · **save_to_clipboard** (default **false**).

[cursor]

style {**shape** **Block**, **blinking** **Off**} · **vi_mode_style** (default **None**) · **blink_interval** (default **750**) · **blink_timeout** (default **5**) · **unfocused_hollow** (default **true**) · **thickness** (default **0.15**).

[terminal]

shell (string | {**program**, **args**}) · **osc52** (**Disabled**/**OnlyCopy**/**OnlyPaste**/ **CopyPaste**, default **OnlyCopy**).

[mouse]

hide_when_typing (default **false**) · **bindings** (array; see [keyboard], action adds **ExpandSelection**).

[hints]

alphabet (default "jfkdl;ahguriewp") · **enabled** [{**regex**, **hyperlinks**, **post_processing**, **persist**, **action**, **command**, **binding**, **mouse**}].

[keyboard]

bindings [{**key**, **mods**, **mode**, **action** | **chars** | **command**}].

[debug]

render_timer · **persistent_logging** · **log_level** (**Off**...**Trace**, default **Warn**) · **renderer** (**glsl3**/**gles2**/**gles2pure**/**None**) · **print_events** · **highlight_damage** · **prefer_egl**.

Restart-only fields

Most fields reload live (Chapter 7), but a few are read once at window creation and take effect only on a new window or a restart. Editing them in the Settings tab saves the value; it applies when the window is next created. The restart-only set:

Field	Why it is read once
<code>window.dimensions</code>	the initial grid size is set when the window is built
<code>window.startup_mode</code>	windowed/maximized/fullscreen is a creation-time choice
<code>window.position</code>	the startup position is applied as the window opens
<code>window.class</code>	the X11/Wayland window class is fixed at creation

Everything else in the tables above reloads live when `general.live_config_reload` is on: fonts, colours, cursor, scrolling, bell, selection, hints, keybindings, and the debug flags all apply to running windows the moment the file is saved. The `zt msg config` path (Chapter 31) and the `-o` override (Chapter 7) reach the same live-reload machinery for the non-restart fields; the restart-only fields above are the exceptions to keep in mind when a change “does nothing” (Appendix G).

The zterminal additions, gathered

The fields this product adds on top of the inherited schema, marked **(zt)** throughout the tables, gathered in one place:

Field	Effect	Chapter
<code>window.background_image</code>	a GPU image drawn behind the cells	25
<code>window.background_image_opacity</code>	how strongly it shows	25
<code>window.overlay_opacity</code>	alpha of the glassy control-panel overlays	18

The rest of the product’s additions live outside `zterminal.toml` — the profiles, layouts, snapshots, triggers, snippets, recent-dirs, and GUI preferences are their own JSON files under `~/.zterminal` (Appendix E), managed by the control panel rather than the config schema. The three fields above are the only extension surface that is a `zterminal.toml` key; everything else the extension layer stores is a managed file, which is why the config reference is otherwise the inherited schema documented faithfully.

Appendix C – Default Keybindings

The defaults from `zterminal-bindings(5)`. `~Vi/~Search` mean the mode must *not* be active. Vi-mode and search-mode bindings are in Chapter 17.

General

`Paste` → Paste · `Copy` → Copy · `Paste` (Vi|Search) → Paste · `Ctrl+L` → ClearLogNotice · `Ctrl+L` (Vi|Search) → clear screen.

Scrolling

`Shift+PageUp/PageDown` (~Alt) → ScrollPageUp/Down · `Shift+Home/End` (~Alt) → ScrollToTop/Bottom.

Mouse

`Right` → ExpandSelection · `Right+Control` → ExpandSelection · `Middle` (~Vi) → PasteSelection.

Windows / Linux / BSD

`Ctrl+Shift+V` → Paste · `Ctrl+Shift+C` → Copy · `Ctrl+Shift+F/B` (~Search) → SearchForward/Backward · `Shift+Insert` → PasteSelection · `Ctrl+0` → ResetFontSize · `Ctrl+=/+/NumpadAdd` → IncreaseFontSize · `Ctrl+-/NumpadSubtract` → DecreaseFontSize. Windows only: `Alt+Enter` → ToggleFullscreen.

macOS

`Cmd+K` → clear screen + ClearHistory · `Cmd+0/=/-` → font reset/up/down · `Cmd+V` → Paste · `Cmd+C` → Copy · `Cmd+H` → Hide · `Cmd+Alt+H` → HideOtherApplications · `Cmd+M` → Minimize · `Cmd+Q/Cmd+W` → Quit · `Cmd+N` → CreateNewWindow · `Cmd+T` → CreateNewTab · `Cmd+Ctrl+F` → ToggleFullscreen · `Cmd+F/B` (~Search) → SearchForward/Backward · `Cmd+Shift+] / [` → Next/ Previous Tab · `Cmd+Tab/Cmd+Shift+Tab` → Next/Previous Tab · `Cmd+1–8` → SelectTab1–8 · `Cmd+9` → SelectLastTab.

The MenkeTechnologies extension shortcuts

The extension layer’s default shortcuts, gathered from Chapters 18–30. Every one is also a command-palette entry and, on macOS, a menu item with the same key-equivalent (Chapter 18), so the table is a convenience rather than something to memorize. macOS accelerators are shown; Linux/Windows route the same actions through `^␣` variants or the palette per the non-collision rule of Chapter 15.

Shortcut	Action	Chapter
⌘,	open the control panel / Settings	18–19
⌘D	open the Dashboard	21
⌘K	open the command palette	22
⌘R	open the shell-history palette	22
⌘↑ / ⌘↓	jump to previous / next prompt (OSC 133)	22, 26
⌘⇧O	select the last command's output	22, 26
⌘E	MRU quick-switch to the last-used tab	24
⌘⇧E	open Exposé (native + tmux panes)	24
⌘⇧P	cross-pane scrollback search	29
⌘⇧B	cross-session broadcast overlay	29
⌘⇧S	save / restore tmux sessions	27
⌘⇧`	toggle the Quake dropdown	24
⌘D / ⌘⇧D	split the pane vertically / horizontally	23
⌘] / ⌘[	focus next / previous pane	23
⌘⇧←↑↓→	focus the pane in a direction	23
⌘⇧←↑↓→	move (swap) the focused pane	23
⌘⇧←↑↓→	resize along the split	23
⌘⇧↔	zoom a pane to fill, toggle back	23
⌘⇧W	close the focused pane	23
⌘⇧L	save / restore a native split layout	23

Appendix D – Escape Sequence Support

The matrix from `zterminal-escapes(7)`. Statuses: IMPLEMENTED, PARTIAL, REJECTED (Chapter 6).

ESC codes

`ESC ( ) * +` (charsets B and O only), `ESC => 7 8 # 8 D E H M Z` — all IMPLEMENTED.

CSI

`CSI ` @ A/a B/b C/c D/d E/e F/f G/g H I J K L M n P $ p ? $ p SP q r S s T u ? u = u < u > u X Z` — IMPLEMENTED. `CSI m` — IMPLEMENTED for SGR `0–9, 21–25, 27–49, 58, 59, 90–97, 100–107`; REJECTED for `11–19, 51–55`. `CSI h/l` — PARTIAL (modes 4, 20). `CSI ? h/? l` — PARTIAL (modes 1, 3, 6, 7, 12, 25, 1000, 1002, 1004, 1005, 1006, 1007, 1042, 1049, 2004, 2026). `CSI t` — PARTIAL (22, 23); REJECTED (1–13, 15, 19–21, 24).

OSC

`OSC 0` (no icon names) `2 4 8 10 11 12 50` (CursorShape only) `52` (clipboard + primary only) `104 110 111 112` — IMPLEMENTED. `OSC 1` — REJECTED (icon names unsupported).

DCS

`DCS = s` — REJECTED (`CSI ? 2026 h/l` is used instead).

ESC codes, described

Sequence	Meaning
<code>ESC 7 / ESC 8</code>	save / restore cursor and pen (Chapter 3)
<code>ESC D / ESC M</code>	index / reverse index (move down / up, scrolling at margins)
<code>ESC E</code>	next line (carriage return + index)
<code>ESC H</code>	set a tab stop at the cursor column
<code>ESC = / ESC ></code>	keypad application / numeric mode
<code>ESC # 8</code>	DEC alignment test (fill the screen with <code>E</code>)

Sequence	Meaning
<code>ESC () * +</code>	designate character set (B and O only)
<code>ESC Z</code>	identify terminal (Device Attributes)

CSI, described

Sequence	Meaning
<code>CSI A–H, a–f</code>	cursor motion (up/down/forward/back, position)
<code>CSI J / CSI K</code>	erase in display / in line
<code>CSI X</code>	erase N characters in place
<code>CSI L / CSI M</code>	insert / delete lines
<code>CSI P / CSI @</code>	delete / insert characters
<code>CSI S / CSI T</code>	scroll up / down
<code>CSI m</code>	SGR — set colour and attributes
<code>CSI n</code>	device status report
<code>CSI g</code>	clear tab stop(s)
<code>CSI r</code>	set the scroll region (top; bottom)
<code>CSI s / CSI SP q</code>	save cursor / set cursor style
<code>CSI u, ? u, = u, < u, > u</code>	Kitty keyboard protocol (Chapters 15, 37)
<code>CSI ? h / CSI ? l</code>	set / reset a private mode (see below)

Private modes, described

The `CSI ? h / CSI ? l` modes zterminal handles, and what each does — the set an application toggles during its startup handshake (Chapter 6):

Mode	Meaning
1	application cursor keys (arrows send SS3)
3	132-column mode
6	origin mode (address relative to the scroll region)
7	autowrap at the last column
12	cursor blink
25	cursor visibility
1000 / 1002	mouse reporting: clicks / clicks + drag
1004	focus reporting
1005 / 1006	mouse encoding: UTF-8 / SGR
1007	alternate-scroll (wheel sends arrows on the alt screen)
1042	urgency-hint bell

Mode	Meaning
1049	alternate screen (save/restore the primary)
2004	bracketed paste (Chapter 35)
2026	synchronized output (Chapters 5, 6)

OSC, described

Sequence	Meaning
<code>OSC 0 / OSC 2</code>	set the window title (icon-name form ignored)
<code>OSC 4</code>	set / query a palette colour
<code>OSC 8</code>	hyperlink (Chapter 14)
<code>OSC 10 / 11 / 12</code>	set foreground / background / cursor colour
<code>OSC 50</code>	set the cursor shape
<code>OSC 52</code>	clipboard access, gated by <code>terminal.osc52</code> (Chapters 13, 36)
<code>OSC 104 / 110 / 111 / 112</code>	reset palette / fg / bg / cursor colour

Appendix E — The ~/.zterminal Directory

All zterminal state and logs live under ~/.zterminal. The files and what writes them:

- `zterminal.toml` — the canonical config (Chapter 7); edited by hand or by the Settings tab.
- `zterminal.log` — the runtime log; level set by `debug.log_level`, kept on exit if `debug.persistent_logging` is on; shown in the Logs tab.
- `layouts/<name>.json` — saved native split layouts (Chapter 23).
- `snapshots/<name>.json` and `<name>.contents/` — saved tmux session snapshots and their captured pane scrollback (Chapter 27).
- `recent_dirs.json` — the harvested recent-directories list (Chapter 30).
- `triggers.json` — per-pane output triggers (Chapter 30).
- plus the saved snippets, profiles, and GUI preferences (colour scheme, effects, light/dark, `tmux_restore_processes` whitelist) the control panel manages.

As a tree, the whole state directory:

```
~/.zterminal/
├── zterminal.toml           the canonical config (Chapter 7)
├── zterminal.log           the runtime log (Chapter 16)
├── layouts/
│   └── <name>.json         native split layouts (Chapter 23)
├── snapshots/
│   ├── <name>.json         tmux session snapshots (Chapter 27)
│   └── <name>.contents/    captured pane scrollback (Chapter 27)
├── recent_dirs.json        harvested working directories (Chapter 30)
├── triggers.json           per-pane output triggers (Chapter 30)
└── (snippets, profiles, GUI preferences the panel manages)
```

The organizing principle is that everything zterminal persists is a plain file under one directory you own — no scattered dotfiles, no registry, no hidden state elsewhere. Back up ~/.zterminal and you have backed up the whole environment: the config, the saved layouts and sessions, the harvested lists, the profiles. The formats are TOML for the config and JSON for the managed state, both text, both diff-able, both versioned so an older file upgrades cleanly (Chapter 7's migration discipline applies to the config; the JSON managers tolerate older shapes).

On Windows the config lives at %APPDATA%\zterminal\zterminal.toml.

Appendix F — Glossary

Alternate screen — the non-scrolling secondary grid full-screen TUIs paint on; entered via private mode 1049. (Chapter 3.)

Cell — one grid position: a character plus its colours and attributes. (Chapter 3.)

Block selection — a rectangular selection of a column range across a row range, for lifting one column out of tabular output. (Chapter 35.)

Bracketed paste — private mode 2004; wraps pasted text so a program can tell it from typed input and not execute it immediately. (Chapter 35.)

CSI — Control Sequence Introducer, `ESC [`; the largest escape family. (Chapters 3, 6.)

Damage — the cells changed since the last frame; the renderer redraws only damage. (Chapter 4.)

Frame coalescing — deferring the draw so all damage since the last frame is drawn once at the display's refresh cadence. (Chapter 5.)

Glyph atlas — the GPU texture holding each rasterized glyph once, so drawing a cell is drawing a textured quad. (Chapter 4.)

Grapheme cluster — a base scalar plus its combining marks or joiners, stored against one cell (or a wide cell and its spacer). (Chapters 3, 10.)

Hint — a regex- or hyperlink-matched on-screen target that can be opened, copied, selected, or piped. (Chapter 14.)

`include_dir!` — the macro that embeds the control-panel frontend into the binary. (Chapter 18.)

`img` — the framing of tmux's wire protocol (version 8) the tmux client speaks. (Chapter 27.)

Ligature — a single glyph substituted for a multi-character sequence; not performed by the per-cell grid renderer, by design. (Chapter 10.)

OSC — Operating System Command, `ESC [`; titles, palette, hyperlinks, clipboard. (Chapters 3, 6.)

OSC 133 — the FinalTerm/iTerm2 semantic-prompt marks for prompt and output boundaries. (Chapter 26.)

Profile — a named snapshot of the whole config, GUI look, and tmux server state. (Chapter 20.)

PTY — pseudo-terminal; the kernel object pairing the emulator with the shell. (Chapters 1, 23.)

Scanner — the single pre-`vte` stage that peels image and OSC 133 sequences out of the PTY stream. (Chapters 25, 26.)

Scale factor — the monitor's DPI multiplier the window manager reports, used to turn point-based font metrics into pixels. (Chapter 9.)

Scroll region — the top/bottom margins (`CSI r`) within which scrolling happens, leaving rows outside them fixed. (Chapter 3.)

Scrollback — the buffer of lines scrolled off the primary screen; sized by `scrolling.history`. (Chapters 3, 12.)

SGR — Select Graphic Rendition, `CSI m`; sets colour and attributes. (Chapters 3, 6.)

Split tree — zterminal's native i3-style binary tree of panes, one PTY each. (Chapter 23.)

Synchronized output — private mode 2026; the application asks the terminal to hold the frame until a repaint is complete, so it lands atomically. (Chapters 5, 6.)

vi mode — the keyboard-driven scrollbar-navigation and selection mode. (Chapter 17.)

vte — the VT parser library the core uses. (Chapters 3, 4.)

Wire protocol — tmux's client/server imsg protocol the client speaks directly. (Chapter 27.)

zgui-core — the shared component library the control panel is built from. (Chapter 18.)

zterminal:// — the custom scheme the in-process WebView is served over. (Chapter 18.)

zt msg — the IPC command for controlling a running instance. (Chapter 31.)

Appendix G – Troubleshooting

A terminal has a small number of failure modes, and nearly every one of them has a named cause somewhere in this book. This appendix is the index from symptom to cause: what you see, why it happens, and which chapter's mechanism is responsible. Nothing here is a workaround for a bug; each entry is a place where the terminal's documented behaviour meets an environment that has not been told about it.

Colours are wrong or missing over SSH

The symptom is a remote application that renders in eight colours, or with no colour, or that errors with “terminal not fully functional.” The cause is almost always terminfo (Chapter 37): the remote host has never heard of `TERM=zterminal`, so ncurses cannot find the capability entry and either falls back badly or refuses. The two honest fixes are the ones Chapter 37 states — compile the entry onto the remote once with `infocmp zterminal | ssh host tic -x -`, or set `TERM=xterm-256color` for hosts you will not revisit and accept the common-denominator capabilities. If colours are wrong *locally*, the cause is different: check whether the application is emitting truecolor SGR that a 256-colour theme is quantizing (Chapter 3), or whether `draw_bold_text_with_bright_colors` (Chapter 11) is coupling bold to the bright palette in a way a colour scheme did not expect.

Arrow keys or function keys send the wrong bytes

The symptom is arrows that print `^[` in the shell, or an editor that does not recognize a modified key. The usual cause is application-cursor-keys mode (private mode 1, Chapter 6): an application that set it and exited without resetting it leaves the terminal sending SS3-prefixed arrows the shell does not expect. Resetting the terminal (`reset`, or `tput rmkx`) clears it. For modified keys that collapse together — `Ctrl+I` indistinguishable from `Tab`, `Ctrl+M` from `Enter` — the cause is the legacy keyboard encoding's ambiguity, and the fix is an application that negotiates the Kitty keyboard protocol (`CSI u`, Chapters 6, 15, 37), which zterminal implements; an application that does not ask for it gets the legacy encoding by design.

Box drawing has gaps, or icons are missing

If a TUI's borders show hairline gaps, the text font's box-drawing glyphs are the culprit and `font.builtin_box_drawing` (Chapters 4, 10) is the fix — it defaults on, so a gap usually means it was turned off. If powerline separators or Nerd Font icons render as tofu boxes, the primary face lacks them and the fallback chain (Chapter 9) has nothing to fall back to — install a Nerd Font, or let first-run seeding pick one (Chapter 7). If an emoji or a composed glyph is missing while its components render, that is the grapheme story of Chapter 10: the layout is correct and stable, but the fallback font has no glyph for the composed cluster, so it decomposes rather than shifting the grid.

Images do not display

If an inline image prints as a wall of base64 or simply nothing, walk the causes in order. Inside tmux, the image protocols require `set -g allow-passthrough on` (Chapters 25, 36) — off by default in tmux for the security reason Chapter 36 gives — and without it tmux swallows the sequences. Outside tmux, confirm the

tool is sending a protocol `zterminal` decodes (Kitty, Sixel, or iTerm2, Chapter 25) and, ideally, that it queried support first (`a=q`, Chapter 37) rather than assuming. The bundled `zterminal-icat` (Chapter 25) sidesteps both problems by enabling passthrough itself and placing via Unicode placeholders that survive a tmux redraw; if a plain display escape vanishes on the next repaint inside tmux, that placeholder behaviour is exactly what is missing.

Prompt jumping and failed-command marks do nothing

The `⌘↑/⌘↓` prompt jumps, the `⌘⇧0` output selection, the gutter marks, and the command history all depend on OSC 133 marks that the shell must emit (Chapter 26). If they do nothing, the shell integration snippet is not sourced — add the `source ../zterminal-shell-integration.<shell>` line from `extra/shell-integration/` — and inside tmux the same `allow-passthrough on` the images need (Chapter 36) is required for the marks to survive the multiplexer. The marks are zero-width and harmless in other terminals, so there is no reason to guard the source line per terminal (Chapter 26).

A configuration change did nothing

If editing `zterminal.toml` had no effect, three causes account for nearly all of it. The change may be to a restart-only field — `window.dimensions`, `window.startup_mode`, and the few others marked in Appendix B are read once at window creation (Chapter 7). Or `general.live_config_reload` may be off, in which case nothing reloads until restart (Chapter 7). Or the file being edited is not the file being read: the resolution order of Chapter 7 picks the first match, so a stale `$XDG_CONFIG_HOME` copy or a `--config-file` override can shadow the canonical `~/.zterminal/zterminal.toml` the Settings tab reports it is editing (Chapter 19). `zt msg get-config` (Chapter 31) reads back the runtime config so you can confirm what the running instance actually believes.

The window is opaque when transparency was configured

`window.opacity` below `1.0` with no visible transparency usually means the compositor is the obstacle, not the config. On Linux, EGL and transparency interact — `debug.prefer_egl` (Chapter 16) notes transparency may not work under EGL on Linux/BSD — and a compositor that is not running or not compositing this window shows a black background instead of the desktop behind it. `window.blur` (Chapter 8) additionally needs a compositor that honours the blur request (macOS, KDE Wayland); elsewhere it is silently ignored. If only *some* cells are transparent, that is `colors.transparent_background_colors` (Chapter 11) being off, which limits opacity to the default background rather than every cell.

Diagnosing anything else: the debug surface

When a symptom is not in this list, the `[debug]` table (Chapter 16) is the instrument panel. `debug.log_level = "Debug"` (or `-vvv` on the command line, optionally with `ZTERM_EXTRA_LOG_TARGETS` to include `winit` or `vte`) writes a verbose trace to `~/.zterminal/zterminal.log`, which the Logs tab (Chapter 18) renders live. `debug.highlight_damage` (Chapters 5, 16) tints what the renderer actually redraws, which diagnoses “the screen looks wrong” problems by showing which cells the terminal thinks changed. `debug.render_timer` (Chapters 5, 16) overlays per-frame draw time for “it feels slow” problems. `debug.renderer` forces a specific backend (`glsl3`, `gles2`, `gles2pure`) when the automatic choice misbehaves on a particular GPU or driver. And `--print-events` logs the raw window events when input itself is suspect. None of these is stable API (Chapter 16), but together they turn “something is wrong” into a specific, locatable fact — which is the same disposition the whole book takes toward the program.

Appendix H – Worked Configurations

The field-by-field walk of Chapters 7–16 and the reference of Appendix B describe every setting in isolation. This appendix assembles them into complete, realistic `zterminal.toml` files for common working styles. Each is a file you could drop into `~/.zterminal/` and use; each is annotated with the chapter that explains the choice. None uses a field not documented elsewhere in the book.

A minimal, fast config

The smallest useful file — a good starting point that changes only what most people change first:

```
[font]
normal = { family = "JetBrainsMono Nerd Font", style = "Regular" }
size = 13.0

[scrolling]
history = 20000

[cursor]
style = { shape = "Block", blinking = "Off" }
```

Everything else is the default of Chapters 8–16: the base16 palette, full window decorations, the OnlyCopy OSC 52 posture (Chapter 36), live reload on. This is the “I just want a good terminal” file.

A translucent, themed config

For a desktop with a compositor, leaning on the renderer’s compositing pass (Chapter 4):

```
[general]
import = ["~/zterminal/theme.toml"]      # the full [colors] block (Appendix)

[window]
opacity = 0.88
blur = true                               # macOS / KDE Wayland only (Chapter 8)
padding = { x = 10, y = 10 }
dynamic_padding = true
decorations = "Buttonless"               # macOS (Chapter 8)

[colors]
transparent_background_colors = true      # opacity on every cell bg (Chapter 11)

[font]
normal = { family = "FiraCode Nerd Font", style = "Regular" }
size = 12.5
offset = { x = 0, y = 2 }                # a little line spacing (Chapter 9)

[bell]
animation = "EaseOutExpo"                 # visual bell instead of audible
duration = 120
color = "#f4b755"
```

The `transparent_background_colors` line is the one people miss: without it, `opacity` affects only the default background, so a cell with an explicit background colour stays opaque and the translucency looks patchy (Chapter 11). The visual bell replaces the audible one — `bell.duration` above zero enables it, and the easing curve is cosmetic (Chapter 12).

A tiling-heavy, keyboard-driven config

For a user who lives in native splits (Chapter 23) and drives everything from the keyboard, adding custom hints (Chapter 14) and a couple of bindings:

```
[window]
option_as_alt = "Both"                   # free the Option key for Alt bindings

[mouse]
hide_when_typing = true

[[hints.enabled]]
regex = "(https?|ftp|file)://[^\u0000-\u001F\u007F-\u009F<>\"\\s{-}\\^\\`']+)"
action = "Copy"
binding = { key = "U", mods = "Control|Shift" }

[[hints.enabled]]
regex = "[0-9a-f]{7,40}"                  # a git hash - copy on Ctrl+Shift+G
action = "Copy"
binding = { key = "G", mods = "Control|Shift" }

[[hints.enabled]]
regex = "(/[^\s]+)"                       # a filesystem path - open in $EDITOR
command = { program = "sh", args = ["-c", "$EDITOR \"$1\"", "sh"] }
binding = { key = "E", mods = "Control|Shift" }

[keyboard]
bindings = [
  { key = "Return", mods = "Command|Shift", action = "ToggleViMode" },
]
```

The third hint is the general pattern worth stealing: a `command` hint receives the match as its final argument (Chapter 14), so wrapping it in `sh -c '... "$1"' sh` lets you do anything with the matched text — open it, grep it, feed it to a tool. Nothing here needs the GUI, but the Hints tab (Chapter 14) edits this exact `[hints]` table if you would rather click than type.

A remote-work config

For a machine used mainly to SSH elsewhere, tuned for the terminfo and clipboard realities of Chapters 36–37:

```
[env]
TERM = "zterminal"                # local; copy the entry to remotes too

[terminal]
osc52 = "CopyPaste"              # allow remote clipboard both ways...

[selection]
save_to_clipboard = true

[scrolling]
history = 100000                 # the maximum (Chapter 12)
```

The one deliberate risk here is `osc52 = "CopyPaste"`, which opens clipboard *reading* to remote applications (Chapter 36). It is convenient when you trust your hosts and want a remote editor to paste from your local clipboard; it is a choice to make eyes-open, because the default withholds read for the exfiltration reason Chapter 36 gives. Everything else on this file is about surviving the SSH hop: advertise `zterminal` locally, keep a deep scrollback for long remote sessions, and mirror selections to the clipboard so a copy on either side is available everywhere.

Appendix I – The Escape Sequence Cookbook

Chapter 6 and Appendix D catalogue what zterminal answers; this appendix shows the answers in use. Each recipe is the actual byte sequence for a common operation, written with `ESC` for the escape byte (`0x1B`) and `ST` for the string terminator (`ESC \`). These are the sequences your shell, editor, and prompt already emit — seeing them by hand demystifies the byte stream the grid consumes. You can try any of them with `printf`; where noted, a sequence expects a reply.

Cursor and screen

<code>ESC [H</code>	move the cursor to the top-left (row 1, col 1)
<code>ESC [10 ; 40 H</code>	move to row 10, column 40
<code>ESC [2 J</code>	erase the whole screen
<code>ESC [K</code>	erase from the cursor to the end of the line
<code>ESC [s</code>	save the cursor position; <code>ESC [u</code> restores it
<code>ESC 7 ... ESC 8</code>	save/restore cursor AND pen+charset (Chapter 3)
<code>ESC [3 J</code>	erase the scrollbar as well as the screen

Colour and attributes (SGR)

<code>ESC [1 m</code>	bold on
<code>ESC [3 m</code>	italic on
<code>ESC [4 m</code>	underline on
<code>ESC [31 m</code>	foreground = ANSI red (palette index 1)
<code>ESC [91 m</code>	foreground = bright red (index 9)
<code>ESC [38 ; 5 ; 208 m</code>	foreground = 256-palette index 208
<code>ESC [38 ; 2 ; 0 ; 175 ; 255 m</code>	foreground = truecolor #00aaff
<code>ESC [48 ; 2 ; 24 ; 24 ; 24 m</code>	background = truecolor #181818
<code>ESC [0 m</code>	reset all attributes and colours

The parameters accepted here are exactly those in the `CSI m` partials of Appendix D — `38;5;n` and `38;2;r;g;b` for foreground, `48;...` for background — and a parameter outside the accepted set (say, `53` for overline) is ignored rather than half-answered, per Chapter 6's REJECTED discipline.

The alternate screen

<code>ESC [? 1049 h</code>	switch to the alternate screen, saving the primary
...full-screen app draws here...	
<code>ESC [? 1049 l</code>	switch back, restoring the primary exactly

This is the round trip every TUI performs (Chapter 6). It is why `vim` leaves your scrollbar untouched when it exits: it painted on the alternate screen the whole time, and switching back restored the primary screen and cursor.

Titles, hyperlinks, and the clipboard

```
ESC ] 0 ; my session BEL          set the window/icon title
ESC ] 8 ; ; https://example.com ST begin a hyperlink...
click me
ESC ] 8 ; ; ST                    ...end the hyperlink (Chapter 14)
ESC ] 52 ; c ; SGVsbG8= ST        set the clipboard to base64 "Hello" (Chapter 36)
```

BEL (**0x07**) or **ST** terminates an OSC string. The hyperlink form (**OSC 8**) wraps the visible text between a begin with a URI and an empty end; zterminal's hints and the platform opener handle the click (Chapters 14, 36). The **OSC 52** clipboard write succeeds under the default **OnlyCopy** posture; a *read* request would be refused unless **terminal.osc52** is widened (Chapter 36).

Sequences that expect a reply

```
ESC [ c          Device Attributes – the terminal replies with its type
ESC [ 6 n        Cursor Position Report – replies ESC [ row ; col R
ESC [ ? 2026 h ... ESC [ ? 2026 l   synchronized output: hold the frame between
ESC [ > u        push a Kitty keyboard flag; ESC [ ? u queries the mode
```

The reply-bearing sequences are the runtime capability queries of Chapter 37: an application sends one and adapts to the answer rather than trusting **TERM**. The Cursor Position Report is also the classic hazard of Chapter 36 — a sequence that returns attacker-influenceable data — which is why a terminal reports only what it legitimately knows and never echoes planted text back onto the command line.

Appendix J – Environment Variables

zterminal reads and sets a handful of environment variables, scattered through the chapters where they matter. Gathered here with what each does and which chapter covers it, both the ones the terminal *reads* to configure itself and the ones it *sets* for the programs it spawns.

Read by zterminal

Variable	Effect	Chapter
<code>SHELL</code>	the shell to spawn when <code>terminal.shell</code> is unset	13
<code>TERM</code>	the terminfo entry an application consults; set it to <code>zterminal</code> where the entry is installed, else falls back to <code>xterm-256color</code>	37
<code>TMUX</code>	resolves the tmux server socket, pid, and session for the wire client	27
<code>XDG_CONFIG_HOME</code>	part of the config resolution order	7
<code>ZTERM_EXTRA_LOG_TARGETS</code>	adds other libraries (<code>winit</code> , <code>vte</code>) to the log	16
<code>WINIT_X11_SCALE_FACTOR</code>	overrides the reported DPI scale on X11	9
<code>HISTFILE</code>	read from the focused shell's process env by the history palette	22

Set by zterminal for spawned processes

Variable	Effect	Chapter
<code>ZTERM_WINDOW_ID</code>	identifies the window, the default target for <code>zt msg config</code>	28, 31
<code>ZT_TRIGGER_TEXT</code>	the matched text, in a trigger command's environment	30
<code>ZT_TRIGGER_NAME</code>	the trigger's name, in its command's environment	30
anything in <code>[env]</code>	injected into every spawned process, including the shell	7

Two of these repay a second look. `HISTFILE` is not read from zterminal's own environment but from the *focused child shell's* live process environment (Chapter 22) — which is what lets the history palette open the actual history file of the shell you are looking at, in the format that shell uses, rather than guessing a default path. And `ZTERM_WINDOW_ID` is the seam that makes `zt msg config -w "$ZTERM_WINDOW_ID"` (Chapter

31) able to retheme the specific window a script is running in — the terminal tells each child which window it lives in, and the child can address that window back over the IPC socket. The `[env]` table is the general mechanism for the rest: anything you put there is exported into every process the terminal launches, and the Env Vars manager (Chapter 30) both writes it and hot-injects it into shells already running.